LONDON METROPOLITAN UNIVERSITY

ITAHARI INTERNATIONAL COLLEGE

CS5054NT Advanced Programming and Technologies

50% Group Coursework

Submission: Final Submission

Autumn 2025/26

Project Title: RojgarSetu

Name	London Met ID
Ayush Chaudhari (Leader)	24045834
Manoj Katuwal	24045922
Kristina Gurung	24045908
Biman Hang Limbu	24045848
Kushi Limbu	24045902

Team Name: Java Beans

Submitted to: Sujan Subedi

Submission Due Date: 2026-05-21

Submission Date: 2026-05-21

GitHub link: https://github.com/manojctrl/RojgarSetu.git

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before deadline for my assignment to be accepted and marked. I am fully aware that late submission will be treated as non-submission and mark of zero will be awarded.

Table of Contents

Table of Figures

Table of Tables

1 Introduction

The **RojgarSetu** online job portal has been designed in order to bridge the gap between job seekers and employers within Nepal. This web application is specifically for use in urban and semi-urban areas of Nepal where there is an increasing need for employment but such need is not organized in any way. Despite the fact that many businesses are growing and creating jobs, there remains no efficient means by which to link employers with potential candidates. Due to this problem, job seekers end up relying on unconventional means to find employment such as social media posts and word of mouth. For their part, employers are unable to locate candidates who possess the required skill and integrity, thereby delaying the process.

In order to tackle such challenges, RojgarSetu offers a user-friendly online platform wherein employers can upload authentic vacancies while the job aspirants can search and filter out suitable jobs for themselves through a very convenient interface. The whole process becomes much more effective and easygoing owing to minimal dependency on third parties. It becomes easier for job seekers to gain access to all kinds of authentic and up-to-date vacancies, since the information about jobs is centralized. Moreover, with an emphasis on finding local job opportunities, the platform enables people to stay in their own area and earn a livelihood there.

1.1 Purpose

RojgarSetu is designed to develop a web-based portal that makes it easier for potential employees and employers to interact within the locality. The portal gives an employer the ability to post jobs, while it enables the employee to browse for a job and apply to one, without having to use any intermediary services or any external informal medium. With all job-related information being available at one place, the user gets easy access to valid and accurate information.

In addition to making the recruitment process easier for both parties, RojgarSetu provides an easier way to track applications and responses of applicants. An applicant will be able to see whether his/her application has been considered, while an employer will be able to check out the applications for his/her postings. The project will also contribute positively to local employment and development as well as cut down the necessity of migration.

1.2 Audience

The RojgarSetu system is designed for job seekers, employers, and admin. Job seekers comprise students, graduates, and skilled people looking for jobs in their respective locations and requiring a convenient way to search and apply for jobs. Employers comprise companies, firms, and other organizations that want to post job vacancies, review applications, and hire employees effectively. The admin controls the system by validating users, posting jobs, tracking activities, and managing the operations of the system. The system may also benefit local government institutions and policymakers, as it gives employment-related data and analysis that could help in decision making.

1.3 Aim and Objectives

Aim:

RojgarSetu aims at linking the people in the Koshi Province to jobs within the region so that they can more easily find employment.

Objectives:

- RojgarSetu hopes to create a site through which people can look to get jobs and businesses can post advertisements of job opportunities.
- RojgarSetu will ensure that people can easily apply to get jobs.
- RojgarSetu desires to decrease the number of individuals who need to seek employment in locations or transfer to a different city.
- RojgarSetu would like to eliminate the individuals in the middle who collect a fee to match employees with employers.
- RojgarSetu is interested in gathering data on the jobs that can assist people in making decisions.
- RojgarSetu is interested in raising the number of individuals with employment in the region.

1.4 Features List

There are varied features provided by RojgarSetu that serve various purposes of jobseekers, employers, and the admin to ensure a smooth and easy job process.

Features for Job Seekers

- Registration and logging in.
- Personal information setup (Education, Skills, Experience).
- Job search options based on category, location, and salary.
- Reviewing full job descriptions before applying.
- Job application.
- Application status tracking (pending, shortlisted, rejection, and hiring).
- Saving jobs for future reference.

Features for Employers

- Registration and logging in.
- Setting up company profile.
- Posting vacancies for jobs.
- Updating and editing job posts.
- Applicants view for job posts.
- Candidate shortlisting and hiring status updates.

Admin Features

- Admin login facility.
- Employer registration verification.
- Users management (jobseekers and employers).
- Updating jobs post statuses (approve/remove jobs).
- Monitoring system activities.
- Report generation for jobs and applications.

2 Database and Development

2.1 Identification of Entities and attributes.

1. USERS TABLE

Table 1 User Table

S. N	Attributes	Constraints
1	user_id	Primary Key
2	full_name	-
3	email	Unique
4	phone	-
5	password	-
6	role	-
7	status	-
8	location	-
9	dob	-
10	created_at	-

2. SEEKER_PROFILE TABLE

Table 2 Seeker Table

S. N	Attributes	Constraints
1	seeker_id	Primary Key
2	user_id	Foreign Key
3	address_city	-
4	skills	-
5	education	-
6	experience_year	-
7	resume_path	-

3. EMPLOYER_PROFILE TABLE

Table 3 Employer

S. N	Attributes	Constraints
1	employer_id	Primary Key
2	user_id	Foreign Key
3	company_name	-
4	company_address	-
5	company_category	-
6	company_city	-

7	company_description	-
8	contact_person	-
9	verified_at	-
10	verified_by	-

4. JOBS TABLE

Table 4 Jobs Table

S. N	Attributes	Constraints
1	job_id	Primary Key
2	employer_id	Foreign Key
3	title	-
4	description	-
5	category	-
6	location_city	-
7	salary_range	-
8	job_type	-
9	posted_at	-
10	deadline	-
11	status	-
12	approved_at	-
13	approved_by	-

5. APPLICATIONS TABLE

Table 5 Application Table

S. N	Attributes	Constraints
1	application_id	Primary Key
2	job_id	Foreign Key
3	seeker_id	Foreign Key
4	cover_letter	-
5	resume_path	-
6	status	-
7	applied_at	-
8	reviewed_at	-

6. SAVED_JOBS TABLE

S. N	Attributes	Constraints
1	saved_jobs_id	Primary Key
2	seeker_id	Foreign Key
3	job_id	Foreign Key
4	saved_at	-
5	(seeker_id, job_id)	Unique Key

2.2 Business Rule for the RojgarSetu System

- A user can have one seeker profile.
- A user can have one employer profile.
- An employer can post multiple jobs.
- Each job belongs to one employer.
- A seeker can apply for multiple jobs.
- Each job can receive multiple applications.
- Each application belongs to one seeker and one job.
- A seeker can save multiple jobs.
- A job can be saved by multiple seekers.

2.3 Entity Relationship Diagram

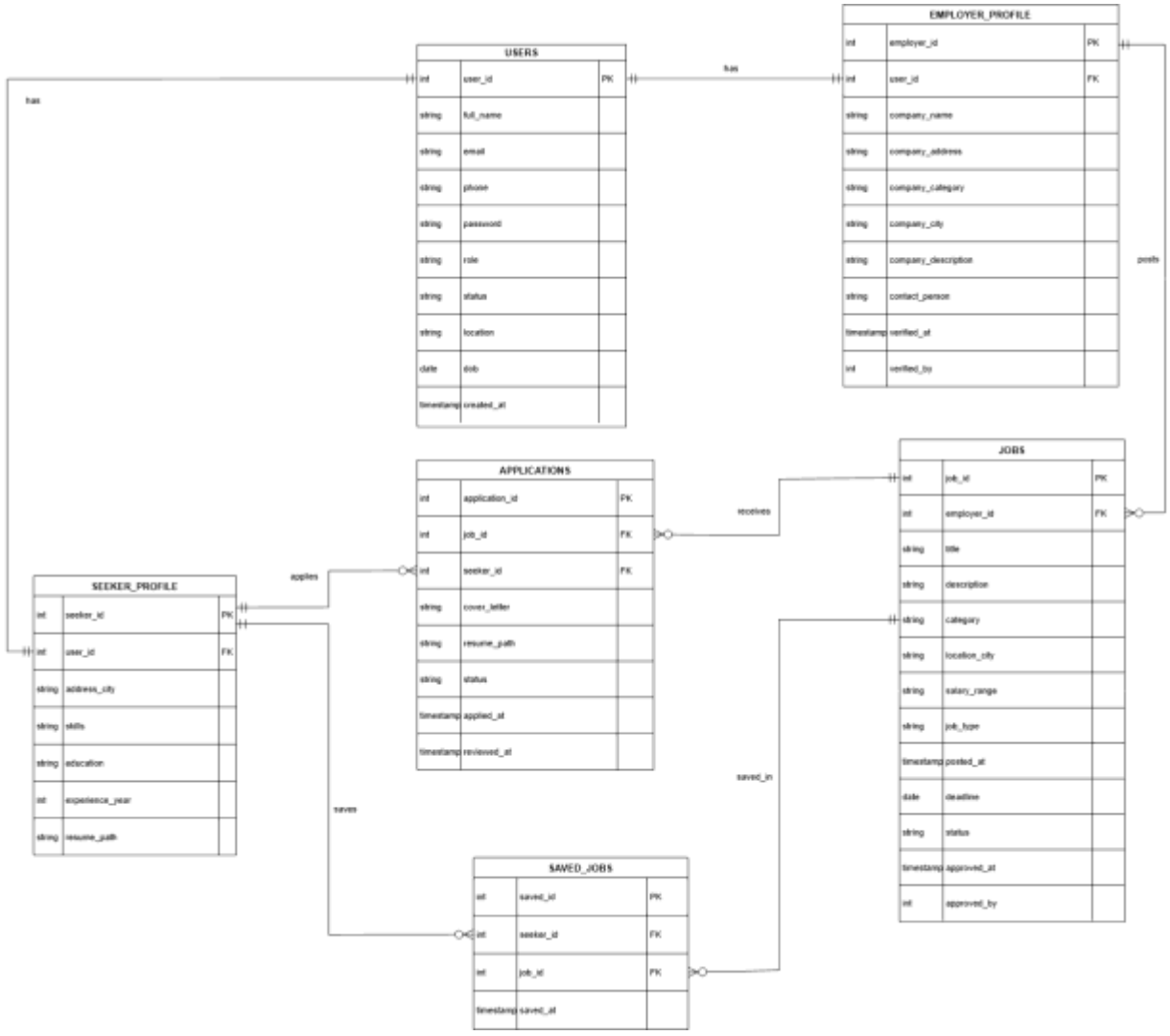


Figure 1 ERD

2.4 Normalization

Normalization is an important process in designing databases that plays an important role in improving efficiency, consistency, and accuracy of the database. The process makes it easier to manage and control data in the database and makes it more flexible so as to adapt to the ever-changing business environment (GeeksforGeeks, 2026).

Some benefits of normalization include minimization of data redundancy, which means that data is stored in one place only, hence ensuring consistency. This process also makes data more accurate, because any update of data will have to be done in one place only. Normalization makes data more consistent, whereby it is organized well in well-structured tables. Moreover, normalization makes it possible to create queries easily through combining tables, as desired.

2.4.1 UNF

User(user_id, full_name, email, phone, password, role, status, location, dob, created_at, {seeker_id, user_id, address_city, education, experience_year, resume_path, skills}, {employer_id, user_id, company_name, company_address, company_category, company_city, company_description, contact_person, verified_at, verified_by}, {job_id, employer_id, title, description, category, location_city, salary_range, job_type, posted_at, deadline, status, approved_at, approved_by, {application_id, job_id, seeker_id, cover_letter, resume_path, application_status, applied_at, reviewed_at}, {saved_job_id, seeker_id, job_id, saved_at}})

2.4.2 First Normal Form (1NF):

Normalization of 1NF form of a database is when all the columns contain one value only, and each row is unique. Elimination of repeating groups and making sure that a column contains only one value ensures that there is data integrity, reduction of redundancy, and a simpler database design that will be easier to query, update and manage (GeeksforGeeks, 2026).

Rule of 1NF

1. All fields consist of atomic data.

2. There are no repeating groups.

3. The records are uniquely identifiable using primary keys.

In order to satisfy 1NF, all repeating groups in UNF must be decomposed into separate relations and each field contains only one value.

1NF

users-1 (user_id, full_name, email, phone, password, role, status, location, dob, created_at)

seeker_profile-1 (seeker_id, user_id*, address_city, skills, education, experience_year, resume_path)

employer_profile-1 (employer_id, user_id*, company_name, company_category, company_city, company_description, contact_person, verified_at, verified_by)

jobs-1 (job_id, employer_id*, job_title, job_description, job_category, job_location_city, job_salary_range, job_type, job_posted_at, job_deadline, job_status, job_approved_at, job_approved_by)

applications-1 (application_id, job_id*, seeker_id*, cover_letter, resume_path, application_status, applied_at, reviewed_at)

saved_jobs-1 (saved_id, seeker_id*, job_id*, saved_at)

2.4.3 Second Normal Form (2NF)

Partial Dependency problem is solved by applying 2nd Normal Form, where the non-key attribute should be fully functionally dependent on primary key and there should not be any partial dependency (GeeksforGeeks, 2025).

Rule for 2NF

1. It should be in 1NF

2. There is no partial dependency (a non-key attribute cannot depend on part of primary key).

2NF

Checking Functional Dependency in Users-1 table

users-1 (user_id, full_name, email, phone, password, role, status, location, dob, created_at)

FFD: user_id → full_name, email, phone, password, role, status, location, dob, created_at

Since all non-key attributes are fully dependent on the primary key user_id, the table is in 2NF.

Checking Functional Dependency in Seeker_Profile-1 table

seeker_profile-1 (seeker_id, user_id*, address_city, skills, education, experience_year, resume_path)

FFD: seeker_id → user_id, address_city, skills, skils, education, experience_year, resume_path

Since all non-key attributes are fully dependent on the primary key seeker_id, the table is in 2NF.

Checking Functional Dependency in Employer_Profile-1 table

employer_profile-1 (employer_id, user_id*, company_name, company_category, company_city, company_description, contact_person, verified_at, verified_by)

FFD: employer_id → user_id, company_name, company_address, company_category, company_city, company_description, contact_person, verified_at, verified_by

Since all non-key attributes are fully dependent on the primary key employer_id, the table is in 2NF.

Checking Functional Dependency in Jobs-1 table

jobs-1 (job_id, employer_id*, title, description, category, location_city, salary_range, job_type, posted_at, deadline, status, approved_at, approved_by)

FFD: job_id → employer_id, title, description, category, location_city, salary_range, job_type, posted_at, deadline, status, approved_at, approved_by

Since all non-key attributes are fully dependent on the primary key job_id, the table is in 2NF.

Checking Functional Dependency in Applications-1 table

applications-1 (application_id, job_id*, seeker_id*, cover_letter, resume_path, status, applied_at, reviewed_at)

FFD: application_id → job_id, seeker_id, cover_letter, resume_path, status, applied_at, reviewed_at

Since all non-key attributes are fully dependent on the primary key application_id, the table is in 2NF.

Checking Functional Dependency in Saved_Jobs-1 table

saved_jobs-1 (saved_id, seeker_id*, job_id*, saved_at)

FFD: saved_id → seeker_id, job_id, saved_at

Since all non-key attributes are fully dependent on the primary key saved_id, the table is in 2NF.

2NF

users-2 (user_id, username, email, phone, password, role, status, location, dob, created_at)

seeker_profile-2 (seeker_id, user_id*, address_city, skills, education, experience_year, resume_path)

employer_profile-2 (employer_id, user_id*, company_name, company_category, company_city, company_description, contact_person, verified_at, verified_by)

jobs-2 (job_id, employer_id*, job_title, job_description, job_category, job_location_city, job_salary_range, job_type, job_posted_at, job_deadline, job_status, job_approved_at, job_approved_by)

applications-2 (application_id, job_id*, seeker_id*, cover_letter, resume_path, application_status, applied_at, reviewed_at)

saved_jobs-2 (saved_id, seeker_id*, job_id*, saved_at)

2.4.4 Third Normal From (3NF)

The third normal form removes the transitive dependencies from the database; it is such that there are no attributes other than the primary key that has a dependency on the primary key. A> B and B – > C implies that C is transitively dependent on A. So, in 3NF, this kind of transitive dependency is removed using a separate table and hence all kinds of anomalies are removed (Geeksforgeeks, 2025).

Rule For 3NF

1. The Table is in Second Normal Form.

2. No Transitive Dependencies (Non-key Attribute cannot depend on other Non-Key Attribute)

3NF – Checking Transitive Dependency

Checking Transitive Dependency in Users-2 table

users-2 (user_id, full_name, email, phone, password, role, status, location, dob, created_at)

user_id → full_name → X
user_id → email → X
user_id → phone → X
user_id → password → X
user_id → role → X
user_id → status → X
user_id → location → X
user_id → dob → X
user_id → created_at → X

Since no non-key attribute depends on another non-key attribute, the table is in 3NF.

Checking Transitive Dependency in Seeker_Profile-2 table

seeker_profile-2 (seeker_id, user_id*, address_city, skills, education, experience_year, resume_path)

seeker_id → user_id → X

seeker_id → address_city → X

seeker_id → education → X

seeker_id → skills → X

seeker_id → experience_year → X

seeker_id → resume_path → X

Since no transitive dependency exists, the table is in 3NF.

Checking Transitive Dependency in Employer_Profile-2 table

employer_profile-2 (employer_id, user_id*, company_name, company_category, company_city, company_description, contact_person, verified_at, verified_by)

employer_id → user_id → X

employer_id → company_name → X

employer_id → company_address → X

employer_id → company_category → X

employer_id → company_city → X

employer_id → company_description → X

employer_id → contact_person → X

employer_id → verified_at → X

employer_id → verified_by → X

Since no transitive dependency exists, the table is in 3NF.

Checking Transitive Dependency in Jobs-2 table

jobs-2 (job_id, employer_id*, title, description, category, location_city, salary_range, job_type, posted_at, deadline, status, approved_at, approved_by)

job_id → employer_id → X

job_id → title → X

job_id → description → X

job_id → category → X

job_id → location_city → X

job_id → salary_range → X

job_id → job_type → X

job_id → posted_at → X

job_id → deadline → X

job_id → status → X

job_id → approved_at → X

job_id → approved_by → X

Since no non-key attribute depends on another non-key attribute, the table is in 3NF.

Checking Transitive Dependency in Applications-2 table

applications-2 (application_id, job_id*, seeker_id*, cover_letter, resume_path, status, applied_at, reviewed_at)

application_id → job_id → X

application_id → seeker_id → X

application_id → cover_letter → X

application_id → resume_path → X

application_id → status → X

application_id → applied_at → X

application_id → reviewed_at → X

Since no transitive dependency exists, the table is in 3NF.

Checking Transitive Dependency in Saved_Jobs-2 table

saved_Jobs-2 (saved_id, seeker_id*, job_id*, saved_at)

saved_id → seeker_id → X

saved_id → job_id → X

saved_id → saved_at → X

Since no non-key attribute depends on another non-key attribute, the table is in 3NF.

3NF

users-3 (user_id, username, email, phone, password, role, status, location, dob, created_at)

seeker_profile-3 (seeker_id, user_id*, address_city, skills, education, experience_year, resume_path)

employer_profile-3 (employer_id, user_id*, company_name, company_category, company_city, company_description, contact_person, verified_at, verified_by)

jobs-3(job_id, employer_id*, job_title, job_description, job_category, job_location_city, job_salary_range, job_type, job_posted_at, job_deadline, job_status, job_approved_at, admin_id*)

applications-2 (application_id, job_id*, seeker_id*, cover_letter, resume_path, application_status, applied_at, reviewed_at)

saved_jobs-3 (saved_id, seeker_id*, job_id*, saved_at)

2.5 Database Development

The database for the **RojgarSetu** system was designed and implemented using MySQL Workbench. It is a relational database system that ensures proper organization of data using tables, relationships, primary keys, and foreign keys. The main goal of database development is to store user information, job postings, applications, and saved jobs in a structured format so that data redundancy is minimized and data integrity is maintained. The system follows normalization rules and supports efficient data retrieval for both job seekers and employers.

The database consists of multiple interconnected tables such as USERS, SEEKER_PROFILE, EMPLOYER_PROFILE, JOBS, APPLICATIONS, and SAVED_JOBS. Each table represents a specific entity of the system and is connected through relationships such as one-to-one and one-to-many.

2.5.1 USERS Table

The USERS table is the main table of the system. It holds basic details of all users including job-seekers, employers, and admins. This table is primarily used for authentication purposes and authorization purposes as well. All user gets a unique ID and email is made unique to avoid duplicates. User table acts as the parent table in the database because SEEKER_PROFILE and EMPLOYER_PROFILE tables depend on it and it holds critical data including the full name of the user, email, phone number, password, role, account status, location, DOB, and date of account creation.

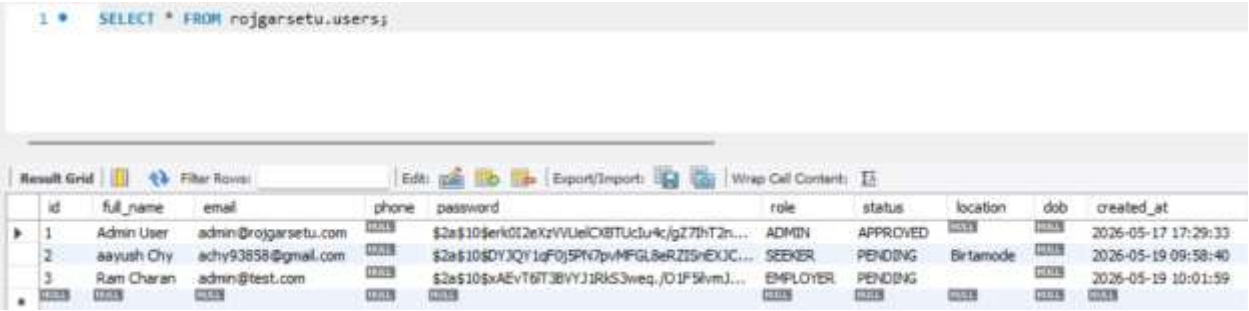


Figure 2 User table

2.5.2 SEEKER_PROFILE Table

SEEKER_PROFILE table has the details of the job seekers' professional life. These details include personal career related details like address city, skills, education level, experience years, and resume file path. The SEEKER_PROFILE table is associated with USERS table by means of a foreign key (user_id). one-to-one relationship between the tables is there. One seeker profile can only belong to one user.

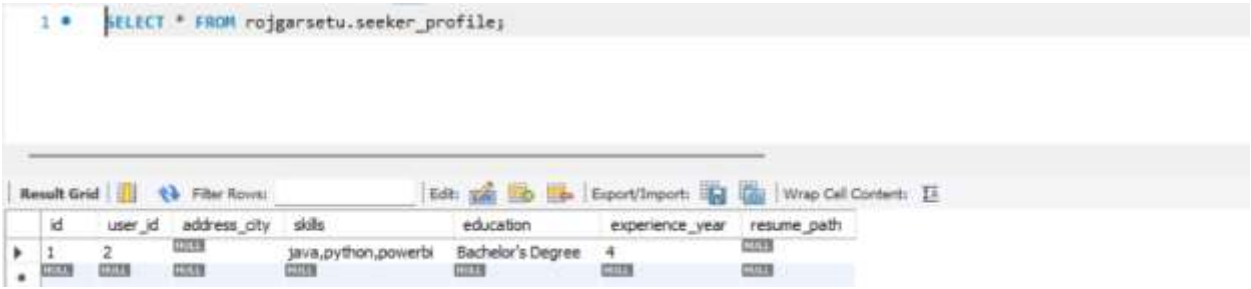


Figure 3 Seeker table

2.5.3 EMPLOYER_PROFILE Table

EMPLOYER_PROFILE is a table where detailed information about companies of those who advertise jobs through the system is stored. This table is consists of such fields as company_name, address, category, city, description, contact_person, and verify. Moreover, this table is related to the USERS table via a foreign key (user_id). Thus, an employer profile corresponds to exactly one user.

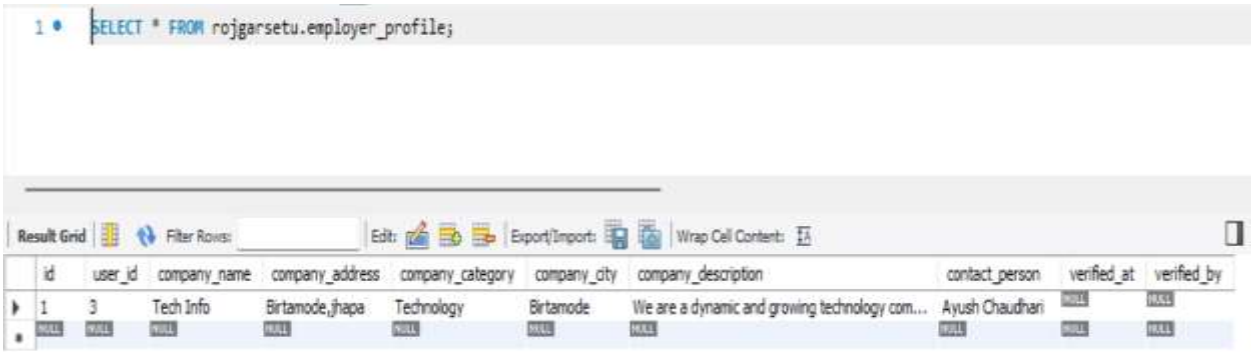


Figure 4 Employer table

2.5.4 JOBS Table

The JOBS table is the main table in the database and which contains all the jobs that have been created by the employers, and it consists of fields such as title, description, category, location, salary range, type of job, date posted, expiration date, and whether it has been approved. It has a relationship with the EMPLOYER_PROFILE table based on `employer_id`, whereby one employer can post multiple jobs.

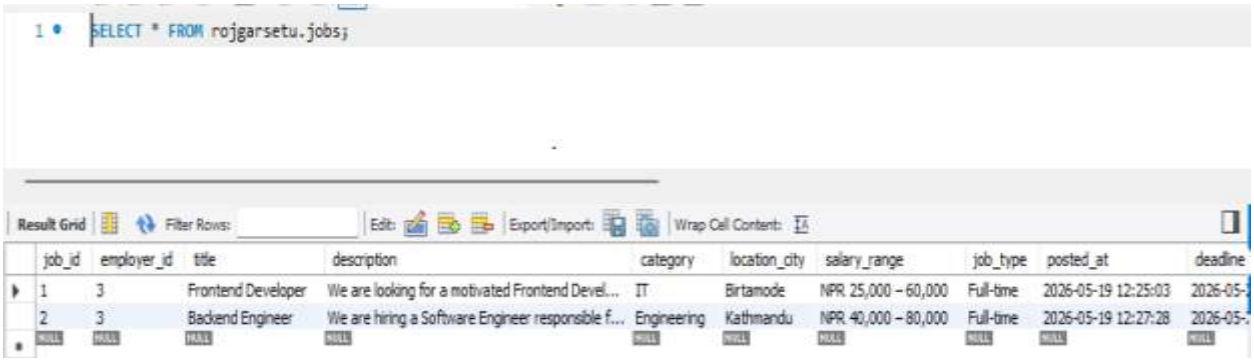


Figure 5 Job table

2.5.5 APPLICATIONS Table

The APPLICATIONS table stores all applications made by applicants. This table plays an intermediary role between SEEKER_PROFILE and JOBS tables. The table contains information on which applicant applied for which job. The table contains key information like cover letter, resume path, application status, application date, and review date. An application is related to one applicant and one job but an applicant may apply for many jobs.

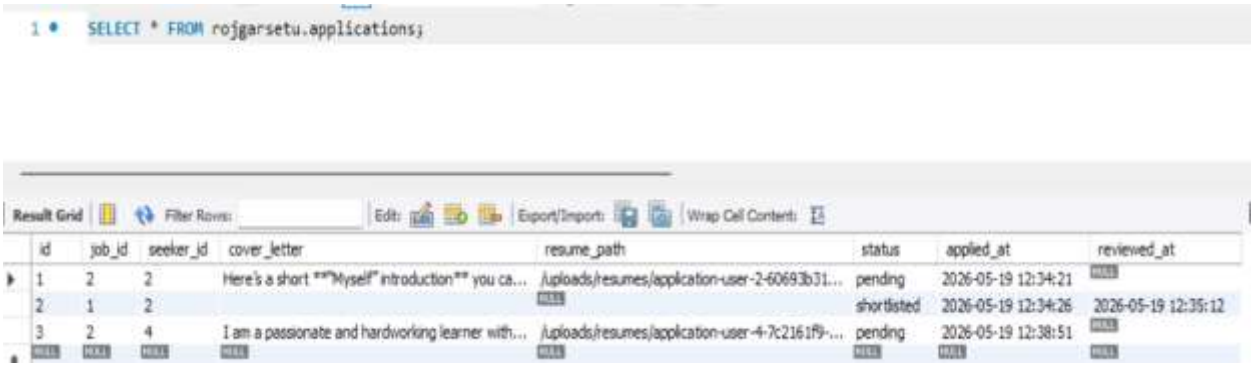


Figure 6 Application table

2.5.6 SAVED_JOBS Table

The SAVED_JOBS table stores jobs saved by job seekers for future reference. It allows users to bookmark jobs they are interested in without applying immediately. This table also acts as a bridge between SEEKER_PROFILE and JOBS tables. It contains foreign keys for both seeker_id and job_id along with the timestamp when the job was saved. A seeker can save multiple jobs, and a job can be saved by multiple seekers.

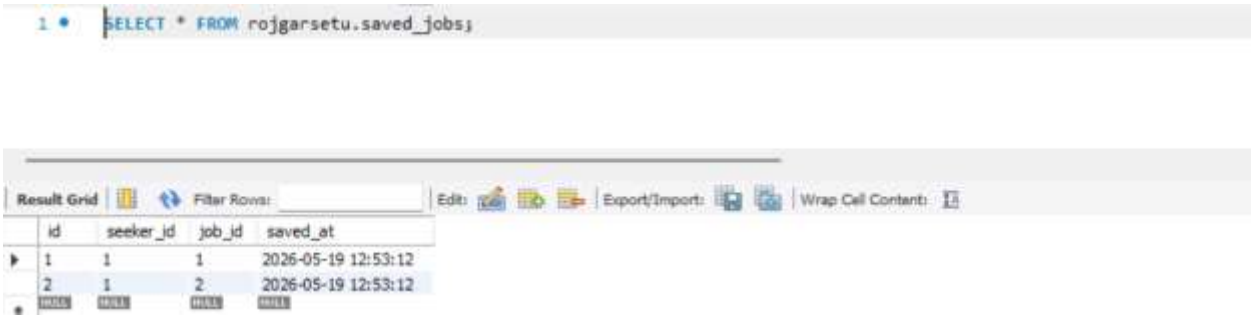


Figure 7 Saved_job table

2.6 Relationship Diagram

A relationship diagram is the visualization of a database through which connection between various tables (entities) can be identified through the use of primary key and foreign key. Relationship diagram provides an understanding of structure of database and the process through which data flows from one entity to another within the database (Geeksforgeeks, 2025). The relationship diagram for the system has been generated by using MySQL Workbench. This diagram provides the visualization of structure of database and connections between major tables such as USERS, SEEKER_PROFILE, EMPLOYER_PROFILE, JOBS, APPLICATIONS and SAVED_JOBS.

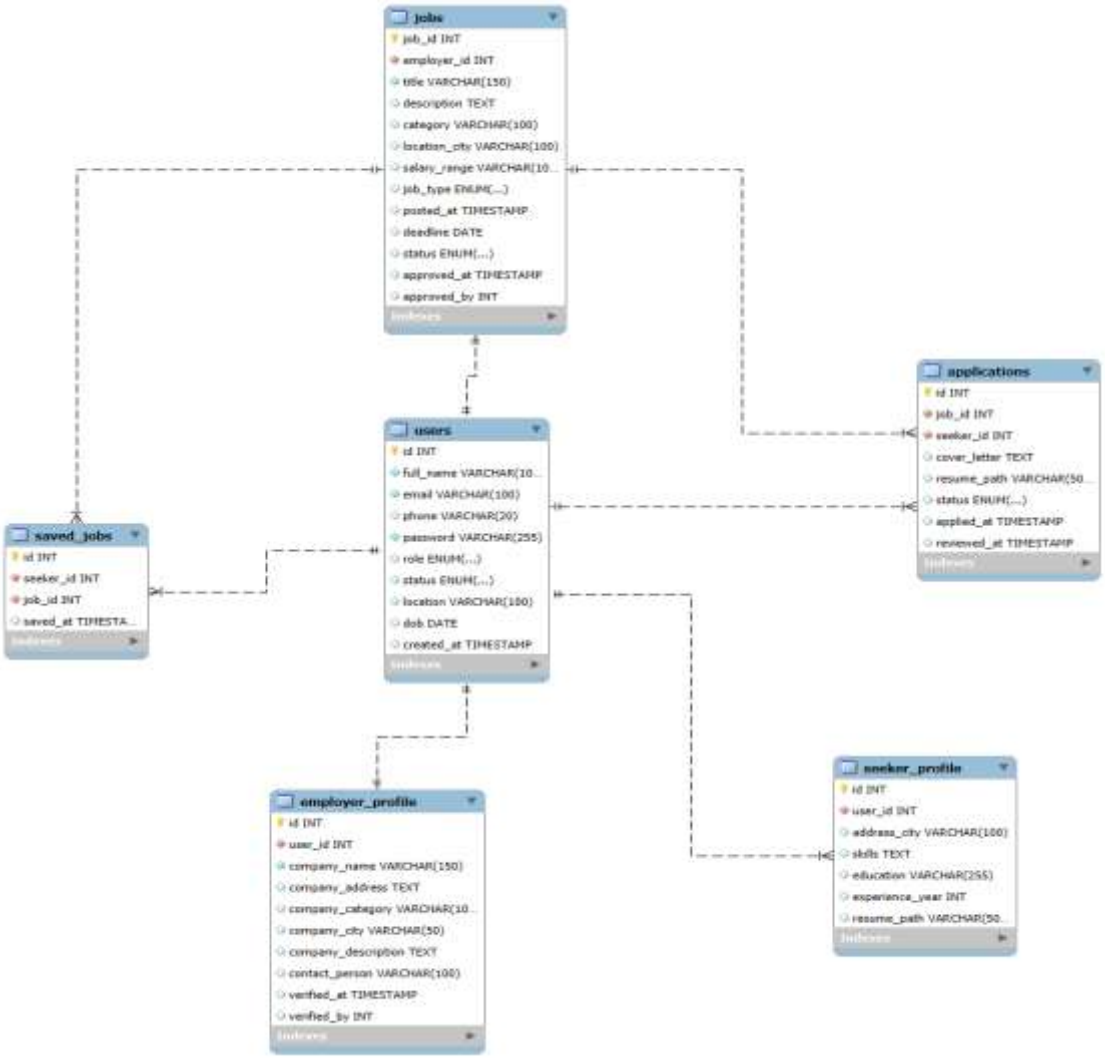


Figure 8 Relational ship Diagram

3 wireframes and UI/UX design.

3.1 Introduction to UI/UX Design

The development of the **RojgarSetu** web application was greatly influenced by the User Interface (UI) and User Experience (UX) design process. An effective UI/UX design can make a significant impact on increasing the usability, accessibility, ease of navigation, and user-system interaction. As the application was used by both job seekers and employers, it was essential to keep the interface simple, responsive, visually consistent and user-friendly across devices.

Low fidelity wireframes were created to sketch out each page to see how it would be structured and organized before implementing. The wireframes enabled the front-end development team to determine where navigation menus, forms, buttons, content sections and interactive elements would be placed before they start coding the front end. This helped to eliminate usability problems and uniformity across the application.

UI/UX process was user-centred. The layout structures, navigations, and responsive design principles of different professional platforms, including LinkedIn, indeed, Glassdoor and merojob.com were examined. Adhered to coursework requirements for the frontend, which included using JSP, CSS Flexbox, CSS Grid and media queries without Bootstrap or other external CSS libraries.

For aesthetics and modern-looking and to avoid eye strain, a dark theme was chosen as the design language. The typography, spacing, color palette, and consistency of the navigation and accessibility were retained throughout the application for professional user experience.

This section contains a detailed description of each of the web application user interface screens for RojgarSetu. The designs follow the principle of user centered design, responsive layout (CSS Flexbox and Grid) and consistent dark theme. Each figure is discussed with the focus on its functional intent, structural makeup and usability factors.

3.2 Admin

3.2.1 Admin dashboard

This is the main control panel of the administrator. It has a left-hand navigation menu, a top status bar and a central content pane with important metrics like total users, active jobs, reported content, and recent activities. Card based layout enhances visual hierarchy and facilitates quick access to the management functions.

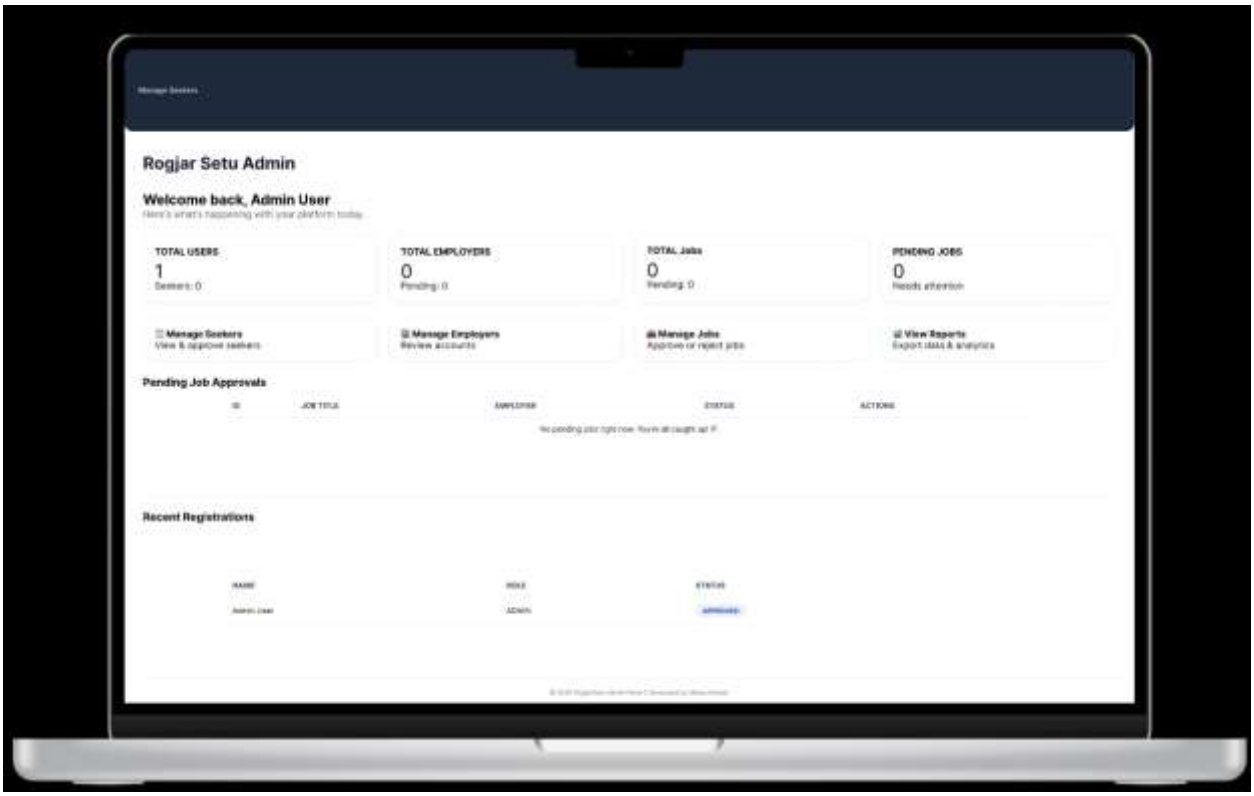


Figure 9 Admin dashboard

3.2.2 Admin Employer

This screen displays a list of all the registered employers in a tabular format. Some of the crucial data fields are the employer name, company name, contact email, account status and registration date. Action buttons (view, edit, block, delete) are made available for each row. This design allows to create efficient user moderation and role management.

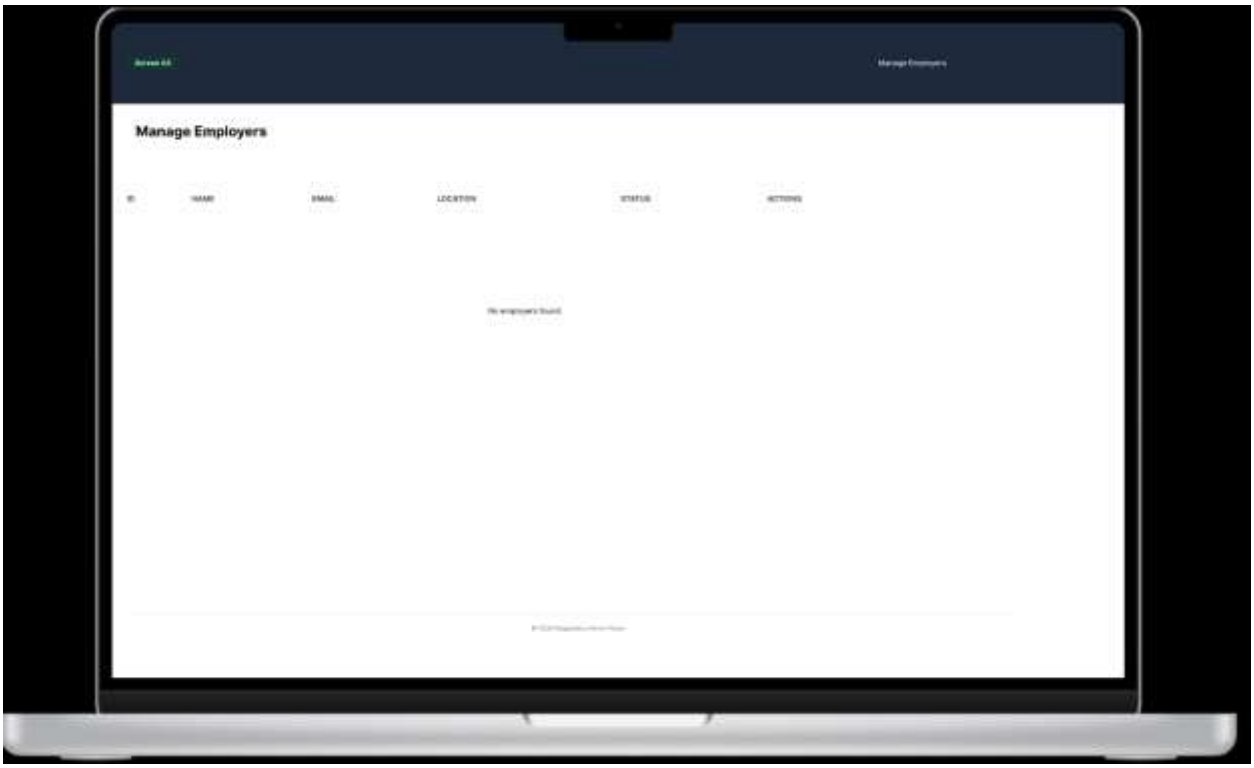


Figure 10 Employer Management

3.2.3 Admin jobs

The admin jobs interface shows an overview of all the jobs that have been published throughout the platform. The standard information found on a column is the job title, company name, date posted, status (active/expired/flagged), and number of applicants. Search and Filter options are given at the top of the table for the retrieval and moderation.

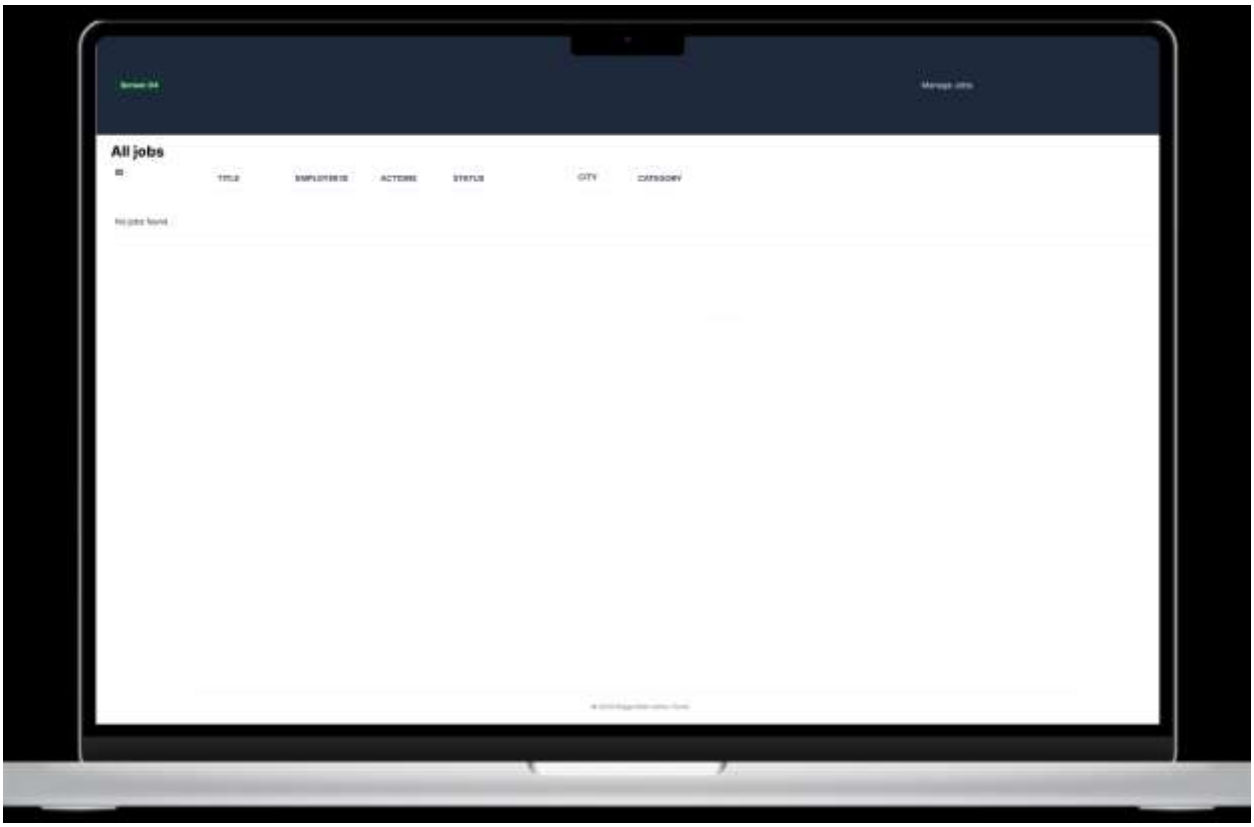


Figure 11 Jobs Management

3.2.4 Admin Profile

This screen enables the admin to look at and update their profile information. It contains a full name, email, profile picture and option to change the password. It is designed in a two column format with a focus on providing a balance between the density of information and readability, and includes validation indicators for mandatory fields.

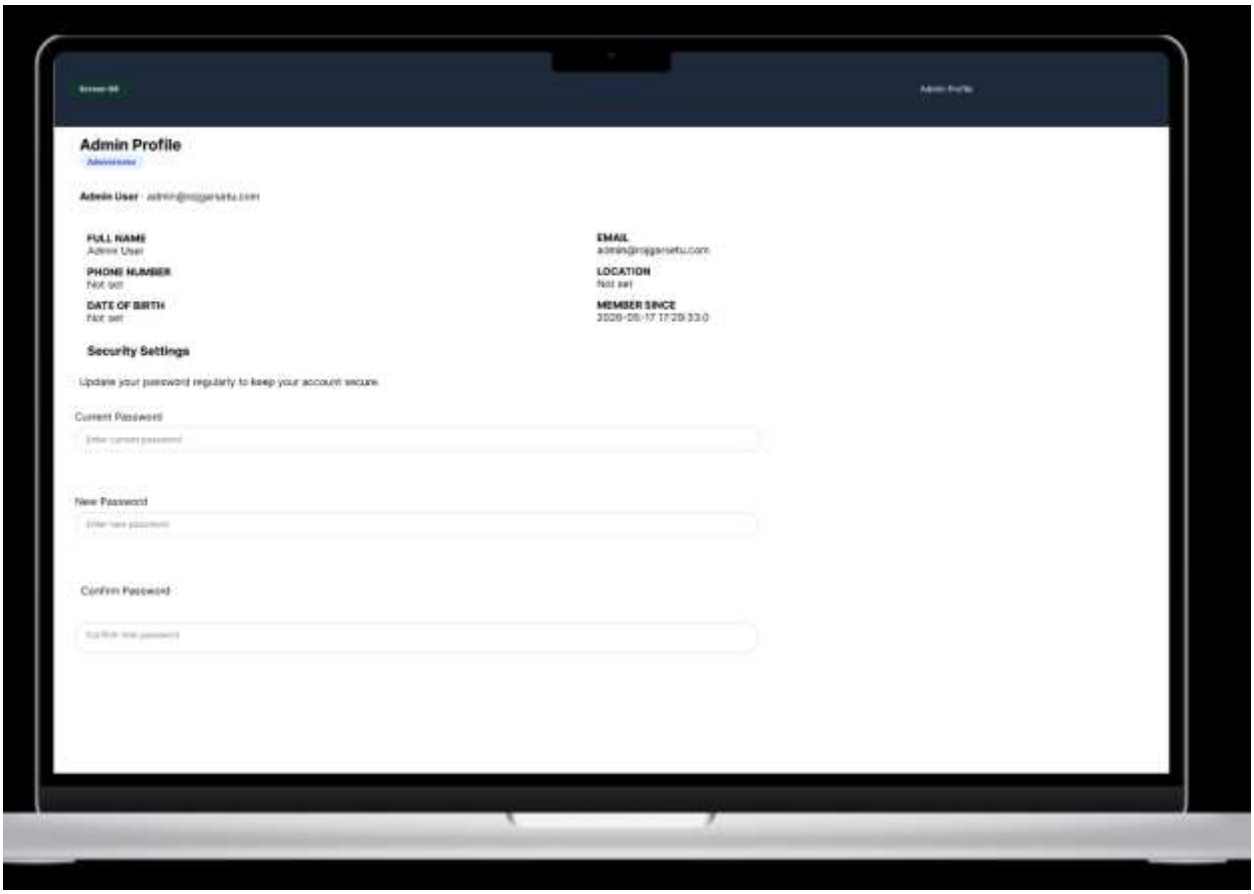


Figure 12 Profile Management

3.2.5 Admin Report

A report management interface shows the user reports created by the user on job posts, employers or job seekers. All reports contain a reporter, the reported entity, the reason for reporting, the time of reporting and the status of the report (pending/resolved). Action buttons enable the admin to investigate and take the appropriate actions. This helps to advance platform accountability.

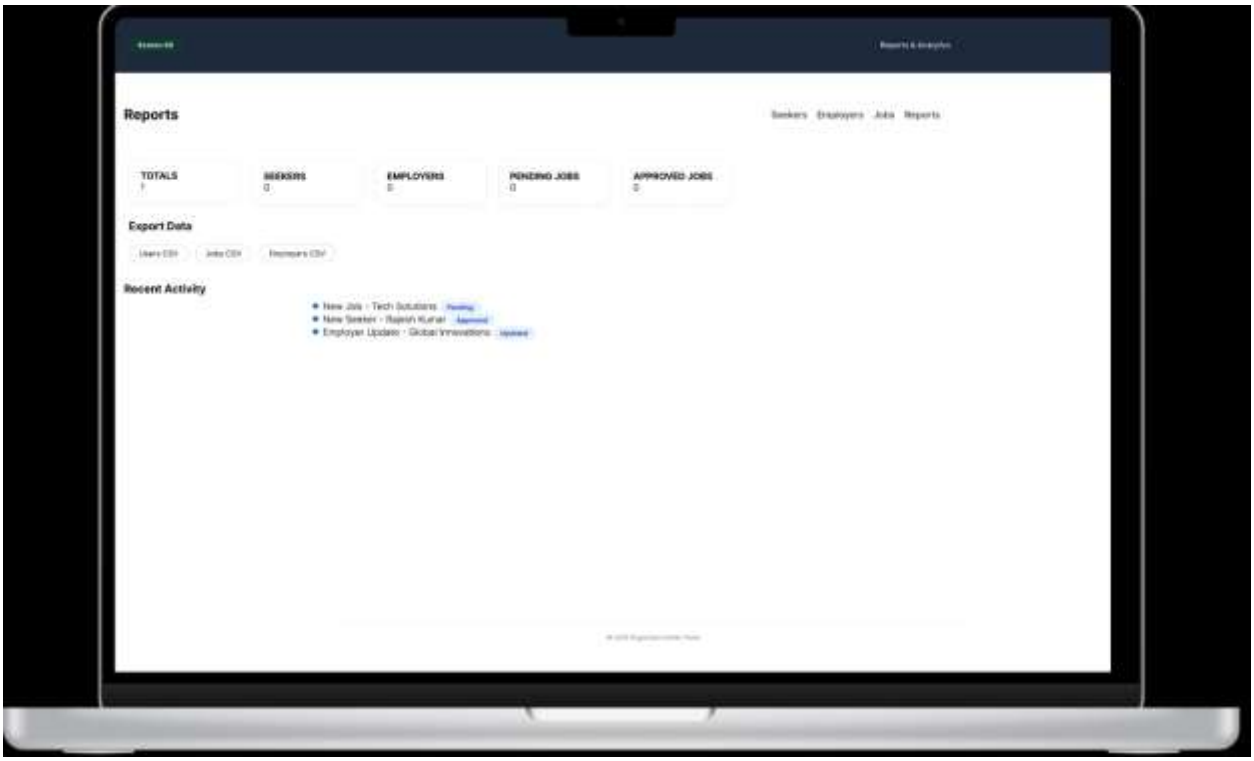


Figure 13 Report

3.2.6 Admin Seeker

Like an employer's management this screen displays a list of all job seekers registered. Seekers' information includes seeker name, contact information, skills summary, account status and activity log. The interface has features of searching, filtering by status and batch operation for user administration.

Figure 14 Job seeker management

3.3 Employer

3.3.1 Demo employer

The main page that employers will see after they log in. It shows summary of recent jobs posted, number of applicants and quick buttons (like Post New Job, View Applicants). The design allows for the most efficient performance of the tasks, with the most important metrics displayed at the top.

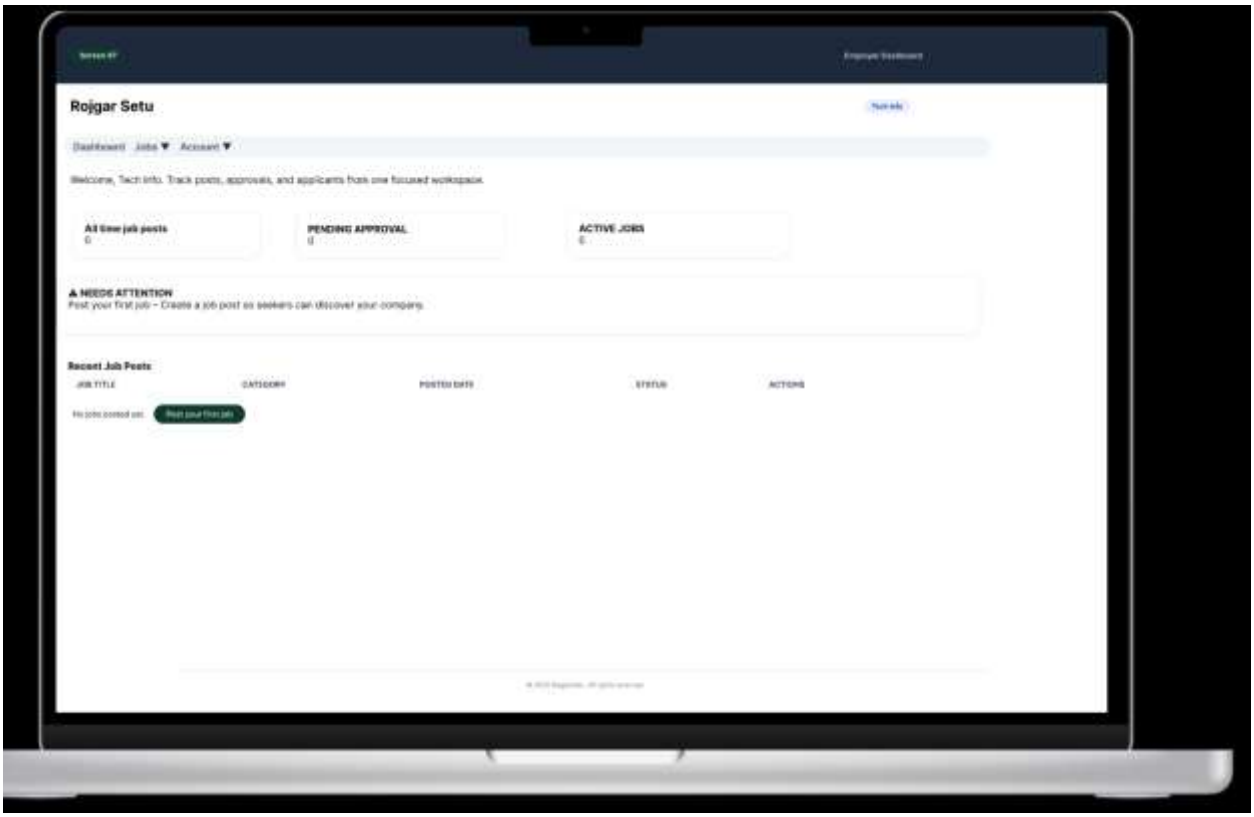


Figure 15 Employer Dashboard

3.3.2 Applicants

The applicants screen displays all of the job applications that have been received for the employer's posted jobs. The data shown are the applicant name, applied job title, application date and status (reviewed/pending/shortlisted/rejected). Dropdown filters and status toggle buttons support recruitment workflow management.

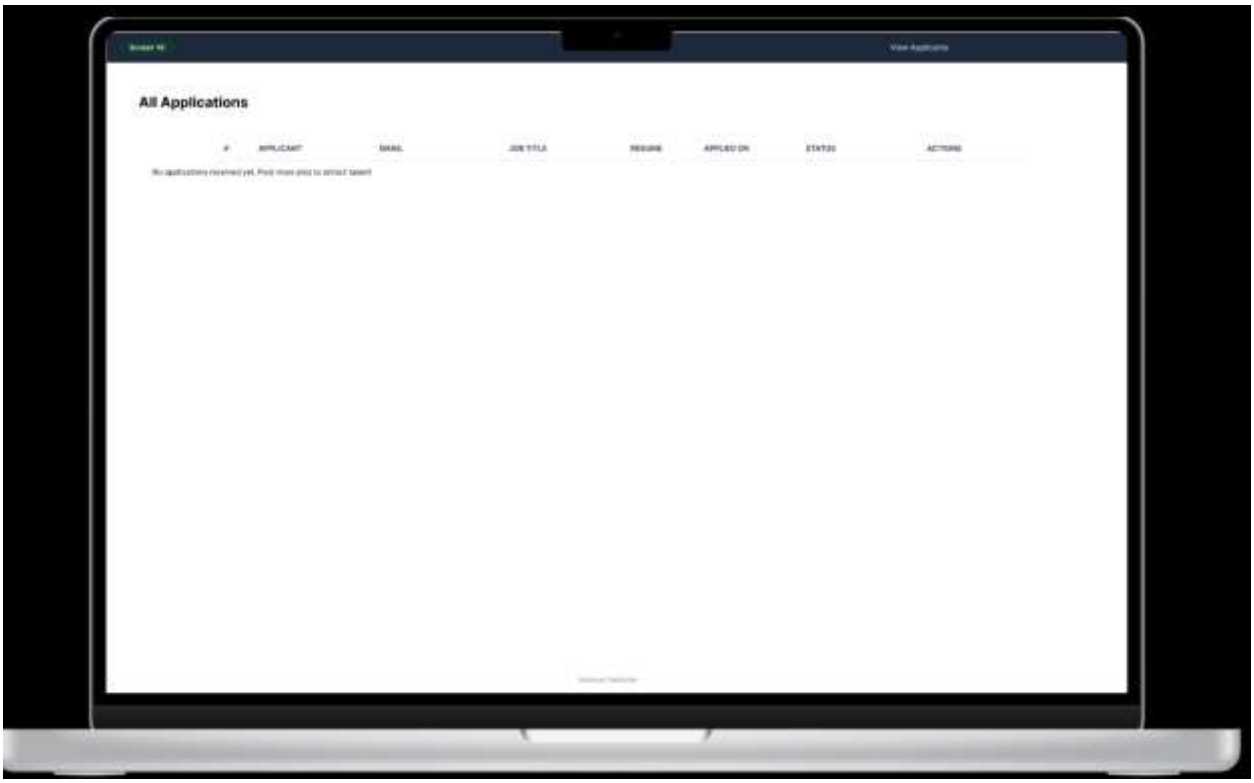


Figure 16 Applicants Management

3.3.3 My job

All jobs listed by the logged-in employer. Job titles, location, posting date and expire date are included for each entry along with the number of applicants. Edit, delete and view icons provided. The design allows employers to monitor and oversee their current and terminated job announcements.

Figure 17 My job

3.3.4 Post job

A form interface with structure to create new job postings. Applicants can fill out details like job title, job description, skills required, level of experience, salary range, full-time/part-time jobs, contract, application deadline, and company information. Data is validated to assure quality including input validation and character limits.

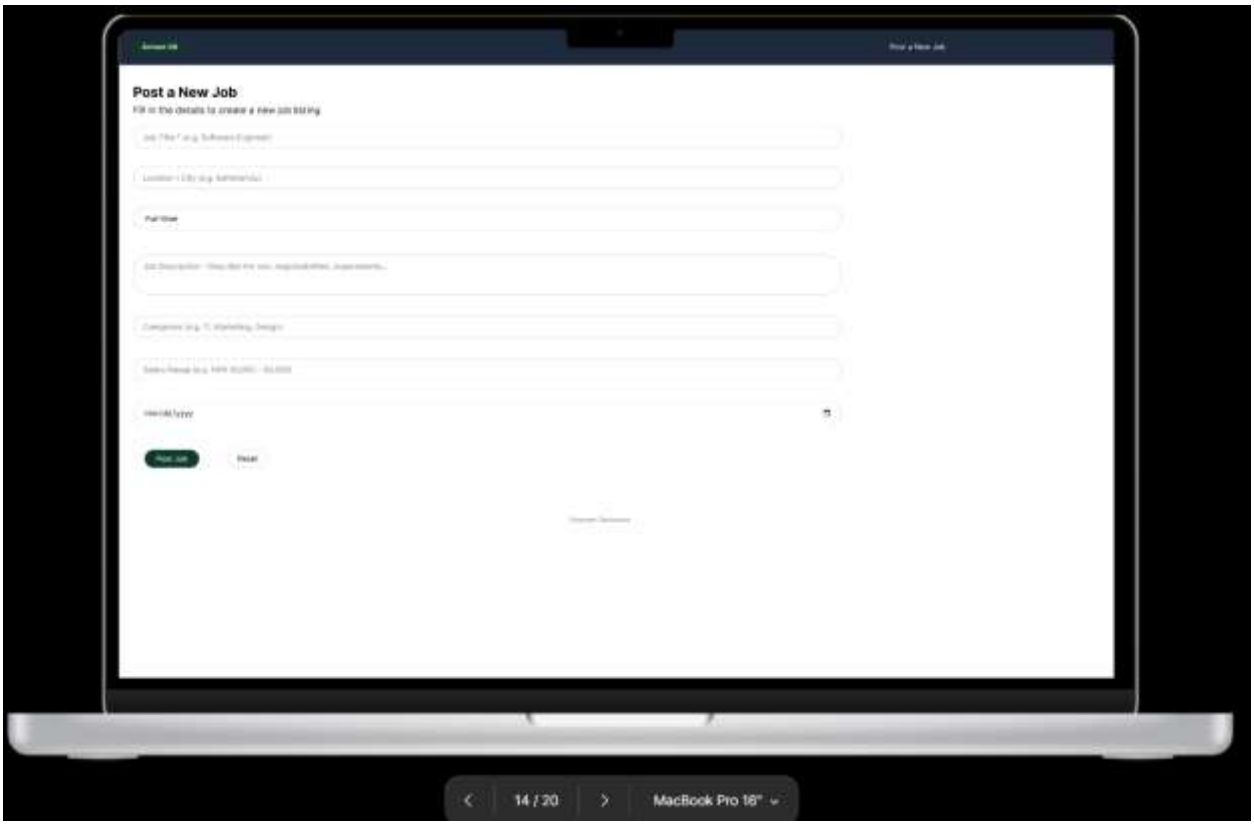


Figure 18 New Job Post

3.3.5 Profile

This screen enables employers to control their company profile. The parts consist of uploading your company logo, entering your company description, entering your contact information, and entering your website link. The interface offers a preview panel of the profile's appearance for the people seeking employment, thus boosting users' confidence.

Figure 19 Employer Profile

3.4 Login registration

3.4.1 Demo login

The login screen has a card-based layout with an email/username field and a password field and checkbox to save the password and username, and a login button. There is a “Forgot password?” link as well as a redirection link to registration. The design will be as lean as possible, leaving only authentication actions.

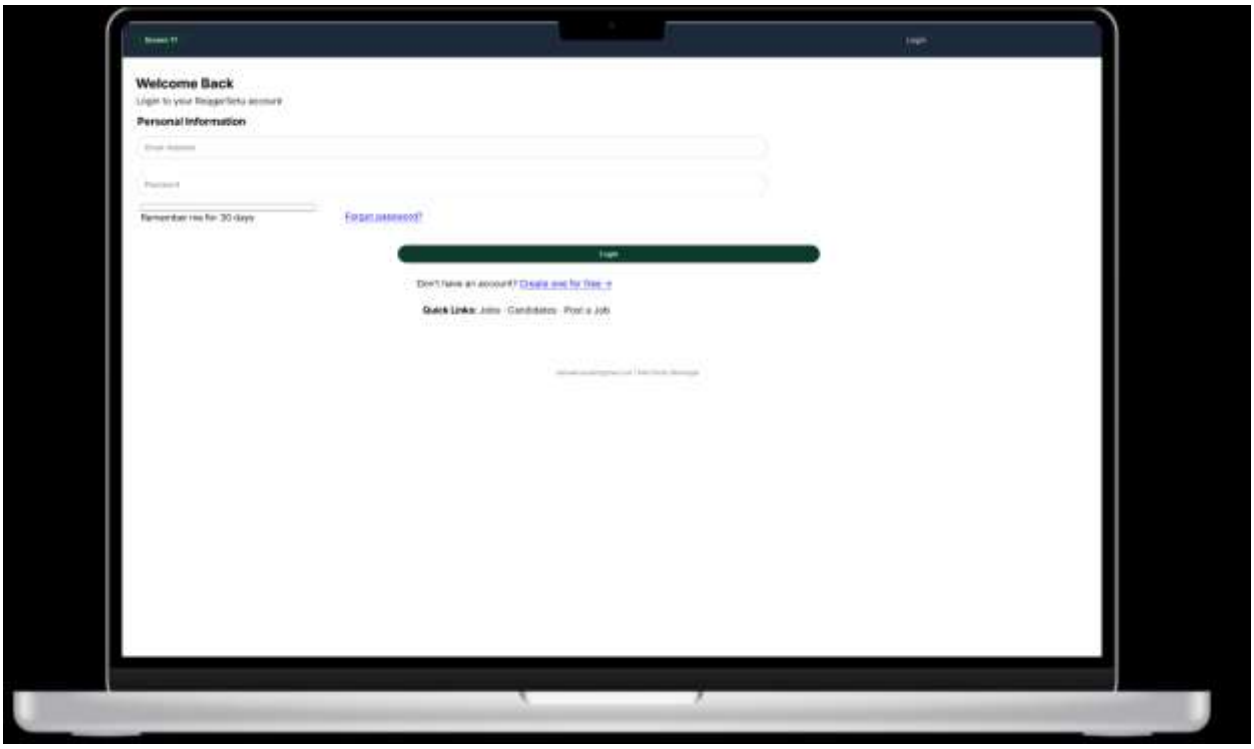


Figure 20 Login Demo

3.4.2 Demo registration

The registration form gathers the required information from the user like his/her full name, e-mail ID, password, confirm password, and user type (employer or job seeker). Hints and strength indicator for password are added for real time validation to minimize input errors and enhance data integrity.

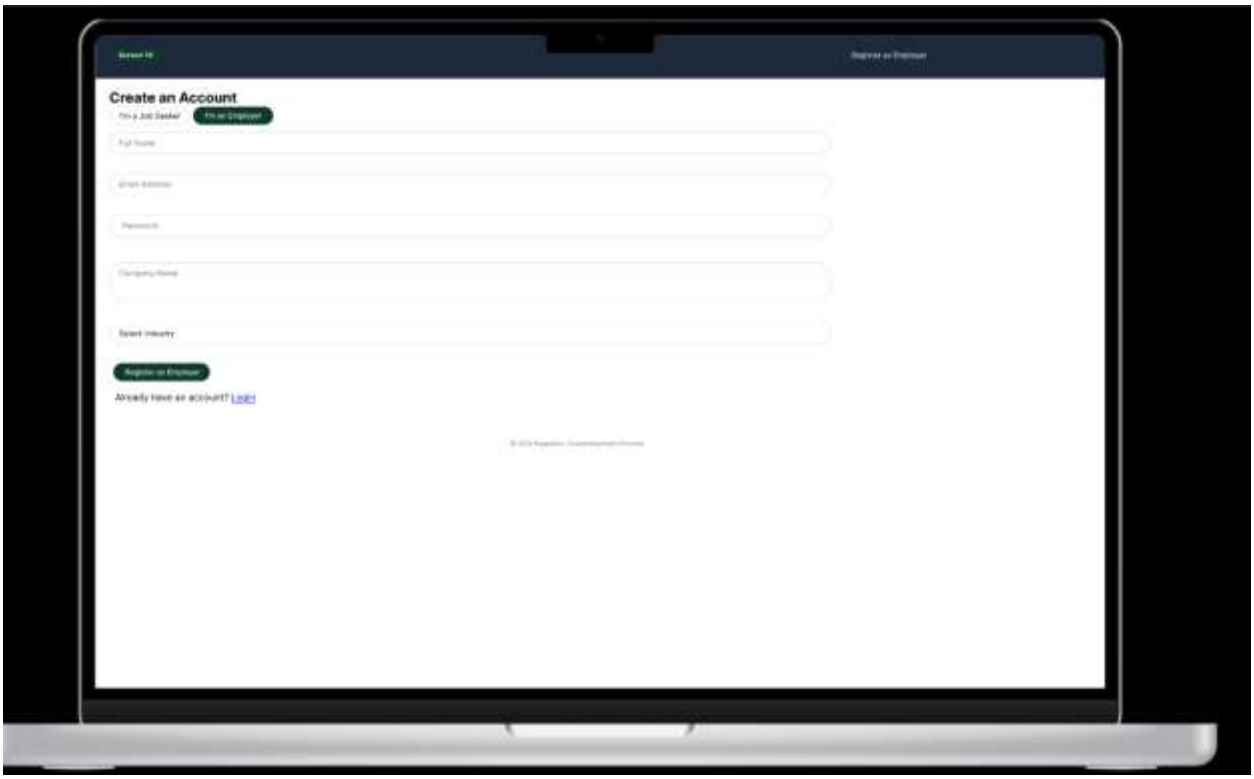


Figure 21 Registration Demo

3.5 Public page

3.5.1 Demo about

This is a public facing page, which communicates information about the mission, vision and team of the RojgarSetu platform. The layout consists of a hero section, blocks of descriptive text and, optionally, statistics (such as number of users placed, companies registered). The dark theme is used to enable readability on all devices.

Figure 22 About page

3.5.2 Demo Contact

The contact interface contains a contact form containing the name, email, subject, and message fields, along with additional contact details like email address, phone numbers, and additional address (when it comes to a physical address). It is recommended to add map integration placeholder. The form is client valid, so that it does not accept blank submissions.

Figure 23 Contact Page

3.5.3 Demo Home

The public home page is the starting page for all users that are not authenticated. It has a hero banner that includes a call-to-action (CTA), a search feature that allows users to look for jobs by title or location, featured job listings, and platform highlights. It is responsive and scrolls vertically.

Figure 24 Home page

3.5.4 Demo Jobs

This page will show you all the currently available job posts, without having to log in. Jobs appear as cards or list items that include the job title, company, location and a short description. Facilitating job discovery, there are jobs that are paginated, sorted (by date, relevance) and filtered (by category, location).

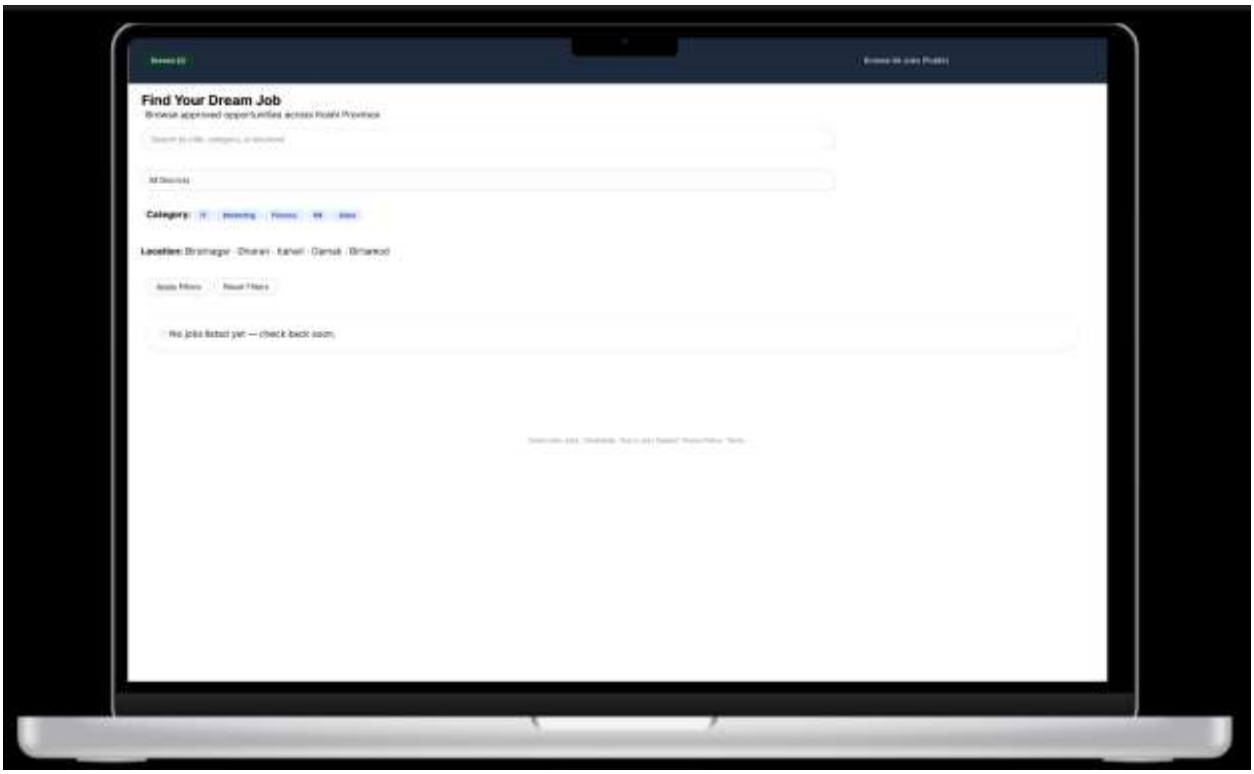


Figure 25 Available job post

3.6 Seeker

3.6.1 Demo seeker

The job seeker's Dashboard provides an overview of what has been saved, applied to, and recommended jobs in the past. Personalized greeting and completion of user profile encourages user interaction. There are links to browse for jobs and edit profile conveniently displayed.

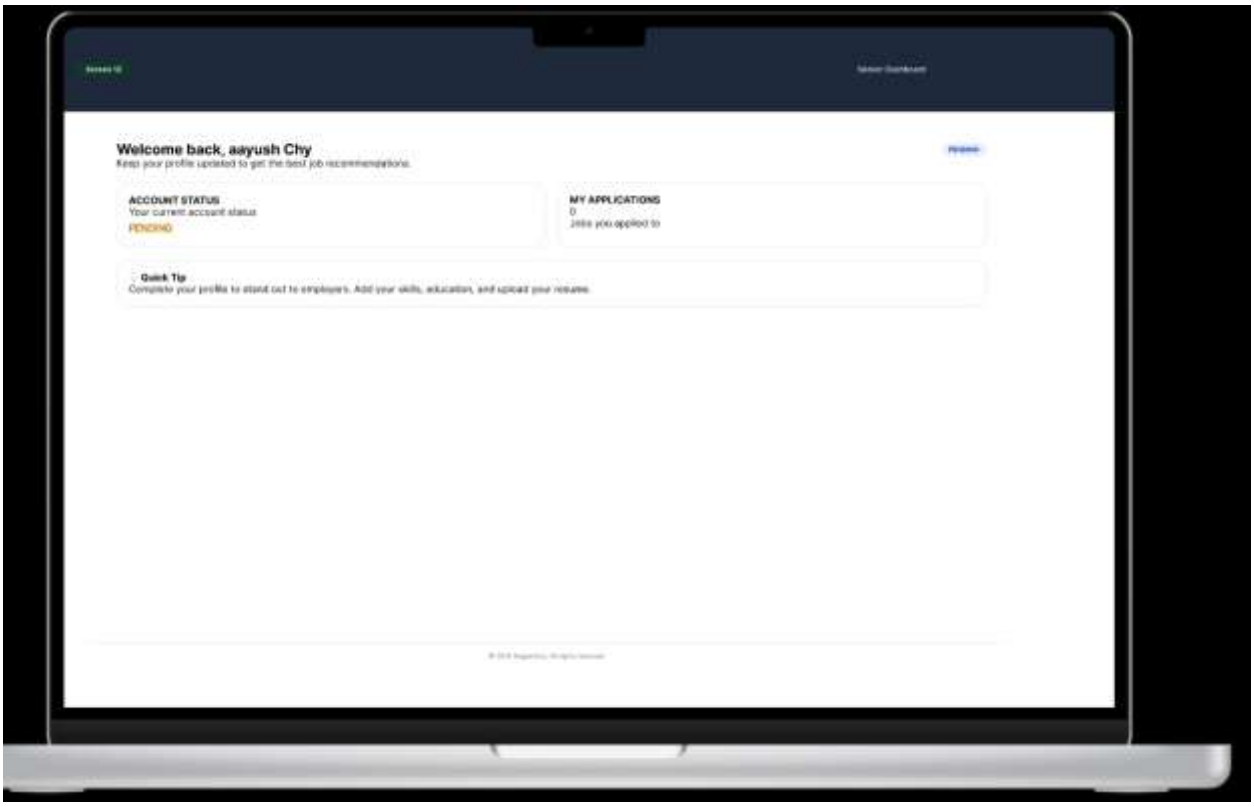


Figure 26 Seeker Dashboard

3.6.2 Applications

The interface will be monitoring all of the jobs to which the seeker has applied. The entries list the job title, company name, date applied for, status (pending, reviewed, rejected, shortlisted), and (optional) withdrawal or message options. The tabular format facilitates the tracking of the stage of application.

Figure 27 My Applications

3.6.3 Browse

An interface for enterprising people to search and discover available positions. Advanced filters are available for job type, salary range, location and experience level. Search results are dynamically updated. There is a “Save” or “Apply” button on each job card, and there is graphical feedback for those that have been saved.

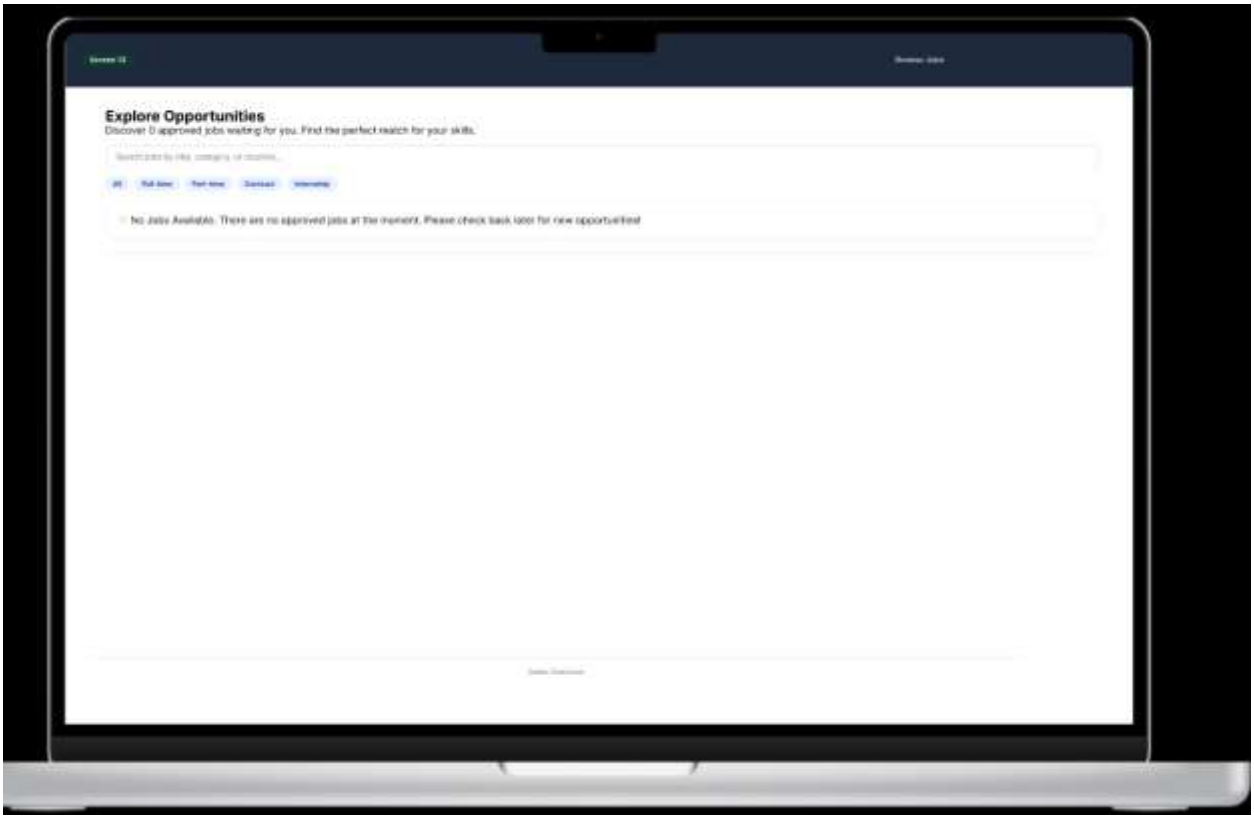


Figure 28 Browse Jobs

3.6.4 Profile

Job seekers' profile management screen contains personal data, contact details, resume upload, skills list, work experience and education history. A preview section gives you an insight into how your profile will look to employers. Form sections are collapsible to minimize clutter.

Figure 29 Seeker Profile

3.6.5 Saved job

This interface is for keeping track of jobs that the seeker has marked for future consideration. Every saved job entry has a job title, company, save date and action to apply or remove details. If no jobs are saved, an empty state message is provided. This helps in promoting the asynchronous nature of job searching.

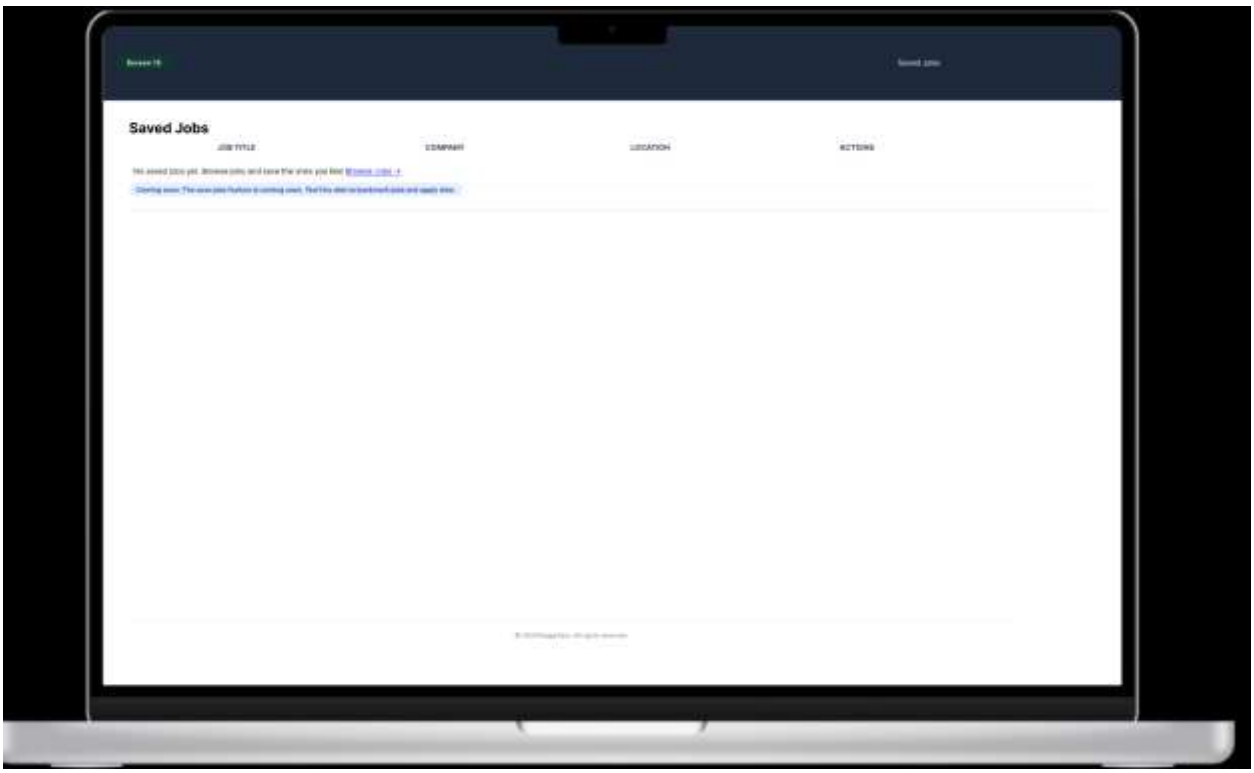


Figure 30 Saved Jobs

4 Class Diagram

A class diagram is one form of UML diagram that graphically represents the internal structure of a system. The diagram is made up of classes, which serve as prototypes for objects, and these classes are made up of attributes and methods (also referred to as functions). Besides depicting classes, the class diagram depicts their associations, including those of association, inheritance, aggregation, and composition. This enables the relationships among different parts of a software system to be understood better. Class diagrams are used extensively in object-oriented software engineering since they facilitate planning and understanding of the entire system before the development of code takes place (Geeksforgeeks, 2025).

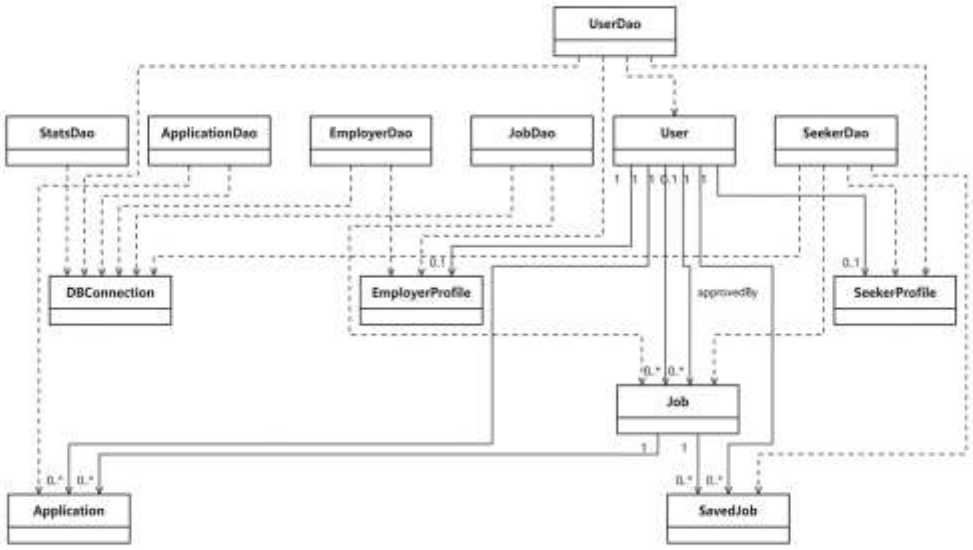


Figure 31 Class Diagram

5 Dao

5.1 UserDao

1. registerUser()

Functionality:

This method is used to register a new user to the system and add role specific profile information. It ensures that both user creation and profile creation (SEEKER or EMPLOYER) are governed by a single transaction, keeping each other in sync in multiple tables.

Input:

- User user : Contains user basic details like full_name, email, password, role, location and dob
- SeekerProfile seekerProfile: Contains seeker-specific details like education, skills, experience_year and resume_path used only if role is seeker.
- EmployerProfile employerProfile: Contains employer-specific details like company_name, company_category, company_address, company_city, company_description and contact_person used only when role is employer.

Output:

- True: User and profile successfully registered.
- False: Registration failed due to error or rollback.

Working Process:

The method starts by turning off auto-commit, which means all operations are used as one transaction. Firstly, the basic information of the user is securely stored, including their full name, email, password, role, location and date of birth. The password is first hashed using BCrypt for strong security, before being stored. After user record is added, system fetches the user ID generated. Additional information for the profile is then added, such as education, skills and experience details for a SEEKER and company information like name, category etc. and description for an EMPLOYER, depending on the role. In case of errors, the task will be rolled back to keep the database consistent. At last, auto-commit is turned back on and the procedure returns a success or failure return code.

Key Features:

- Transaction management for atomic operations.
- Password hashing with BCrypt for secure storage.
- Role based profile creation.
- Rollback mechanism endures data consistency.

Screenshot

Figure 32:RegisterUser

2. getUserByEmailAndPassword()

Functionality:

This method is used to authenticate the user while logging in. This is done by fetching the user from the database through searching the provided email and determining if the account exists by comparing the password provided by the user to the hash of that password saved in the database. This function returns an instance of User if credentials match or null if they don't.

Input:

- String email : The email address entered by the user.
- String password: The plain text password entered by the user.

Output:

- User Object: Returned if authentication is successful.
- Null: Returned if authentication fails (invalid email or password)

Working Process:

Starting with a SQL query, the system pulls a user entry using the given email address. Through a PreparedStatement, execution happens securely, blocking potential SQL injection risks. When data appears, the encrypted password comes out of the result set directly. With BCrypt in play, the typed password faces verification against that saved hash. Success leads to transforming raw fields into a User instance, handed back immediately afterward. Should the password fail verification or the system locate nothing, a null value gets returned by the method. When errors occur mid-process, they're intercepted - then recorded - to keep the program running smoothly.

Key Features:

- Secure password verification using BCrypt.
- Prevent SQL injection by using PreparedStatement.
- Returns a fully mapped User object upon successful login.
- Handles exception gracefully to maintain application stability.

Screenshot

UserDao.java

```
public User getUserByEmailAndPassword(String email, String password) {
    User user = null;
    try {
        String sql = "SELECT * FROM users WHERE email = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString( parameterIndex: 1, email);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    String hashedPassword = rs.getString( columnLabel: "password");
                    if (PasswordUtils.checkPassword(password, hashedPassword)) {
                        user = mapRow(rs);
                    }
                }
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return user;
}
```

Figure 33: GetUserByEmailAndPassword

3. getUserById()

Functionality:

This method obtains information about a particular user from the database by using his or her unique ID number and is mainly used when the software program wants to retrieve complete information about a particular user.

Input:

- int userId: The unique identifier of the user stored in the database.

Output:

- User object: Returned if a matching record is found.
- Null: Returned if no user exists with the given ID or if an error occurs.

Working Process:

The method start with creating an SQL query for selecting a user record with the ID given. The userId parameter is safely bound with a PreparedStatement which helps in to prevents SQL injection. The query is run and, if it is successful, the result set is converted to a User object with the help of the mapRow() helper method and if there is no record, the method returns null. Exceptions during execution are caught and recorded for stability of the application.

Key Features:

- Prevents SQL Injection using PreparedStatement.
- Efficient retrieval of user details by primary key.
- Returns a fully mapped User object for further use.

Screenshot

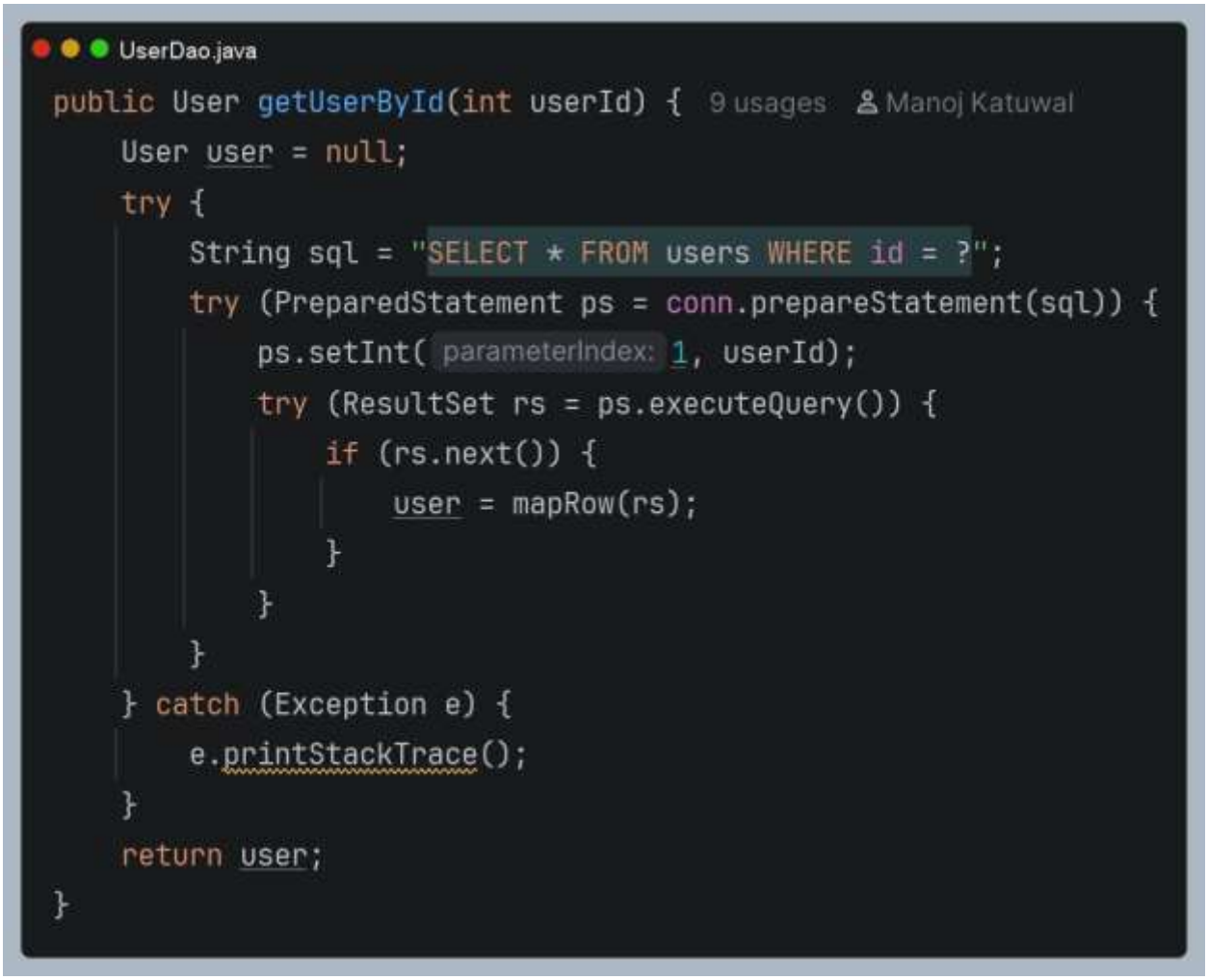


Figure 34:GetUserById

4. emailExists()

Functionality:

This method is used to check whether a particular email ID is registered with the system or not and applied when a new user registers himself.

Input:

- String email: The email address to be verified.

Output:

- True: If the email already exists in the database.
- False: If the email does not exist or an error occurs.

Working Process:

The methods prepare the query for the SQL of an email address entered by a user. The supplied email address will be used on the preparedStatement object in the query, thus preventing SQL injection. Then the query will be run and analyzed for its results and if the record does exist in the database, then the method will return true, indicating that other user has this email address. Otherwise, the method will return false.

Key Features:

- Prevents duplicate user registrations
- Secure query execution using PreparedStatement.

Screenshot

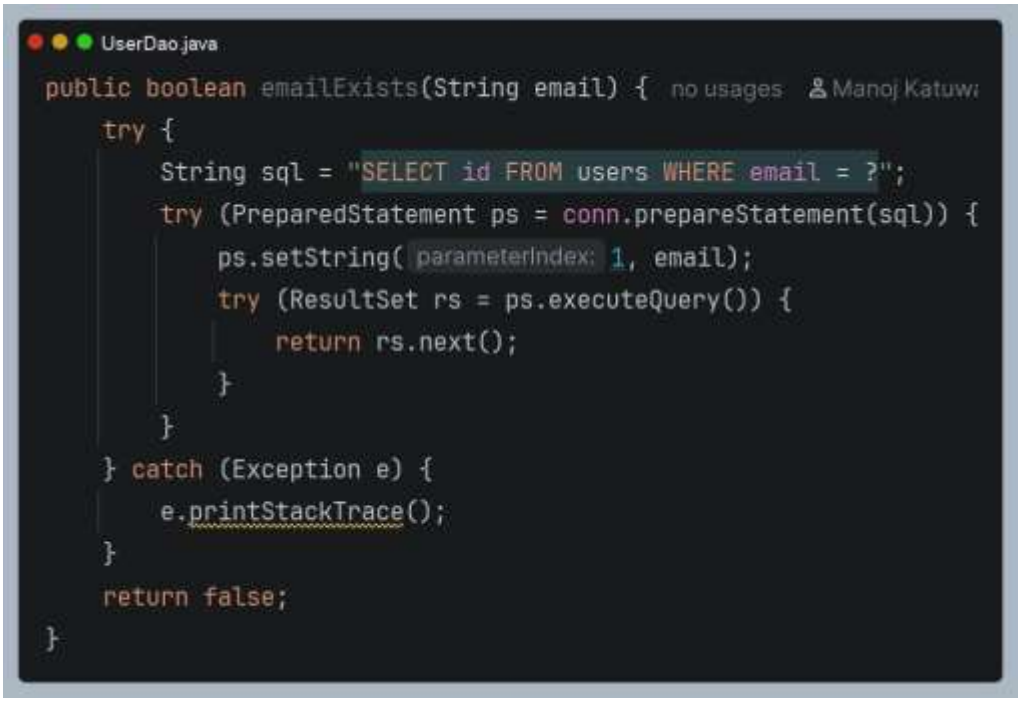


Figure 35:EmailExists

5. getUsersByFilter()

Functionality:

This method would fetch users in the system optionally filtered by their particular role. Users belonging to that particular role EMPLOYER OR SEEKER only would be fetched in case a role is provided.

Input:

- String role: The role to filter users.

Output:

- List<User>: A list of users objects matching the filter criteria.
- Empty list: If no users match the filter or an error occurs.

Working Process:

The method begins with the creation of a SQL query. If there is a role then the query will contain a WHERE clause and which will filter the users based on the user role and if there is no role then it will select all users. The use of PreparedStatement helps from preventing against SQL injection attacks because of the binding of the role. After execution of the query, iteration through the result set will start. The mapRow() helper function maps the records in the result set to the user objects. Lastly, a list of users is returned that contains all of the records processed.

Key Features :

- Maps each record into a User object for easy use in controllers or services.
- Exception handling ensures stability even if error occurs.
- Secure query execution with PreparedStatement.

Screenshot:

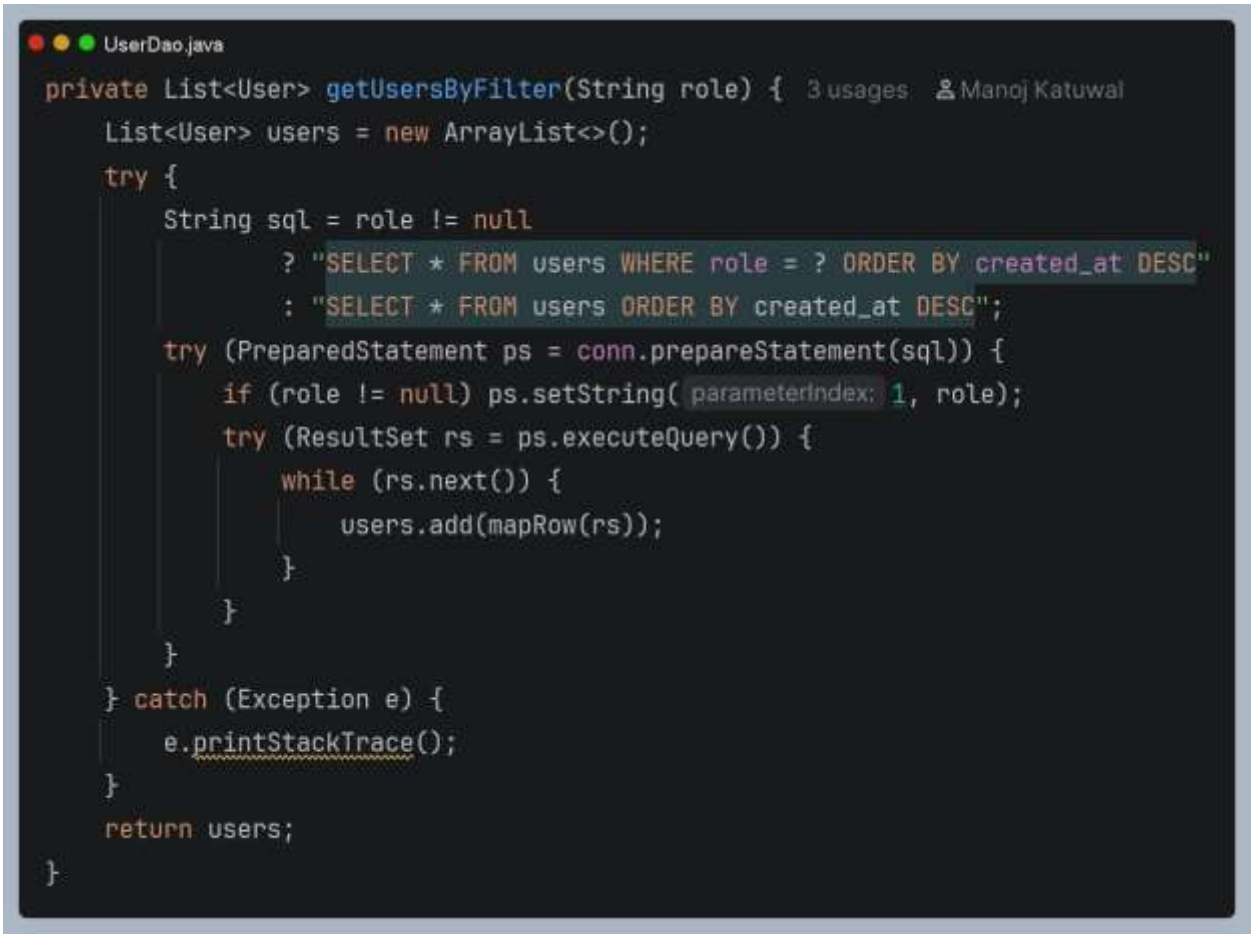


Figure 36:GetUserByFilter

6. updateUserProfile()

Functionality:

This approach is used to modify an already existing user's profile information in the system. It can be edited to change the important information like full name, phone number, location and date of birth. The update is then done in a secure way in this way, with a prepared statement, to keep data intact and safe from SQL injection attacks.

Input:

- User user: Contains updated details including id, full_name, phone , location , dob

Output:

- True: If the user profile was successfully updated.
- False: If the update failed or an error occurred.

Working Process:

The approach starts by creating an SQL update statement to change the profile information of the user, depending on the ID of the particular user. The values to be updated are then securely bound into a PreparedStatement which helps also to prevent from SQL Injection. When the date of birth is given then it is automatically stored as a SQL Date object or otherwise the field is set to NULL. The statement is run and the method tests to determine if a row was changed. The method returns true value if the at least one row is updated and if there are no rows affected or if an exception is thrown then the method returns false. Any exceptions are caught and recorded to ensure that the application remains stable.

Key Features:

- Supports optional fields mean date of birth can be null.
- Returns a clear success or failure status for reliable feedback.
- Exception handling ensures smooth application flow.

Screenshot

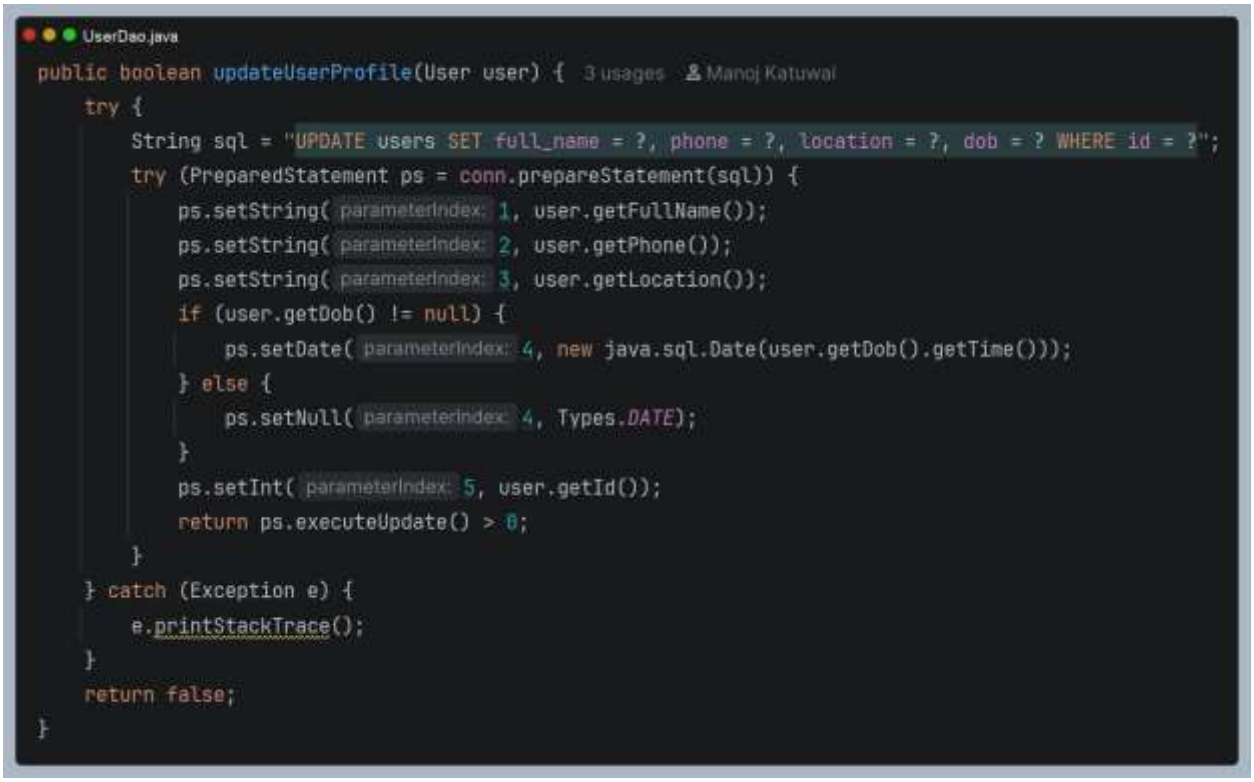


Figure 37:UpdateUserProfile

7. updateUserStatus()

Functionality:

This approach is used to change the status of the user in the system. It is usually used by administrators to accept, deny or change a user's account status. The update is made securely with the use of a prepared statement to maintain data integrity and preventing limitations of SQL injection.

Input:

- int userId: The unique identifier of the user whose status needs to be updated.
- String status: The new status value e.g. PENDING, APPROVED and REJECTED

Output:

- True: If the user status was successfully updated.
- False: If the update failed or an error occurred.

Working Process:

The initial step of the method is to create an SQL statement to update the user record whose userId is passed in and change their status field. To overcoming SQL injection we use Prepared statement . The statement is run and the method determines if a row was affected. The method returns true if the addition or modification of at least one row was made. If neither any rows changed nor an exception occurred, the method returns false. Anything after this is caught and logged for stability.

Key Features:

- Allows administrators to manage user account status.
- Returns a clear success and failure status for reliable feedback.
- Exception handling ensures smooth application flow.

Screenshot

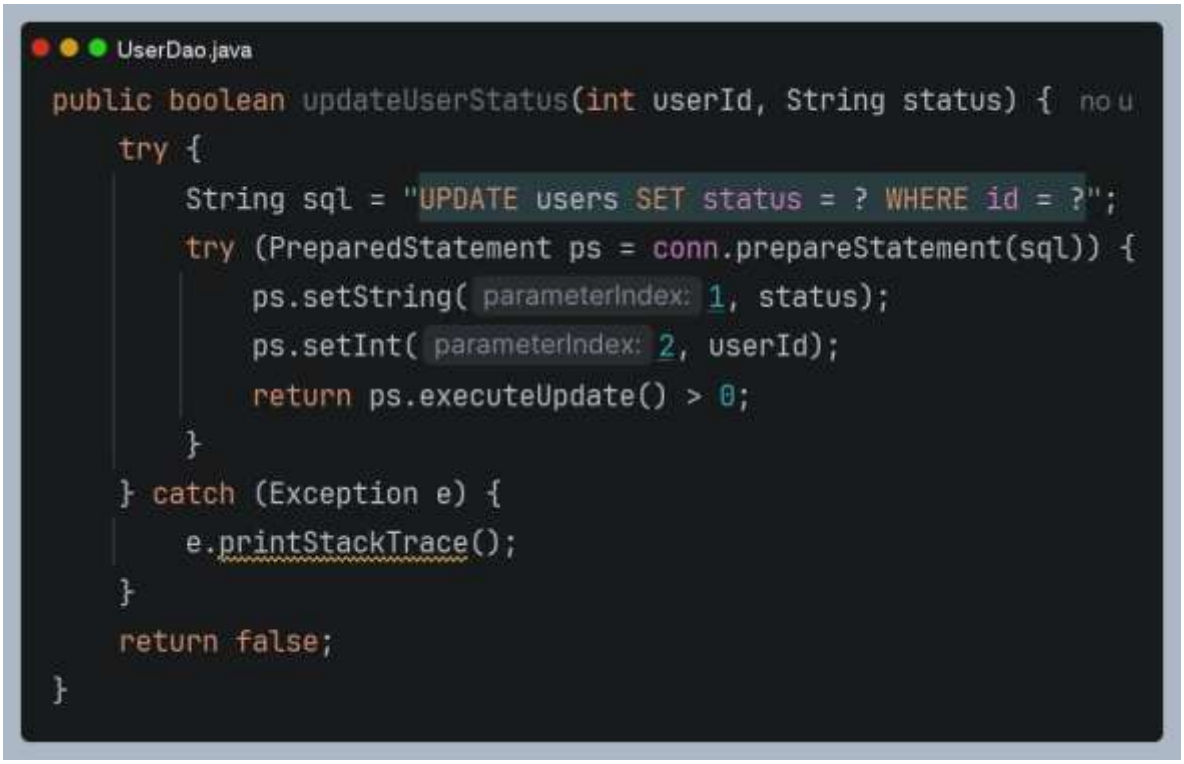


Figure 38:UpdateUserStatus

8. approveUserIfPending()

Functionality:

This method is used to approve a user account, if the user is in PENDING status AND the role provided equals the user's role. It provides for conditional approval, so that users who accidentally become approved are not approved.

Input:

- int userId: The unique identifier of the user to be approved,
- String role:The role of the user e.g. SEEKER, EMPLOYER and ADMIN

Output:

- True: If the user status was successfully updated to approved.
- False: If the updated failed, the user was not pending or an error occurred.

Working Process:

The method prepares an update statement to set the user's status to APPROVED if the status is PENDING and role is the one given. To prevent from SQL Injection, we use PreparedStatement. After Executing the prepared statement, it checks if there are any rows being updated. Returns true if there are rows being updated (user was pending status) and false otherwise (user was not pending or role mismatch). Catches any exceptions caused during execution and notifies the user.

Key Features:

- Conditional approvals only apply to users with PENDING status.
- Role validation ensures correct user type is approved.

Screenshot

```
public boolean approveUserIfPending(int userId, String role) {  2 usages  & Manoj Kafuwal
    try {
        String sql = "UPDATE users SET status = 'APPROVED' WHERE id = ? AND UPPER(TRIM(role)) = ? AND UPPER(TRIM(status)) = 'PENDING'";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt( parameterIndex: 1, userId);
            ps.setString( parameterIndex: 2, role.toUpperCase());
            return ps.executeUpdate() > 0;
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return false;
}
```

Figure 39:ApproveUserIfPending

9. deleteUser()

Functionality:

This methos will delete the definition of a user from the system that has been obtained by his/her unique id. In general, it is used by the administrators to manage accounts and make sure that possible undesirable or inactive ones can be eliminated safely.

Input:

- int userId: The unique identifier of the user to be deleted.

Output

- true: If the user was successfully deleted.
- False: If the deletion failed or an error occurred.

Working Process:

This function will construct an SQL query that will delete the row from user table, it will use the userId passed to the function. The parameter will be set using PreparedStatement (to avoid SQL injection attacks). Then it will execute the query and return true if the query affects at least one row, else if does not affect on table (may be the user was not exist or some other error occurred).

Key Features:

- Secure deletion using PreparedStatement.
- Ensures that only the specified user record is removed.
- Exception handling prevents application crashes.

Screenshot

Figure 40:DeleteUser

10. deleteUserWithRole()

Functionality:

This method removes a user record from the system by using their user id and role. Only users with the selected role are deleted and the system explicitly will not delete users, ADMIN.

Input:

- int userId : The unique identifier of the user to be deleted.
- String role: The role of the user example SEEKER and EMPLOYER.

Output:

- True: If the user was successfully deleted.
- False: If the deletion failed then the role did not match or the user was an ADMIN.

Working Process:

This prepares this SQL delete statement, which deletes the user record only if the userId parameters is equal to the field value, and the role parameter is equal to the value. Also, it is protecting systems administrator account from being deleted by including a additional condition that finds and deletes the record only if the role is not equal to Admin value and leaving the record untouched. It using a PreparedStatement as its object, which binds the 'id' and the 'role' parameters to the statement. Then, it executes the Query using the executeUpdate() method, followed by (name has rows affected). If the number of rows affected is greater than zero, then it returns true; else, it is returning false because no system administrator is being deleted or the 'role' parameter is not matched to the value. Also catch exception and logged to maintain application stability.

Key Features:

- Role based validation ensures only the intended user type is deleted.
- Explicit safeguard against deleting ADMIN accounts.
- Exception handling prevents application crashes.

Screenshot

Figure 41:DeleteUserWithRole

5.2 JobDao

1. createJob()

Functionality:

This method is used to add and publish a new job to the system. It fills the jobs table with information about the job supplied by the employer. The job is saved by default as pending and will be approved by an administrator.

Input:

- Job job: Contains job details including employerId, title , description, category ,locationCity , salaryRange ,jobType and deadline.

Output:

- True: If the job was successfully created and inserted into the database.
- False: If the insertion failed or an error occurred.

Working Process:

The first operation of the method is to create an SQL insert for inserting a new job into the jobs table. For security reason, the job details are securely bound with a PreparedStatement to avoid SQL injections. The job related information including the employer id, the position, the description, the category, the location, the minimum salary, the maximum salary, the type and the date are all used as parameters. The status is automatically changed to pending, which will be waiting for activation. Then it execute update statement then the method will decide whether the primary key has been affected by it. If yes the method return true and if not return false. The trapped exception in the run will also be recorded to guarantee the system's well functioning.

Key Features:

- The method secure job creation using PreparedStatement.
- The method set default status to pending for admin approval workflow.
- The method supports multiple job types like full-time,part-time, contract and internship.

Screenshot

```
public boolean createJob(Job job) {
    try {
        String sql = "INSERT INTO jobs(employer_id, title, description, category, location_city, salary_range, job_type, deadline, status) VALUES(?,?,?,?,?,?,?,?,?)";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, job.getEmployerId());
            ps.setString(2, job.getTitle());
            ps.setString(3, job.getDescription());
            ps.setString(4, job.getCategory());
            ps.setString(5, job.getLocationCity());
            ps.setString(6, job.getSalaryRange());
            ps.setString(7, job.getJobType());
            ps.setDate(8, new java.sql.Date(job.getDeadline().getTime()));
            ps.setString(9, "pending");
            return ps.executeUpdate() > 0;
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return false;
}
```

Figure 42:Create Job

2. getJobById()

Functionality:

This methos or features fetches a particular job record from the system with the job ID. It is often utilized with displaying detailed information about a job to a job seeker or when an administrator wants to look at a specific job posting.

Input:

- int jobId: The unique identifier of the job stored in the database.

Output:

- Job object: Returned if a matching record is found.
- Null: Returned if no job exists with the given ID or if an error occurs.

Working Process:

At the beginning, a SQL query is prepared to select a job using jobId. To avoid possible SQL injections, PreparedStatement object is used to make the query "safe" by setLong() method on the jobId parameter. Then, the query can be fired by executing executeQuery() method on the statement; the method order the database to look for the resulting specified record. If a value is returned, ResultSet object is used as parameter of the helper method mapRow(), which convert the result to a java object; otherwise, the method return null.

Key Features:

- The method securely execute query using PreparedStatement.
- The method efficiently retrieve job details using primary key(job_id)

Screenshot

Figure 43 GetJobById

3. getDistinctCategories()

Functionality:

This method or features will fetch a list of different job categories from the system. Only categories that are associated with a job that has an approved status are displayed, providing seekers with valid and active categories.

Input:

- The method automatically queries the database for distinct categories of approved jobs.

Output:

- List<String>: A list of distinct job categories.
- Empty list: If no approved jobs exists or an error occurs.

Working Process:

The code constructs a query that selects all uniquely presented job categories from the jobs table, in which the jobs are approved and the value is neither null nor empty. Then, the query is called in a PreparedStatement and executed, everything in the result set is collected in a List (categories). After the whole result set is read, the List of categories is returned, catching and logging any exceptions.

Key Features:

- The method retrieves distinct categories to avoid duplicates.
- The method filter approved jobs to ensure relevance.
- The methods ignores null or empty category values.

Screenshot

Figure 44 GetDistinctCategories

4. getDistinctLocations()

Functionality:

This process gets the list of unique job locations from the system. Only jobs with an approved status are displayed, so that job seekers will only see valid and active job postings.

Input:

- The method automatically queries the database for distinct locations of approved jobs.

Output:

- List<Strng>: A list of distinct job locations.
- Empty List: If no approved jobs exist or an error occurs.

Working Process:

This method prepares a SQL statement to get all the unique job locations from the jobs table. It excludes jobs with not-approved at the status or a null or empty value for location. It uses a PreparedStatement to execute it and for SQL injection. It loops over the result set of the query and adds each location in a list. It finally returns the list of locations. If anything goes wrong it catches and logs the exception and returns an empty list.

Key Features:

- The method retrieves distinct locations to avoid duplicate.
- The method filters approved jobs to ensure relevance.
- The methos ignores null or empty location values.

Screenshot

Figure 45 GetDistinctLocations

5. searchApprovedJobs()

Functionality:

This approach will look for any approved job within the system. It allows for flexible filtering by keyword, district, categories and locations. The results are displayed in reverse chronological order, that is, the latest posted jobs are at the top.

Input:

- String keyword: Optional search term applied to job title, category and description.
- String distinct: Optional filter applied to job location.
- String [] categories : Optional filter for specific job categories.
- String [] locations: Optional filter for specific job locations.

Output:

- List <Job>: A list of approved jobs matching the search criteria.
- Empty List : If no jobs match or an error occurs.

Working Process:

The method first prepare a string sql based on a prepared statement. This statement join the selection on two tables where the join is done on the job_table. Then it prepares the string to select only the approved jobs. Then based on the parameter passed in in the method it modify the sql query to add more criteria in the where clause. The criteria include the keyword if not null matching with the title, category or description. The second criteria include the district in which the job is. It also modifies the sql string based on the category and the location using the appendInFilter helper method. It then prepares a statement and add the parameters to the statement with a PreparedStatement parameter. It then executes the query and iterate through the result set and populate and add Job objects to an array list using the mapRow() helper method. It then returns the list of jobs. If an exception occurs it catch the exception and add to the logger.

Key Features:

- The method searches only approved jobs for relevance.
- The method filters by keyword, distinct, categories and locations.

- The methods provides results ordered by posting fate.
- The methods returns a clean list of Job objects for use in search results or dashboards.

Screenshot

Figure 46:SearchApprovedJobs

6. getJobsByEmployer()

Functionality:

This approach will return all of the jobs that a specific employer has listed. It can be used in an employer's dashboards or admin panels to view and control a job posted with the employer's account.

Input:

- int emploterId : The unique identifier of the employer whose jobs need to be retrieved.

Output:

- List <Job> : A list of jobs posted by the employer.
- Empty List : If no jobs exist for the employer or an error occurs.

Working Process:

This build query defaults to selecting all jobs where the employer_id is the parameter value. It uses a PreparedStatement object (initially null) to safely bind the method parameter to the first statement parameter rather than concatenating a prepended value into the method arguments. The query sorts the results and places the most recent Job objects by posted_at first. It then iterates through the result set and transform each row in the set to a Job object using the helper method mapRow() and adds the object to an ArrayList. The list of objects is returned upon completion and at last the query has been executed and is surrounded by a try/catch block in case an SQLException is thrown, in which case the empty list is returned.

Key Features:

- The method retrieves jobs specific to an employer.
- The method returns a clean list of job objects for use in dashboards or reports.
- Exception handling ensures smooth application flow.

Screenshot

Figure 47 GetJobsByEmployer

7. getRecentJobsByEmployer()

Functionality:

This approach will show a handful of the newest jobs from a particular employer. It is good for employer dashboards or admin panels where you don't want to see all the detail of a job, but just the latest postings.

Input:

- int employerId: The unique identifier of the employer whose jobs need to be retrieved
- int limit: The maximum number of recent jobs to fetch.

Output:

- List<Job>: A list of recent jobs posted by the employer.
- Empty List: If no jobs for the exist for the employer or an error occurs.

Working Process:

Execution of the query takes place and the result set is traversed. All the records are mapped into Job object by using mapRow() utility method and added to the list. When the iteration process is performed, the list is retrieved. If present exception occurs during execution, it is caught and logged properly, and an empty list is returned instead.

Key features:

- The method retrieves jobs specific to an employer.
- The method supports limiting the number of results for efficiency.
- The method returns a clean list of Job objects for use in dashboard or summaries.

Screenshot

Figure 48 GetRecentJobByEmployer

8. getRecentPendingJobs()

Functionality:

This method or functions returns up to a certain number of the latest jobs which are pending in a pending state. It's a great tool for administrators who must examine and approve new jobs postings before they are made available to job seekers.

Input:

- int limit : The maximum number of pending jobs to fetch.

Output:

- List<Job>: A list of recent pending jobs.
- Empty list : If no pending jobs exist or an error occurs.

Working Process:

The operation begins by creating a SQL statement to fetch all those jobs whose status is 'pending'. The statement is written in a way to arrange the output according to posted_at, from latest to oldest in order to show the latest job postings at first. Then a LIMIT clause is added to limit the output to some value. A PreparedStatement is to be used to protect the parameter from any kind of SQL injection attacks. After the query execution is done, the ResultSet is iterated. The method mapRow() converts each row to a Job object and adds it to the list. An empty list is returned in case of any exception.

Key Features:

- The method retrieves only jobs with pending status.
- The method returns a clean list of Job objects for admin review.
- Results ordered by posting date (posted_at DESC).

Screenshot

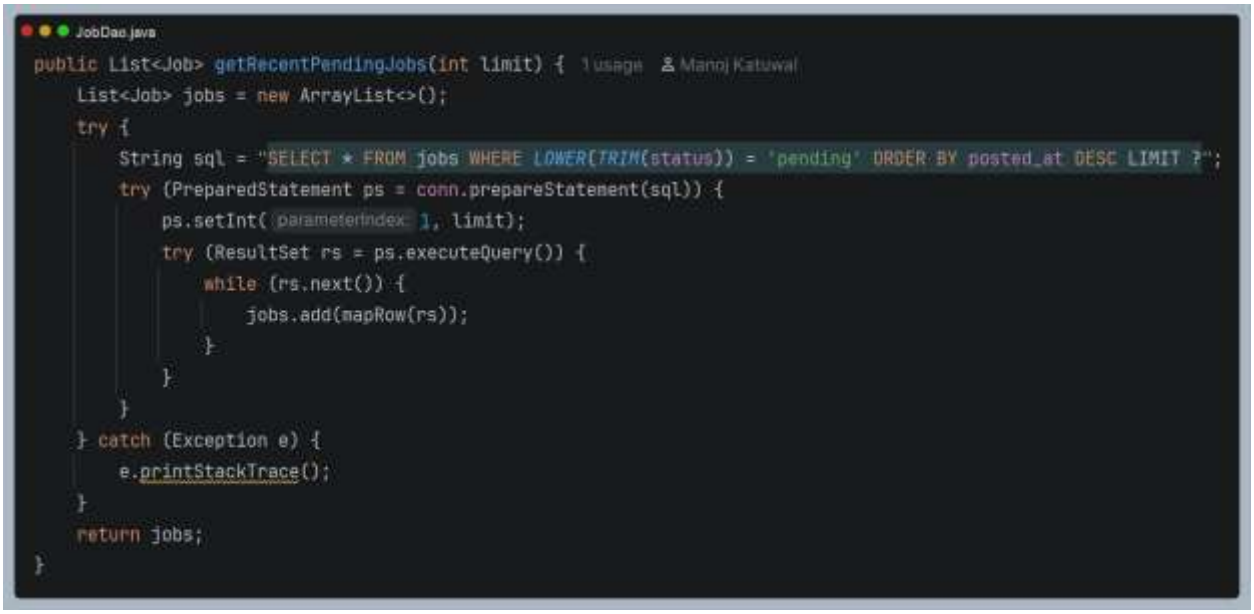


Figure 49 GetRecentPendingJobs

9. getJobsByCategory()

Functionality:

The method filters and gets all of the jobs that are already approved and have a particular category. It is frequently utilized in job search filters, category-based viewing or reporting features with the users seeking jobs or users that are administering their search engines to show their jobs arranged by category.

Input:

- String category : The category name to filter jobs by like IT, Finance, Marketing.

Output:

- List<Job> : A list of jobs in the specified category with approved status.
- Empty list : If no jobs exist in the category or an error occurs.

Working Process:

This technique results in generation of the SQL query that can retrieve jobs which belong to the same category and status and which have been approved. The PreparedStatement technique is applied to guard against the SQL injections, while jobs will be sorted in the descending order depending on the date of posting, thus making the most recent postings displayed at first. Every row within the ResultSet will be converted into the Job object with the help of the mapRow() method and the resulting objects will be added to the list to be returned from this method. Some exception will be logged, while the empty list will be returned.

Key Features:

- The method retrieves jobs filtered by category.
- It ensures only approved jobs are returned.
- The methos returns a clean list of Job objects for use in category filters or dashboards.

Screenshot

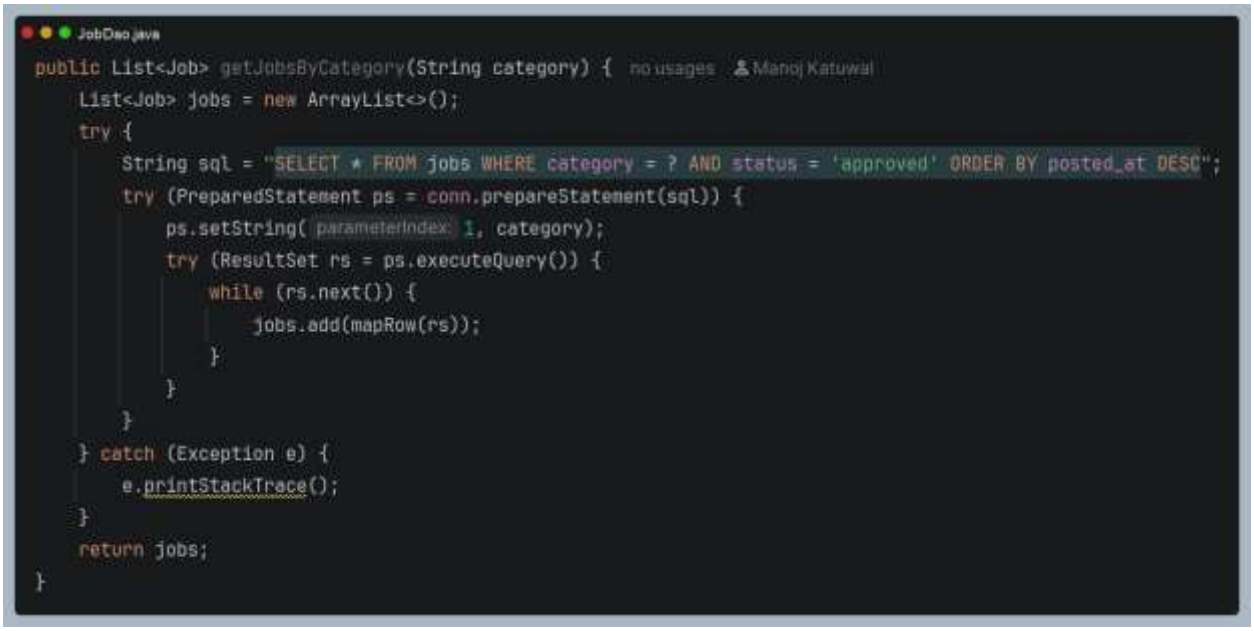


Figure 50 GetJobsByCategory

10. deleteJobByEmployer()

Functionality:

This method removes a record from the job table in the system using the unique id of the job and the ID of the employer. Only an employer who posted the job can delete it - role-based control and no unauthorized deletion.

Input:

- int jobid : The unique identifier of the job to be deleted.
- Int employerId: The unique identifier of the employer to be deleted.

Output:

- true: If the job was successfully deleted.
- False: If the deletion failed then the employer did not match, or an error occurred.

Working Process:

This method will delete a record from the job table in the system by the unique id of the job and the ID of employer. Only employer who posted job can delete the job (role based control; no deletion by others).

Key features:

- Role based validation ensures only the job owner can delete their pasting.
- Returns clear success and failure for reliable feedback.

Screenshot

Figure 51 DeleteJobByEmployer

5.3 ApplicationDao

1. deleteJobByEmployer()

Functionality:

This method will remove a job record from the system, but only when stipulated by the deletion request from the employer who originally posted the record's job. This allows role based control and also enables that users who are not allowed to delete their own jobs won't be able to delete other users jobs.

Input:

- int jobId: The unique identifier of the job to be deleted.
- int employerId: The unique identifier of the employer who owns the job.

Output:

- true : If the job was successfully deleted.
- False: If the deletion failed then the employer did not match, or an error occurred.

Working Process:

The method here would first generate a SQL DELETE statement that would delete the job record only if the job id and employer id correspond to the input values. PreparesStatemet prevents SQL injection by binding parameters. After executing the statement the method gets the count of the rows deleted by the statement and if there is deletion of at least one row, the method returns true or false.

Key Features:

- Secure deletion using PreparedStatement.
- Role based validation ensures only the job owner can delete their posting.
- Clear success/failure return value for reliable feedback.

Screenshot

Figure 52 CreateApplication

2. has Applied()

Functionality:

The purpose of this method is to determine if a given user has already applied for a given job. It has the ability to prevent duplicate applications (in the job application workflow) and helps with the validation process.

Input:

- int seekerId: The unique identifier of the job seeker.
- Int jobId: The unique identifier of the job.

Output:

- True: If the seeker has already applied for the job.
- False: If no application exists or an error occurred.

Working Process:

This algorithm creates an SQL statement using the `SELECT COUNT(*)` clause in order to calculate the number of all records from the `applications` table where the column `seeker_id` equals the entered one. A use of the `PreparedStatement` is applied in order to protect from any SQL injections. As, the query executes successfully, a check is made on the `ResultSet`. If a record is returned, the `total` column is read.

Key Features:

- Duplicate prevention in job applications.
- Secure query execution using PreparedStatement.
- Clean Boolean returns value for easy integration in workflows.
- Supports job seeker dashboards and application validation.

Screenshot

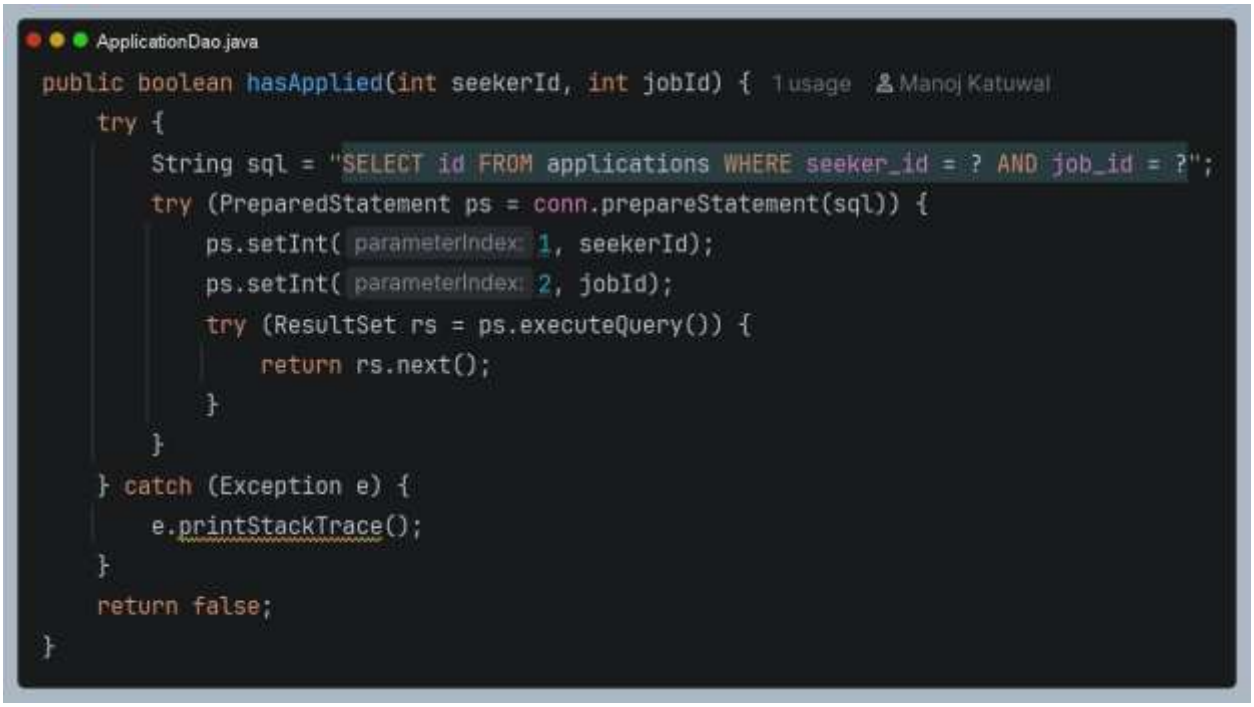


Figure 53:HasApplied

3. getApplicationBySeeker()

Functionality:

With this Method, one can access all applications made by a particular job seeker. The technique gives the entire list of applications arranged according to the dates when the applications were made.

Input:

- int seekerId : The unique identifier of the job seeker.

Output:

- List<Application>: A list of application objects belonging to the seeker.

Working Process

A procedure created a SQL SELECT query that selects all rows from the applications table where seeker_id will be equal to the passed-in value. A results will be sorted in descending order based on the applied_at column, meaning that applications with more recent dates appear at the top and passed-in value is then bound into the PreparedStatement query to prevent SQL injection attacks. The query is executed, and for each row, the mapRow() method maps the row into an Application object. All Application objects are stored in a List and returned.

Key Features:

- Application history retrieval for job seekers.
- Secure query execution using PreparedStatement.
- Exception handling for robust performance.
- Supports seeker dashboards and profile management workflow.

Screenshot

Figure 54 GetApplicationBySeeker

4. getApplicationCountBySeeker()

Functionality:

The method is used to determine the total number of applications sent by a particular job seeker. This approach allows for an easy method of presenting statistics on applications, which may be helpful for dashboards and reports.

Input:

- int seekerId : The unique identifier of the job seeker.

Output:

- Long: The total number of applications submitted by the seeker.

Working Process:

This algorithm creates an SQL statement using the `SELECT COUNT(*)` clause in order to calculate the number of all records from the `applications` table where the column `seeker_id` equals the entered one. A use of the `PreparedStatement` is applied in order to protect from any SQL injections. As, the query executes successfully, a check is made on the `ResultSet`. If a record is returned, the `total` column is read.

Key Features:

- Application statistics for dashboards and reports.
- Secure query execution using PreparedStatement.
- Exception handling ensures stability.

Screenshot

ApplicationDao.java

```
public long getApplicationCountBySeeker(int seekerId) {  1 usage   Manoj Katuwal
    try {
        String sql = "SELECT COUNT(*) AS total FROM applications WHERE seeker_id = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt( parameterIndex: 1, seekerId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next())
                    return rs.getLong( columnLabel: "total");
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}
```

Figure 55:GetApplicationCountBySeeker

5. getApplicationCountByEmployer()

Functionality:

The method or functions helps determine the total number of applications that have been sent by applicants to a certain employer's various job postings. This is a fast way of determining the level of employer engagement.

Input:

- int employerId : The unique identifier of the employer.

Output:

- long: The total number of applications submitted to jobs owned by the employer.

Working Process:

The method or function creates an SQL SELECT COUNT(*) statement that will join the applications table and the jobs table. This guarantees that only those applications which have been posted for jobs by the specified employer will be counted then the PreparedStatement will ensure that the employerId is safely bound.Therfore preventing any SQL injection attacks. The statement is then executed and the result set is scanned and if it matches the total number of applications will be retrieved through the use of an alias named total.

Key Features:

- Employer application statistics for dashboards and reports.
- Secure query execution using PreparedStatement.
- Exception handling ensures stability.

Screenshot

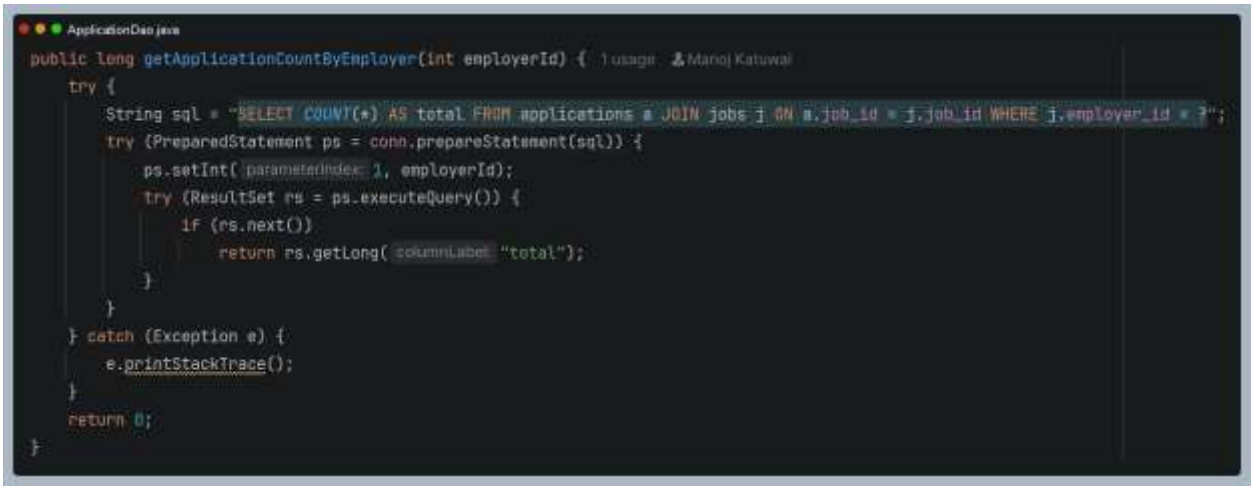


Figure 56 GetApplicationCountByEmployer

6. getApplicationsForEmployer()

Functionality

This methos helps in retrieving all applications for jobs belonging to a certain employer. This gives the employer an insight into the details of the applicant like the job title applied for, name of the applicant, and email id of the applicant.

Input

- int employerId : The unique identifier of the employer.

Output:

- List<Application>: A list of application objects enriched with job and secure details.

Working Process:

This method constructs a SQL SELECT statement that joins all three tables applications, jobs, and users. This guarantees that each row from the applications table will be joined to the relevant job title and seeker information. A PreparedStatement object is used for safely binding the employerId parameter to avoid any SQL injection attacks. The execution of the query and iteration of the resulting ResultSet are done. In addition, for each row, mapRow() method is called to create an Application object from the retrieved data. Further properties like jobTitle, seekerName, and seekerEmail are added by explicitly setting them from the ResultSet.

Key Features:

- Employer application retrieval with the detailed applicant information.
- Secure query execution using PreparedStatement.
- Applications orders by submission date for easy review.
- Exception handling ensures robust performance.

Screenshot

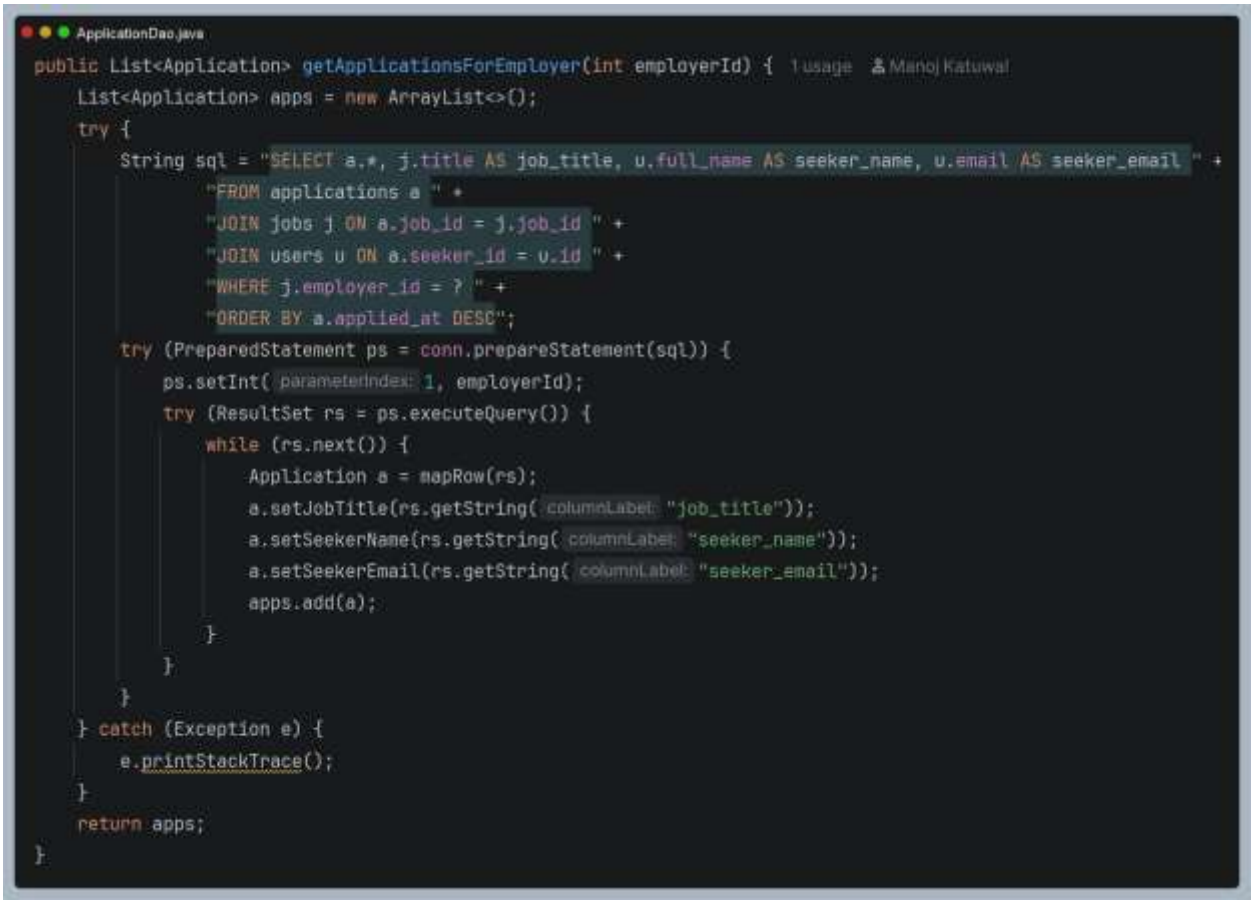


Figure 57 GetApplicationForEmployer

7. updateApplicationStatus()

Functionality:

With this approach, an employer can modify the status of a job application from pending to reviewed, accepted, or rejected which makes sure that only the owner of the job is able to make such changes and that no one else can do it.

Input:

- int applicationId: The unique identifier of the application.
- String status : The new status to be assigned like accepted, rejected and reviewed.

Output:

- True: If the application status was successfully updated.
- False: If the update failed and the employer did not match an error occurred.

Working Process:

The approach will prepare an SQL UPDATE statement, which joins the applications table with the jobs table. This way, the update can be done only when the application is part of a job that has been advertised by the specified employer. A use of PreparedStatement will allow safe binding of the parameters in order to protect against SQL injection and query will set the new status and update the reviewed_at field with the current date and time (NOW()). Later the, running the query the method will check if there have been any changes made. If there is any any changes made it will return true otherwise it will return false.

Key Features:

- Role based validation ensures only the job owner can update application status.
- Secure updating using PreparedStatement.
- Clear success/ failure return value for reliable feedback.
- Exception handling prevents application crashes.

Screenshot

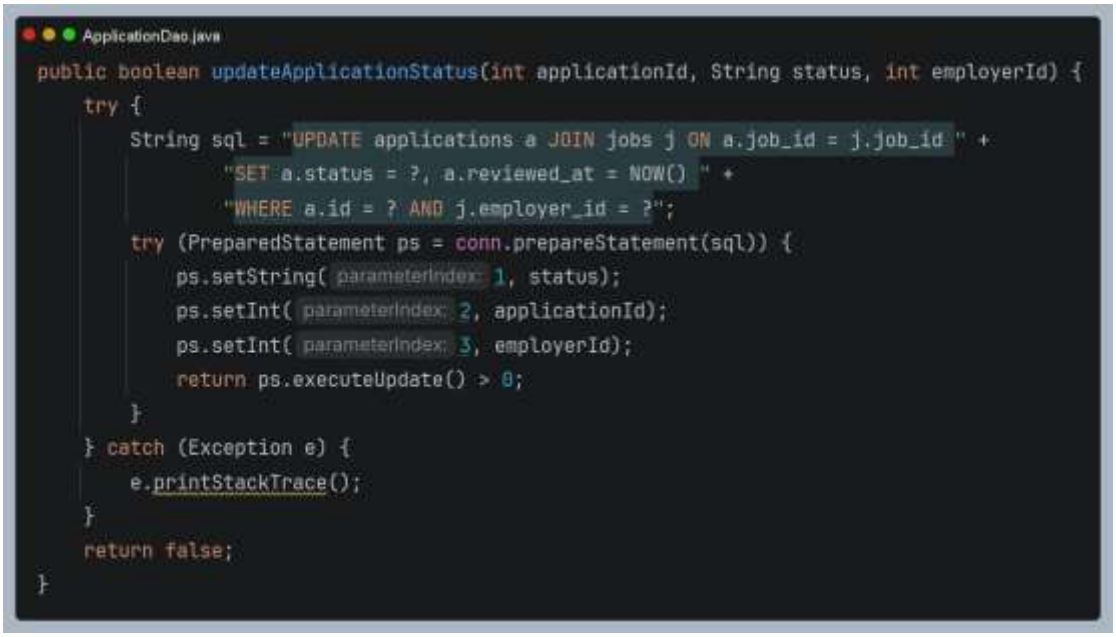


Figure 58 UpdateApplicationStatus

8. ensureResumePathColumn()

Functionality:

This utility function makes sure that the resume_path column is present in the applications table. If the resume_path column does not exist, then it will alter the table to add the column to it. This ensures backwards compatibility.

Input:

- none : The method runs internally without external parameters.

Output:

- void: No direct return value which ensures schema consistency.

Working Process:

The procedure obtains the DatabaseMetaData. It checks whether the column named resumepath exists on the applications table. If the column is present the method does not do anything. If not the statement SQL ALTER TABLE ADD VARCHAR(500) is executed where the resumepath is created and located after the cover_letter field. Thus making sure that the applications table includes the ability to storage the resume paths. The exceptions are handled and logged, so that if this action cannot be performed the application is not interrupted.

Key Features:

- Schema validation using DatabaseMetaData.
- Automatic column creation for backward compatibility.
- Exception handling ensures stability.
- Provides runtime errors when handling resumes.

Screenshot

Figure 59 EnsureResumeColumnn

5.4 EmployerDao

1. getEmployerProfiile()

Functionality:

The function fetches the profile details of the employer based on the user id. It makes it possible for the system to fetch data such as contact details and descriptive details about the company.

Input:

- int userId: The unique identifier of the employers user account.

Output:

- EmployerProfile: An object containing the employers profile details or null if no profile exists.

Working process:

This method prepares a SQL SELECT query which will retrieve all columns from employerprofile table, provided the userid corresponds to the given value. The PreparedStatement is used which safely binds the parameter to prevent any SQL Injection and the query is then executed and if a record is found, a new object of EmployerProfile class is created. All column values company_name, address, category, city, description and contact person are put into the object by calling setter methods of the EmployerProfile object. In case ,no record is found null is returned. Some exceptions are caught and an appropriate message is logged.

Key Features:

- Employer profile retrieval by userID.
- Secure query execution using PrepareStatement.
- Maps database fields into a structured Java Object.

Screenshot

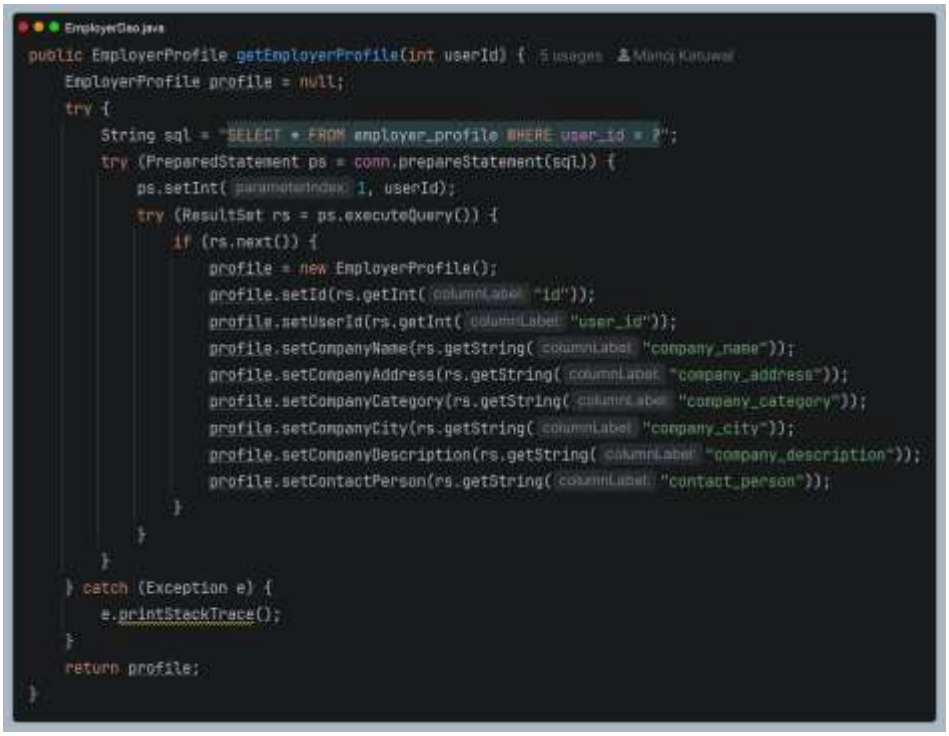


Figure 60:GetEmployerProfile

2. insertEmployerProfileIfMissing()

Functionality:

This approach ensures that each employer will have a profile entry in the system. When there is a profile entry corresponding to the current user id, then the function returns true. Otherwise, a new record will be added with default values, ensuring that an account of each employer has an associated profile entry.

Input:

- int userId: The unique identifier of the employer user account.

Output:

- true: If the profile already exists or was successfully inserted.'
- False: If the insertion failed or an error occurred.

Working Process:

This function/method first uses the getEmployerProfile(userId) method to check whether there is any profile. If there is any profile then the function/method will terminate by returning true value. However if there is no profile then the function/method will make a SQL INSERT query and then insert the details to employerprofile table with specified values of userid, default value of companyname and default value of companycategory by using PreparedStatement to avoid SQL injection and then it will return true value if successful or else it will return false value.

Key Features:

- Profile existence check before insertion.
- Secure insertion using PrepareStatement.
- Guarentes every employer has a valid profile record.
- Exception handling ensures robust performance.
- Supports employer dashboards and job posting workflows.

Screenshot

Figure 61 InsertEmployerProfileIFMissing

3. updateEmployerProfile()

Functionality:

This process is used to update the profile information of the employer in the system and can change the company name, address, category, city, description, and contact person through this process to keep their profiles updated.

Input:

- EmployerProfile profile: An object containing the employers updated profile details.

Output:

- True: If the profile was successfully updated.
- False: If the update failed or an error occurred.

Working Process:

This method is used to build an SQL UPDATE query for employerprofile table, where companyname, address, category, city, description, contact_person, set to values passed in the EmployerProfile object. PreparedStatement securely bind the parameters and avoid SQL Injection. The query execution then proceeds and the number of rows affected is examined. If more then one row has been updated then the function will return true, and false otherwise if no rows have been updated or any exception is being thrown, exceptions are caught and a message is logged for future analysis.

Key Features:

- Employer profile update with structured data.
- Secure update using PreparedStatement.
- Clear success/failure return value for reliable feedback.
- Exception handling ensures robust performance.

Screenshot

Figure 62 UpdateEmploerProfile

5.5 SeekerDao

1. getSeekerProfile()

Functionality:

The profile of the job applicant is fetched using this technique based on the unique user ID. The process enables the system to access personal details like skills, qualifications, experience, and resume file path.

Input:

- int userId: The unique identifier of the seeker user account.

Output:

- SeekerProfile: An object containing the seeker profile details or null if no profile exists.

Working Process:

A SQL SELECT query is formulated to select all columns of seekerprofile where userid equals value and a PreparedStatement is used which is safer against SQL injection. A record will be returned or a null, based on query execution result. New SeekerProfile object is created with retrieved column values using setters. If no record is retrieved, the method returns null and exceptions are handled and logged.

Key Features:

- Seeker profile retrieval by userId.
- Secure query execution using PreaparedStatement.
- Maps database felds into a structured Java object.

Screenshot

Figure 63 GetSeekerProfile

2. insertSeekerProfileMissing()

Functionality:

Through this approach we are ensuring that every job seeker will have a profile entry in the database. If a profile entry with the given user id already exists then this method will simply return a boolean true, else it will insert a record with default values.

Input:

- int userId : The unique identifier of the seekers user account.

Output:

- true: If the profile already exists or was successfully inserted.
- False: If the insertion failed or an error occurred.

Working Process:

First getSeekerProfile(userId) is called to see if the seeker profile already exists. If so then true is returned. If seeker profile doesn't exist then SQL INSERT statement is formed using userid as parameter and with dummy values for addresscity, skills, education, experienceyear set to zero and resumepath set to null. PreparedStatement is used to securely store the values to insert against the SQL statement to prevent against SQL injection. It is executed and returns true if 1 row is inserted. If no row is inserted or an exception occurs then returns false.

Key Features:

- Profile existence check before insertion.
- Secure insertion using PreparedStatement.
- Guarantees every seeker has a valid profile record.
- Exception handling ensures robust performance.

Screenshot

```
public boolean insertSeekerProfileIfMissing(int userId) {
    if (getSeekerProfile(userId) != null) {
        return true;
    }
    try {
        String sql = "INSERT INTO seeker_profile(user_id, address_city, skills, education, experience_year, resume_path) VALUES(?,?,?,?,0,NULL)";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt( parameterIndex: 1, userId);
            ps.setNull( parameterIndex: 2, Types.VARCHAR);
            ps.setString( parameterIndex: 3,  x: "");
            ps.setString( parameterIndex: 4,  x: "");
            return ps.executeUpdate() > 0;
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return false;
}
```

Figure 64 InsertSeekerProfileIfMissing

3. updateSeekerProfile()

Functionality:

This method updates a job seeker's profile information in the system. It allows seekers to modify their personal details such as city, skills, education, experience, and resume path, ensuring that their profile remains accurate and up-to-date for job applications and career management.

Input:

- SeekerProfile profile: An object containing the seeker updated profile details.

Output:

- True: If the profile was successfully updated.
- False: If the update failed or an error occurred.

Working Process:

This method prepares a SQL UPDATE statement to update the seeker_profile table. It binds new values for the address city, skills, education, experience year, and resume path with values provided by the SeekerProfile object. PreparedStatement securely binds the parameters and avoid SQL Injection. The query is executed and checks whether any row has been affected. The method returns true if there was one or more row updated. Returns false if there was no row updated or if any Exception is thrown. All exceptions are caught and logged to avoid system breakdown.

Key features:

- Seeker profile update with structured data.
- Secure update using PreapredStatement.
- Clear success and failure return value for reliable feedback.
- Exception handling insures robust performance.

Screenshot

Figure 65 UpdateSeekerProfile

4. saveJob()

Functionality:

This process makes it possible for the job seeker to bookmark a job listing for future reference. This is made possible without duplication of job listings by the use of INSERT IGNORE.

Input:

- int seekerId: the unique identifier of the job seeker.
- int jobId: The unique identifier of the job to be saved.

Output:

- true: If the job was successfully saved.
- False: If the save failed then the job was already moved or an error occurred.

Working Process:

Prepare a SQL INSERT IGNORE statement that should affect the saved_jobs table. This ensures that if the seeker already has saved the job then duplicate record will not be created. It binds the variables in a PreparedStatement to avoid SQL injection. The query will be executed. If one or more rows are affected then method will return true. If one row is not affected due to any of the reason (either job is already saved or some problem occurs in execution) then it will return false. Catches and log any exceptions for stable application.

Key Features:

- Duplicate prevention using INSERT IGNORE.
- Secure insertion with PreparedStatement.
- Clear success/failure return value for reliable feedback.
- Exception handling ensures robust performance.

Screenshot

SeekerDao.java

```
public boolean saveJob(int seekerId, int jobId) {
    try {
        String sql = "INSERT IGNORE INTO saved_jobs(seeker_id, job_id) VALUES(?, ?)";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt( parameterIndex: 1, seekerId);
            ps.setInt( parameterIndex: 2, jobId);
            return ps.executeUpdate() > 0;
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return false;
}
```

Figure 66 savejob

5. unsaveJob()

Functionality:

Using this approach, the job applicant is able to delete the job he/she had saved earlier on from the list of saved jobs. This helps ensure that applicants are able to manage their saved jobs properly.

Input:

- int seekerId : The unique identifier of the job seeker.
- Int jobId : The unique identifier of the job to be removed.

Output:

- True: If the job was successfully removed form the saved list.
- False: If the removal failed or an error occurred.

Working Process:

Prepare a SQL DELETE query to the savedjobs table for matching seekerid and job_id. PreparedStatement are used to securely bind parameters so as to prevent SQL injection. Query is then executed and method tests if the delete operation effected any rows, returns true if any row(s) was(were) deleted and false if none where deleted or if there was a mistake the job wasn't saved. All exceptions are caught and logged so that the rest of the application doesn't crash.

Key Features:

- Saved job removal for seekers.
- Secure deletion using PreparedStatement.
- Clear success/failure return value for reliable feedback.
- Supports seeker dashboards and job bookmarking workflows.

Screenshot

SeekerDao.java

```
public boolean unsaveJob(int seekerId, int jobId) {
    try {
        String sql = "DELETE FROM saved_jobs WHERE seeker_id = ? AND job_id = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt( parameterIndex: 1, seekerId);
            ps.setInt( parameterIndex: 2, jobId);
            return ps.executeUpdate() > 0;
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return false;
}
```

Figure 67 unsaveJob

6. isJobSaved()

Functionality:

This approach verifies that the given job has not yet been saved by the job seeker before. This avoids duplication and helps implement conditional UI logic such as the Save/Unsave buttons on the dashboard.

Input:

- int seekerId: The unque identifier of the job seeker.
- int jobId: The unique identifier of the job.

Output:

- true: If the job is already saved by the seeker.
- False: If the job is not saved or an error occurred.

Working Process:

The function creates an SQL SELECT query for fetching records from the saved_jobs table where seeker_id and job_id both are equal to the corresponding input values and SQL query is created using the PreparedStatement object which helps in preventing the SQL injection. Then the query run and the function verifies whether the returned ResultSet object has any records or not. In case of the existence of any record the function will return true or it return false.

Key Features:

- Saved job existence check for seekers.
- Secure query execution using PreparedStatement.
- Boolean return value for easy integration in workflows.
- Exception handling ensures robust performance.

Screenshot

SeekerDao.java

```
public boolean isJobSaved(int seekerId, int jobId) {
    try {
        String sql = "SELECT 1 FROM saved_jobs WHERE seeker_id = ? AND job_id = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt( parameterIndex: 1, seekerId);
            ps.setInt( parameterIndex: 2, jobId);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next();
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return false;
}
```

Figure 68 IsJobSaved

7. getSavedJobIds()

Functionality:

This approach returns all the IDs for those jobs that have been bookmarked by the job seeker in question. This makes it easy to find out what jobs the job seeker has bookmarked.

Input

- int seekerId: The unique identifier of the job seeker.

Output:

- Set<Integer>: A set of job Ids saved by the seeker.

Working Process:

It formulates a SQL SELECT statement that retrieves all jobid from the savedjobs table for a seekerid passed as parameter. PreparedStatement makes sure to bind the parameters correctly so that to avoid SQL injection. It runs the prepared statement and loops over the results returned. Every jobid fetched is added to a HashSet to ensure distinctiveness. The set of jobs is then returned. In the case there are no saved jobs, or an exception occurs, an empty set is returned. The exceptions encountered are caught and logged in the application logs.

Key Features:

- Retrieve saved job IDs for seekers.
- Secure query execution using PreparedStatement.
- Uses HashSet to guarentee uniqueness of job IDs.

Screenshot

SeekerDao.java

```
public Set<Integer> getSavedJobIds(int seekerId) {
    Set<Integer> ids = new HashSet<>();
    try {
        String sql = "SELECT job_id FROM saved_jobs WHERE seeker_id = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt( parameterIndex: 1, seekerId);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    ids.add(rs.getInt( columnLabel: "job_id"));
                }
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return ids;
}
```

Figure 69 GetSavedJobIds

8. getSavedJobs()

Functionality:

This method return all the jobs that have been saved by a particular jobseeker. The algorithm will provide the jobseekers with job details in a time-based order that the jobs were saved by the seekers.

Input:

- int seekerId : The unique identifier of the job seeker.

Output:

- List<Job> : A list of job objects containing full job details empty if none are saved.

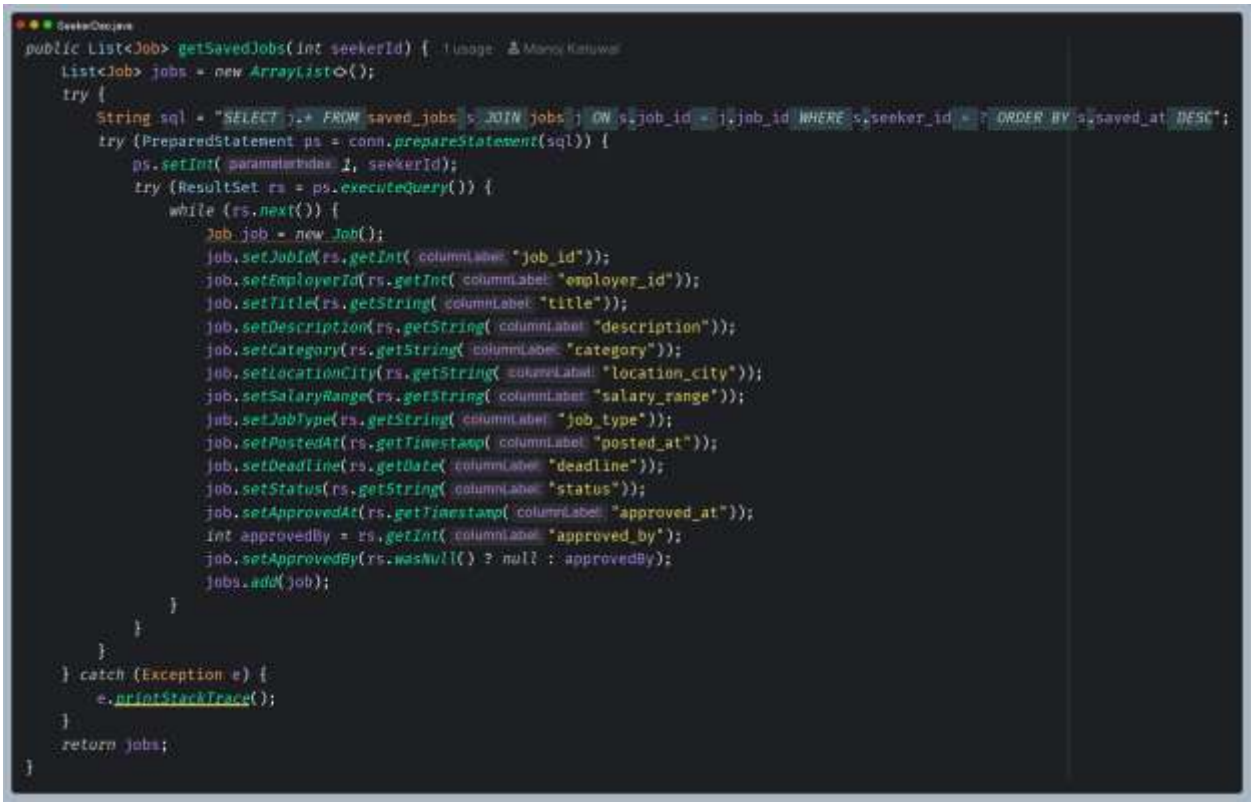
Working Process:

It formulates a SQL SELECT statement that retrieves all jobids from the savedjobs table for a seekerid passed as parameter. PreparedStatement makes sure to bind the parameters correctly so that to avoid SQL injection. It runs the prepared statement and loops over the results returned. Every jobid fetched is added to a HashSet to ensure distinctiveness. The set of jobs is then returned. In the case there are no saved jobs, or an exception occurs, an empty set is returned. The exceptions encountered are caught and logged in the application logs.

Key Features:

- Retrieve saved jobs with full job details.
- Secure query execution using PreparedStaement.
- Jobs ordered by saved_at timestamp for chronological display.

Screenshot



```
public List<Job> getSavedJobs(int seekerId) {
    List<Job> jobs = new ArrayList<>();
    try {
        String sql = "SELECT j.* FROM saved_jobs s JOIN jobs j ON s.job_id = j.job_id WHERE s.seeker_id = ? ORDER BY s.saved_at DESC";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(parameterIndex: 1, seekerId);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    Job job = new Job();
                    job.setJobId(rs.getInt(columnLabel: "job_id"));
                    job.setEmployerId(rs.getInt(columnLabel: "employer_id"));
                    job.setTitle(rs.getString(columnLabel: "title"));
                    job.setDescription(rs.getString(columnLabel: "description"));
                    job.setCategory(rs.getString(columnLabel: "category"));
                    job.setLocationCity(rs.getString(columnLabel: "location_city"));
                    job.setSalaryRange(rs.getString(columnLabel: "salary_range"));
                    job.setJobType(rs.getString(columnLabel: "job_type"));
                    job.setPostedAt(rs.getTimestamp(columnLabel: "posted_at"));
                    job.setDeadline(rs.getDate(columnLabel: "deadline"));
                    job.setStatus(rs.getString(columnLabel: "status"));
                    job.setApprovedAt(rs.getTimestamp(columnLabel: "approved_at"));
                    int approvedBy = rs.getInt(columnLabel: "approved_by");
                    job.setApprovedBy(rs.wasNull() ? null : approvedBy);
                    jobs.add(job);
                }
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return jobs;
}
```

Figure 70 GetSavedJobs

9. getSavedJobCount()

Functionality:

This approach involves computing the total number of jobs saved by a particular job seeker. It is a simple way to summarize the information in numbers.

Input:

- int seekerId: The unique identifier of the job seeker.

Output:

- long: The total number of jobs saved by the seeker.

Working Process:

A SQL SELECT COUNT(*) query is made using the savedjobs table which count all records where seekerid equals the seeker_id passed as parameter. A PreparedStatement is used to bind the parameter in order to protect from SQL injection. The query is executed and the resultset checked, when a record exists, the count is retrieved using the alias total, and is returned. When not existing or when an exception is thrown, an empty value is safely returned to the caller and an exception is logged and the process continues.

Screenshot

SeekerDao.java

```
public long getSavedJobCount(int seekerId) {  1 usage   Manoj Katuwal
    try {
        String sql = "SELECT COUNT(*) AS total FROM saved_jobs WHERE seeker_id = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt( parameterIndex: 1, seekerId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next())
                    return rs.getLong( columnLabel: "total");
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return 0;
}
}
```

Figure 71 GetSavedJobCount

5.6 StatsDao

1. getTotalUsers()

Functionality:

This methos retrieves the total number of registered users in the system. It is primarily used by the admin dashboard and reports pages to display overall user statistics.

Input:

- none: The method runs internally without external parameters.

Output:

- long : The total number of users in the users table.

Working Process:

This method is delegated to the internal helper function countQuery(), passing a SQL statement that returns a count of all rows present in the users table. It is achieved using count(*); it uses "total" as an alias for the query. It returns a long as the query result and 0 if any exception is thrown or if no result is found.

Key Features:

- User statistics retrieval for dashboards and reports.
- Simple numeric output for easy integration.
- Exception handling ensures stability.

Figure 72 GetTotalUsers

2. getTotalSeekers()

Functionality:

This technique will return the total number of users who have been registered as job seekers. This technique is mostly used on the admin dashboard and reports pages for showing statistics regarding the seekers.

Input:

- none : The method runs internally without external parameters.

Output:

- long : The total number of seekers in the Users table.

Working Process:

This method delegates the execution to the private helper method countQuery() and sends it a SQL statement that would count all rows in the users table such that the 'role' field is equal to 'SEEKER'. The query employs the use of COUNT(*) which is aliased to 'total'. The result is cast to a long. In case of an exception or if there are no results, the method will be back to 0.

Key Features:

- Seeker statistics retrieval for dashboards and reports.
- Simple numeric output for easy integration.

Screenshot

Figure 73 GetTotalSeekers

3. getTotalEmployers()

Functionality:

This method retrieves the total number of users registered as employers in the system. It is primarily used by the admin dashboard and reports pages to display employer specific statistics.

Input:

- none : The methos runs internally without external parameters.

Output:

- long: The total number of employers in the users table.

Working Process:

The function simply passes a sql statement to the internal help functioncountQuery,which has the effect to count all rows on the users table having the role 'EMPLOYER'.The query is a count(*) with an alias total, and the function returns a long value (the result of the count). If something goes wrong or nothing is found 0 is returned.

Key Features:

- Employer statistics retrieval for dashboards and reports
- Simple numeric output for easy integration.

Screenshot

StatsDao.java

```
public long getTotalEmployers() {  4 usages  Manoj Katuwal
    return countQuery( sql: "SELECT COUNT(*) AS total FROM users WHERE role = 'EMPLOYER'");
}
```

Figure 74 GetTotalEmployers

9. getPendingUsers()

Functionality

The purpose of this procedure is to get the total number of users who have accounts that are pending approval. This is mostly used by the admin dashboards and reporting pages.

Input:

- none: The method runs internally without external parameters .

Output:

- long : The total number of pending users in the users table.

Working Process:

This method calls the helper function countQuery () internaly, with a SQL query, that counts the number of rows in the table users, that are in the state 'PENDING'. The count Query uses COUNT(*) with an alias total. Returns type long, return 0 in case of exception or not found results.

Key Features:

- Pending user statistics for dashboards and reports.
- Simple numeric output for easy integration.

Screenshot

Figure 75 GetApprovedJobs

10. getApprovedUsers()

Functionality:

The above process gets the total number of users that have their accounts activated by the admin. The main purpose of the above code is in the admin dashboard and reports pages.

Input:

- none : The method runs internally without external parameters.

Output:

- long: The total number of approves users in the users table.

Working Process:

The method calls upon the internal helper function countQuery() and provides a SQL query string that counts all rows in the users table that have a status that matches 'APPROVED'. It utilizes COUNT(*) with the total alias. The return type is long. It safely returns 0 if an exception occurs or if no data is found.

Key Features:

- Approves user statistics for dashboards and reports.
- Simple numeric output for easy integration.

Screenshot

StatsDao.java

```
public long getApprovedUsers() {  no usages   Manoj Katuwal
    return countQuery( sql: "SELECT COUNT(*) AS total FROM users WHERE status = 'APPROVED'");
}
```

Figure 76 GetApprovedUsers

11. getRejectedUser()

Functionality:

The following method fetches the count of all users who have had their accounts blocked by the admin. This algorithm is mainly used by the admin dashboard and reporting pages to keep track of unsuccessful registration attempts.

Input:

- none: The method runs internally without external parameters.

Output:

- Long: The total number of rejected users in the users table.

Working Process:

In this method, execution is delegated to the internal helper method countQuery(), in which it sends a SQL query to count all the records in the users table where the status column has a value 'REJECTED' (this query uses a COUNT(*) statement with the total alias and return as long and it is handling exceptions and returning 0 if there is no record and exception raised).

Key Features:

- Rejected user statistics for dashboards and reports
- Simple numeric output for easy integraton.

Screenshot

StatsDao.java

```
public long getRejectedUsers() {  no usages  & Manoj Katuwal
    return countQuery( sql: "SELECT COUNT(*) AS total FROM users WHERE status = 'REJECTED'");
}
```

Figure 77 GetRejectedUsers

12. getTotalJobs()

Functionality:

The method retrieves the total number of job postings in the system. It is primarily used by the admin dashboard and reports pages to display overall job statistics.

Input:

- none: The method runs internally without external parameters.

Output:

- long: The total number of jobs in the jobs table.

Working Process:

In this method, it passes to the internal helper method countQuery() where it sends the SQL query to count all the records from the users table whose status column has the value of 'REJECTED' in this query it is using the COUNT(*) statement with alias total and returning as long and it is handling the exceptions and if there is no records or any exception is thrown it is returning as 0.

Key Features:

- Job statistics retrieval for dashboards and reports.
- Simple numeric output for easy integration.

Screenshot

Figure 78 GetTotalJobs

13. getPending JobCountByEmployer()

Functionality:

This method is used for fetching the total number of job postings by a particular employer which are yet to be approved. This technique is generally employed by the admin dashboard and reports pages.

Input:

- int employerId : The unique identifier of the employer.

Output:

- long : The total number of pending jobs posted by the employer.

Working Process:

In this method it calls the private helper method countQueryParam() and gives it a SQL statement which counts all the jobs from the jobs table when employer_id is equal to what is passed in and status is 'pending'. It uses the count(*), with an alias called total, and it is returned as a long and will return 0 if there is an error or not any result.

Key Features:

- Employer specific pending jobs for dashboard and reports.
- Simple numeric output for easy integration.
- Exception handling ensures stability.

Screenshot

Figure 79 GetPendingJobCountByEmployer

6 RegisterServlet

1. doGet()

Functionality:

This method will process web service requests (HTTP GET) to the registration page. This makes sure that visitors who are already signed in get directed to their respective dashboards, and new visitors get securely directed to register.jsp, which is located in WEB-INF/pages.

Input:

- HttpServletRequest req : Carries client request data (session, parameters, headers)
- HttpServletResponse resp: Used to send response data back to the client.

Output:

- Forward to the JSP: /WEB-INF/pages/register.jsp if user is not logged in.
- Redirect : Appropriate dashboard if user is already logged in.

Working Process:

On a GET request call to the servlet, the servlet first calls to SessionUtils.Check if user is already logged in by redirectIfLoggedIn(req,resp) and if they are, it will send them to the relevant dashboard page, and this will be returned by the utility method and the servlet will return, so they will not be redirected to the registration page. If the user is not logged in, it securely redirects the request to /WEB-INF/pages/register.jsp using a RequestDispatcher, so the user would not be able to access the JSP by directly going to it in the URL, while the MVC pattern still goes on as the servlet is the controller and the JSP is the view.

Key Features

- Session-based redirection for logged in users.
- Secure JSP forwarding using RequestDispatcher.
- Prevents direct URl access to JSP files.

Screenshot

Figure 80 doGet() of Register Servlet

2. doPost()

Functionality:

This approach is used to process user registration through a call to the server with a POST request sent over HTTP. It receives the data from a seeker's form, creates a User object and a Profile object and passes persistence responsibility to the UserDao. Depending on this, it will display the flash messages and send the user to the login or registration page.

Input:

- HttpServletRequest req : Carries form data submitted by the client.
- HttpServletResponse resp: Used to send response data back to the client.

Output;

- Redirect to login: If registration is successful.
- Redirect to register: If registration fails or an error occurs.

Working Process

The POST method will extract the values of form parameters like Full Name, Email, Password, Role, City, and Date of Birth, and insert them into the User object. Depending upon the user's role, either a new SeekerProfile or EmployerProfile object will be generated with corresponding data about education and skills in case of SeekerProfile and company name and industry in case of EmployerProfile. The connection to the database will be made using DBConnection and UserDao.registerUser() method will be invoked for saving the user and profile information in the database. The registration process will generate a success message that will be displayed after which the user will be directed to the login page.

Key Features

- Role-based registration for seekers and employers.
- Secure database interaction using UserDao and PreparedStatement.
- Flash messaging for success and error feedback.

Screenshot

```
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

    String role = req.getParameter( s: "role");

    User user = new User();
    user.setFullName(req.getParameter( s: "fullName"));
    user.setEmail(req.getParameter( s: "email"));
    user.setPassword(req.getParameter( s: "password"));
    user.setRole(role);
    user.setLocation(req.getParameter( s: "city"));

    try {
        user.setDob(java.sql.Date.valueOf(req.getParameter( s: "dob")));
    } catch (Exception e) {
        user.setDob(null);
    }

    SeekerProfile seeker = null;
    EmployerProfile employer = null;

    if ("SEEKER".equalsIgnoreCase(role)) {
        seeker = new SeekerProfile();
        seeker.setEducation(req.getParameter( s: "education"));
        seeker.setSkills("");
    } else if ("EMPLOYER".equalsIgnoreCase(role)) {
        employer = new EmployerProfile();
        employer.setCompanyName(req.getParameter( s: "companyName"));
        employer.setCompanyCategory(req.getParameter( s: "industry"));
    }

    try (Connection conn = DBConnection.getConnection()) {
        UserDao dao = new UserDao(conn);
        boolean result = dao.registerUser(user, seeker, employer);

        if (result) {
            SessionUtils.setFlashSuccess(req, key: "success", message: "Registration Successful! Please login.");
            resp.sendRedirect( location: req.getContextPath() + "/login");
        } else {
            SessionUtils.setFlashError(req, key: "error", message: "Registration Failed!");
            resp.sendRedirect( location: req.getContextPath() + "/register");
        }
    } catch (Exception e) {
        e.printStackTrace();
        resp.sendRedirect( location: req.getContextPath() + "/register?error=2");
    }
}
}
```

Figure 81 doPost() of Register Servlet

7 Utils

7.1 Password util

1. hashPassword()

Functionality:

This method securely hashes a clear text password with BCrypt hash function. It guarantees that user passwords are never saved in plain text, making it more difficult for users to be breached and promoting the protection of the system in general.

Input:

- String plainPassword : The raw password entered by the user during registration and password update.

Output:

- String : A hashed password string suitable foe secure storage in the database.

Working Process:

In calling this method, BCrypt.hashpw() will take place using the unhashed password and a salt generated by BCrypt.gensalt(). The use of salt guarantees that there would be uniqueness and protection against the attack called rainbow table. The hashed string will then be returned and will serve as the replacement for storing the password in the database.

Key Features:

- Secure password hashing used industry-standard BCrypt.
- Salt generation ensures uniqueness and prevents precomputed attacks.
- Database security by avoiding plain text storage

Screenshot

Figure 82 hashPassword

2. checkPassword()

Functionality:

This approach or methods checks to see if a specified plain text password matches its stored, hashed password, found with the BCrypt algorithm. It is mostly used in the authentication process during a login session to ensure the security of the user.

Input:

- String plainPassword : The raw password entered by the user during login.
- String hashedPassword: The stored hashed password retrieved from the database.

Output:

- Boolean: true if the plain password matches the hashed password otherwise false.

Working Process:

Methods Function verifies if either of the two passwords – clear password and hashed password – is null; when any of the two is null, the methods function returns a false. However, where both values are not null, the BCrypt.checkpw() methods function is called in order to compare the clear password with the saved password. This process leads to a return of either a false or true, depending on the outcome of comparison.

Key Features:

- The method verifies password using BCrypt.
- Null safety check to prevent runtime errors.
- Secure authentication by comparing hashed values.

Screenshot

PasswordUtils.java

```
public static boolean checkPassword(String plainPassword, String hashedPassword) {
    if (plainPassword == null || hashedPassword == null) {
        return false;
    }
    return BCrypt.checkpw(plainPassword, hashedPassword);
}
}
```

Figure 83 checkpassword

7.2 CookieUtils

1. addCookie()

This method adds a new HTTP cookie to the response object and creates it. To securely store small amounts of data (flash messages, preferences, session tokens, etc.) on the client side.

Input:

- HttpServletResponse resp: The response object to which the cookie will be added.
- String name: The name of the cookie.
- String value: The value stored in the cookie.
- Int maxAge: The lifetime of the cookie in seconds.
- Boolean httpOnly: Flag to restrict cookie access to HTTP only.

Output:

- None: The methods adds the cookie to the response, no return value.

Working Process:

Upon calling this method, a Cookie instance will be created based on the parameters passed as the name and value, its maximum age will be set to determine the duration for which it will last, an HttpOnly cookie will be set as an additional security measure that does not allow client-side scripting access, and a path of / will be specified to make it valid for the whole application.

Key features:

- Secure cookie creation with HttpOnly support.
- Session and preference storage for client-side persistence.
- Global path setting ensures cooking availability across the application.

Screenshot

Figure 84 addCookie

2. getCookieValue()

Functionality:

This methods is to find the value of a certain cookie of the incoming HTTP request. If you want to access a session token, user preferences, or any other data that is stored on the client side in a cookie, it is often used to do so.

Input:

- HttpsServletRequest req: The request object containing cookies sent by the client.
- String name: The name of the cookie to be retrieved.

Output

- String : The value of the cookie if found otherwise null.

Working Process:

The invoked method firstly gets the array of cookies from the request by calling the req.getCookies() function. The array of cookies is then checked whether it has been generated, if so each of the cookie names is checked and compared with the name parameter passed. If it matches then the cookie value of that cookie is immediately returned. If no matched cookie is found, then 'null' value is returned (this function is safely executed when the request does not contains any cookies).

Key Features:

- Cookie retrieval from client requests.
- Null safety to avoid runtime errors.
- Session and preference access for user-specific data.

Screenshot

Figure 85 GetCookieValue

3. getCookie()

Functionality:

This method allows obtaining the object representing the cookie within the HTTP request. This approach is frequently employed when there is a need to get access not only to the cookie itself but also to perform other operations like modifying or deleting it.

Input:

- HttpServletRequest req: The request object containing cookies sent by the client.
- String name: The name of the cookie to be retrieved.

Output

- Cookie: The cookie object if found otherwise null.

Working Process:

When the method is called it will get the array of cookies from the request by calling req.getCookies() and if they exist it will loop through each cookie and if the name of that cookie is the same as the name parameter sent it will return that Cookie object immediately. If no cookies match the name parameter, or if there are no cookies at all, the method will return null without causing any exceptions.

Key Features:

- Cookie object retrieval for direct manipulation.
- Null safety to avoid runtime errors.
- Enables advanced operations like cookie deletion or modification.

Screenshot

CookieUtils.java

```
public static Cookie getCookie(HttpServletRequest req, String name) {
    Cookie[] cookies = req.getCookies();
    if (cookies != null) {
        for (Cookie cookie : cookies) {
            if (name.equals(cookie.getName())) {
                return cookie;
            }
        }
    }
```

Figure 86 GetCookie

4. deleteCookie()

Functionality:

This method will delete a certain cookie in the browser of the client through the process of setting its values and its expiry. This technique is widely applied for deleting the session id or even user preferences.

Input:

- HttpServletResponse resp: The response object to which the deletion instruction will be added.
- String name: The name of the cookie to be deleted.

Output:

- None: The method modifies the response to remove the cookie no return value.

Working Process:

If the method is called, it will create a Cookie object of the same name as that of the target cookie and will assign it a value of null and its max age to zero so that the browser deletes the cookie, its path as "/" so that it will delete the cookie across the entire application, and then it will add the cookie to the response using the resp.addCookie() method.

Key Features :

- Cookie deletion by setting MaxAge to 0.
- Global path removal ensures deletion across the application.
- Session cleanup for logout and state reset.
- Response integration via resp.addCookie().

Screenshot

CookieUtils.java

```
public static void deleteCookie(HttpServletResponse resp, String name) {
    Cookie cookie = new Cookie(name, value: "");
    cookie.setMaxAge(0);
    cookie.setPath("/");
    resp.addCookie(cookie);
}
```

Figure 87 DeleteCookie

5. hasCokie()

Functionality:

This method validates if there is any cookie that matches a particular cookie in the incoming HTTP request. It is usually applied to validate the existence of session IDs, authentication cookies, or other user preferences.

Input:

- HttpServletRequest req: The request object containing cookies sent by the client.
- String name: The name of the cookie to be checked.

Output:

- Boolean: true if the cookie exists otherwise false.

Working Process:

If the cookie is present and a value is returned other than null then we are aware that the cookie does exist and the function returns true. If null is returned we are aware that no cookie with the requested name was found and the function returns false.

Key Features:

- Cookie existence check for quick validation.
- Efficient reuse of existing utility methods.
- Boolean output for straightforward integration.

Screenshot

Figure 88 HasCookie

7.3 SessionUtil

1. getCurrentUserId()

Functionality:

This method fetches the ID of the currently logged-in user from the ongoing HTTP session. The approach is frequently employed across servlets and controllers to identify the user who is issuing the request without repeatedly accessing the database.

Input:

- HttpServletRequest req: The request object containing the session data.

Output:

- Integer: The user Id is found in the session, otherwise null.

Working Process:

On execution the method first tries to fetch the current session using the request object req.getSession(false), which will return null if a session does not exist. Once it has a valid session, it fetched the value of the attribute, with the key value KeyUser.KEY_USERID and typecast it to Integer, otherwise, in case of a null session or absence of the attribute, null will be returned.

Key Features:

- Session-based user identification without database lookup.
- Null safety to prevent runtime errors.
- Centralized utility for consistent users tracking

Screenshot

SessionUtils.java

```
public static Integer getCurrentUserId(HttpServletRequest req) {  4 usages  Manoj
    HttpSession session = req.getSession( b: false);
    return session != null ? (Integer) session.getAttribute(KEY_USER_ID) : null;
}
```

Figure 89 GetCurrentUserId

2. getCurrentUserRole()

Functionality:

This method returns the role of the currently loggedin user using the current HTTP session. This method is used in most of the servlets and controllers to restrict access to users.

Input:

- HttpServletRequest req: The request object containing the session data.

Output:

- String: The user role if found in the session otherwise null.

Working Process:

When this method is called, it will first try to retrieve the session using the method call req.getSession(false) to get the existing session, if the session is null, then this method will return null safely. If there is the existing session, it will retrieve the attribute whose key is the constant KEY_USER_ROLE and converted it to String using String.valueOf method if the attribute is not null.

Key Features:

- Session-based role retrieval for authorization checks.
- Null safety to prevent runtime errors.
- Centralized utility for consistent role management.

Screenshot

SessionUtils.java

```
public static String getCurrentUserRole(HttpServletRequest req) {
    HttpSession session = req.getSession( b: false);
    if (session == null) return null;
    Object role = session.getAttribute(KEY_USER_ROLE);
    return role != null ? String.valueOf(role) : null;
}
```

Figure 90 GetCurrentUserRole

3. createLoginSession()

Functionality:

It starts a new session when there is a successful authentication of a user. This will hold critical user information like the user's object, ID, role, and in case the role is an admin, the user's full name is included too.

Input:

- HttpServletRequest req: The request object used to obtain or create the session.
- User user: The authenticated user object containing Id, role and profile details.

Output:

- None: The method sets session attributes no return value.

Working Process:

After being called, the method gets the session which is a new session that can be automatically generated when there is none, using req.getSession(). Authentication of the User object takes place and the same together with the id and role of the user is stored within the session using KEY_USER, KEY_USER_ID, and KEY_USER_ROLE respectively. Moreover, if the role of the user is ADMIN then the name of the admin is also stored in the session using the constant KEY_ADMIN_NAME.

Key Features:

- Session created for authenticated users.
- Role-based session attributes including admin name storage.
- Centralized login handling for consistent access control.
- The method also supports authentication, authorization and dashboard personalization.

Screenshot

SessionUtils.java

```
public static void createLoginSession(HttpServletRequest req, User user) {
    HttpSession session = req.getSession();
    session.setAttribute(KEY_USER, user);
    session.setAttribute(KEY_USER_ID, user.getId());
    session.setAttribute(KEY_USER_ROLE, user.getRole());

    if ("ADMIN".equalsIgnoreCase(user.getRole())) {
        session.setAttribute(KEY_ADMIN_NAME, user.getFullName());
    }
}
```

Figure 91 CreateLoginSession

4. setFlashSucess()

Functionality:

The method works by setting up a temporary success message for the user session. This is usually done after a number of processes have taken place successfully; some examples include user registration, logging in, or updates to database records

Input:

- HttpServletRequest req: The request object used to obtain the session.
- String key: The attribute name under which the message will be stored.
- String message: The success message text to be displayed to the user.

Output:

- None : The methos stores the message in the session no return value.

Working Process:

The method itself works like this. It gets the current session: req.getSession(), creating it if not existing and saves the given success message in session under the given key: session.setAttribute(key, message). The message is accessible for the next page that the user is forwarded to, to show the success message (usually on a JSP or a frontend render and then removed from session).

Key Features:

- Flash success messaging for user feedback.
- Session-based storage ensures temporary persistence.
- Reusable utility for consistent success handling.
- Supports workflows like registration, login and CRUD operations.

Screenshot

SessionUtils.java

```
public static void setFlashSuccess(HttpServletRequest req, String key, String message) {
    HttpSession session = req.getSession();
    session.setAttribute(key, message);
}
```

Figure 92 SetFlashSuccess

5. setFlashError()

Functionality:

The procedure defines a transient message indicating an error. This technique is often employed when there has been an error during tasks like registration or logging in or when there has been a mistake made by filling out a form improperly.

Input:

- HttpServletRequest req: The request object used to obtain the session.
- String key: The attribute name under which the message will be stored.
- String message: The error messaged text to be displayed to the user.

Output:

- None: The methos stores the message in the session no return value.

Working Process:

On invocation, the method takes the current session, either creating one with req.getSession() if it doesn't exist, and stores the passed error message associated with the given key using session.setAttribute(key, message). This message will be printed on the next page(in form of JSP/rendered with the front end) and be removed thereafter.

Key Features:

- Flash error messaging for user feedback.
- Session-based storage ensures temporary persistence.
- Reusable utility for consistent error handling.

Screenshot

SessionUtils.java

```
public static void setFlashError(HttpServletRequest req, String key, String message) {
    HttpSession session = req.getSession();
    session.setAttribute(key, message);
}
```

Figure 93 SetFlashError

6. updateSessionUser()

Functionality:

The purpose of this method is to update the information about the logged-in user for the particular session, in case of any changes in the user's profile or modifications made to his profile data.

Input:

- HttpServletRequest req: The request object used to access the session.
- User user: The updated user object containing new profile details.

Output:

- None: The method modifies session attributes no return value

Working Process:

When the method is called, it is first trying to get current session from thereby using req.getSession(false), if there is no session, it will return null. If the session is null or user isnull, the method will finish without doing anything. Otherwise, it sets the attribute KEY_USER to the user object. If role of user is "ADMIN", it also set KEY_ADMIN_NAME attribute to the whole name of admin,so that the session shows the new profile for the display in admin panel.

Key Features:

- Session update with their new user details.
- Null safety to prevent runtime errors.
- Admin personalization by refreshing the admin full name.

Screenshot

SessionUtils.java

```
public static void updateSessionUser(HttpServletRequest req, User user) {
    HttpSession session = req.getSession( b: false);
    if (session == null || user == null) return;
    session.setAttribute(KEY_USER, user);
    if ("ADMIN".equalsIgnoreCase(user.getRole())) {
        session.setAttribute(KEY_ADMIN_NAME, user.getFullName());
    }
}
```

Figure 94 UpdateSessionUser

8 Test Case

This section details the black-box testing done on the RojgarSetu system. The tests were conducted on the main functions of Admin, Employer and Job Seeker. Multiple sub-tests are provided in each test case to validate the system functionality, security, validation and role-based access control.

8.1 Test Plan

Table 6 Testing Plan

Test ID	Feature Name	Feature Description
1	User Registration	Validates seekers and employers to create accounts, validates fields, validates email, validates password, validates roles, validates database insert.
2	User Login Authentication	Tests login authentication for valid and invalid credentials, dashboard redirection based on roles and creation of a session.
3	Employer Job Posting	Validates the job posting process, such as creating jobs, editing jobs, deleting jobs, and performing validation checks and awaiting approval process.
4	Job Search and Filtering	Advanced search and filtering tools that allow to search by keyword, category, district and location to show relevant jobs.
5	Job Application System	Verifies the submission of job applications, duplicate application avoidance, cover letter submission, and job application status.
6	Resume Upload System	Tests ensure that the upload resume functionality is supported, resume is validated with PDF and is properly stored and retrieved from the tests.
7	Admin User Management	Checks administrator functionality to approve, reject, delete, and manage seeker and employer accounts.

8	Admin Job Approval System	Admin approve/reject job postings, moderation and pending job management.
9	Session and Security Management	Verifies session management, unauthorized route restriction, logout, cache prevention, and role-based access control.
10	Reports and Statistics Dashboard	Evaluate dashboard statistics, aggregate reporting, count of totals, count of pending and export to CSV.

8.2 Test ID 1 Name: User Registration

Validates seekers and employers to create accounts, validates fields, validates email, validates password, validates roles, validates database insert.

The testing process for this feature is given below:

1. Open Registration page.
2. Do not fill in any of the boxes and send it.
3. Enter an invalid email address.
4. Enter weak password that has few characters.
5. Choose a role as a seeker and provide proper information.
6. Click on the employer role, and provide legitimate information.
7. Check validation messages and successful redirects.

8.2.1 Empty Field Validation

Table 7 Testing Empty Field Validation

Test Case ID	1.1
Test Description	Leave the required fields empty
Expected Result	The system should show the validation messages for all required fields.
Actual Result	The validation messages were successfully displayed.
Status	Test was successful.

Evidence:

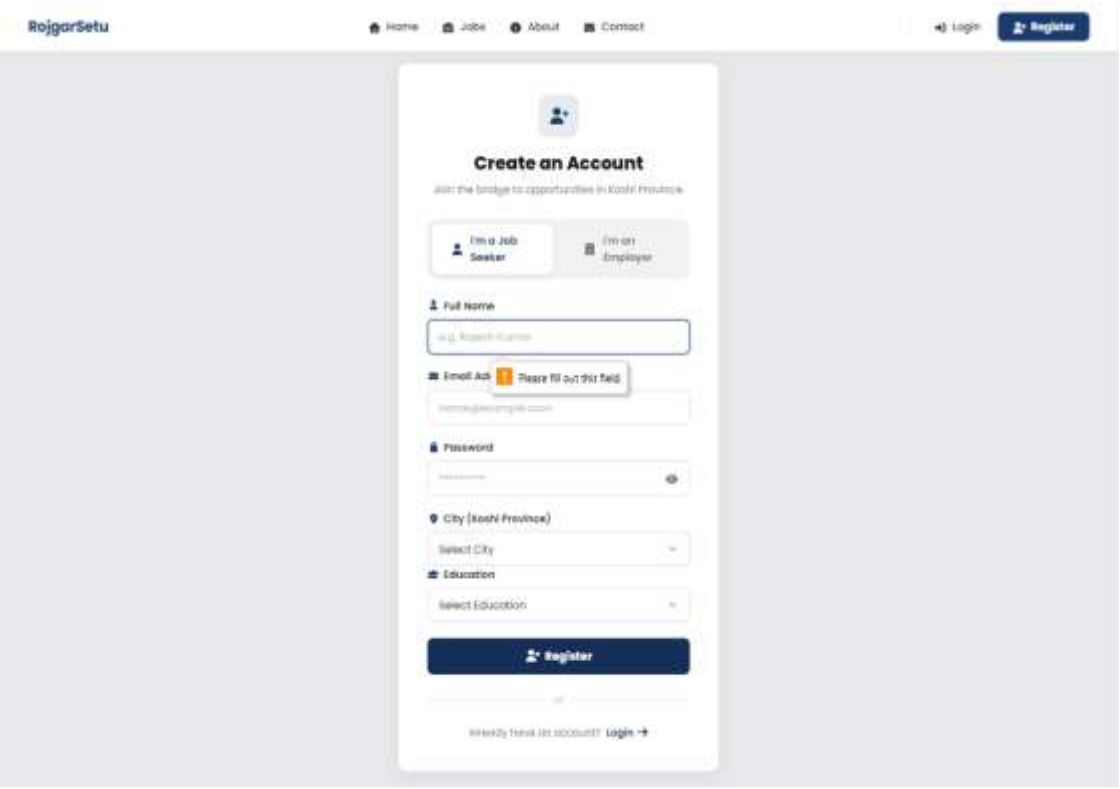


Figure 95 Empty Field Validation

8.2.2 Invalid Email Validation

Table 8 Testing Invalid Email Validation

Test Case ID	1.2
Test Description	Enter an invalid email format
Expected Result	The system should show invalid email format validation message.
Actual Result	Invalid email message was successfully shown.
Status	Test was successful.

Evidence:

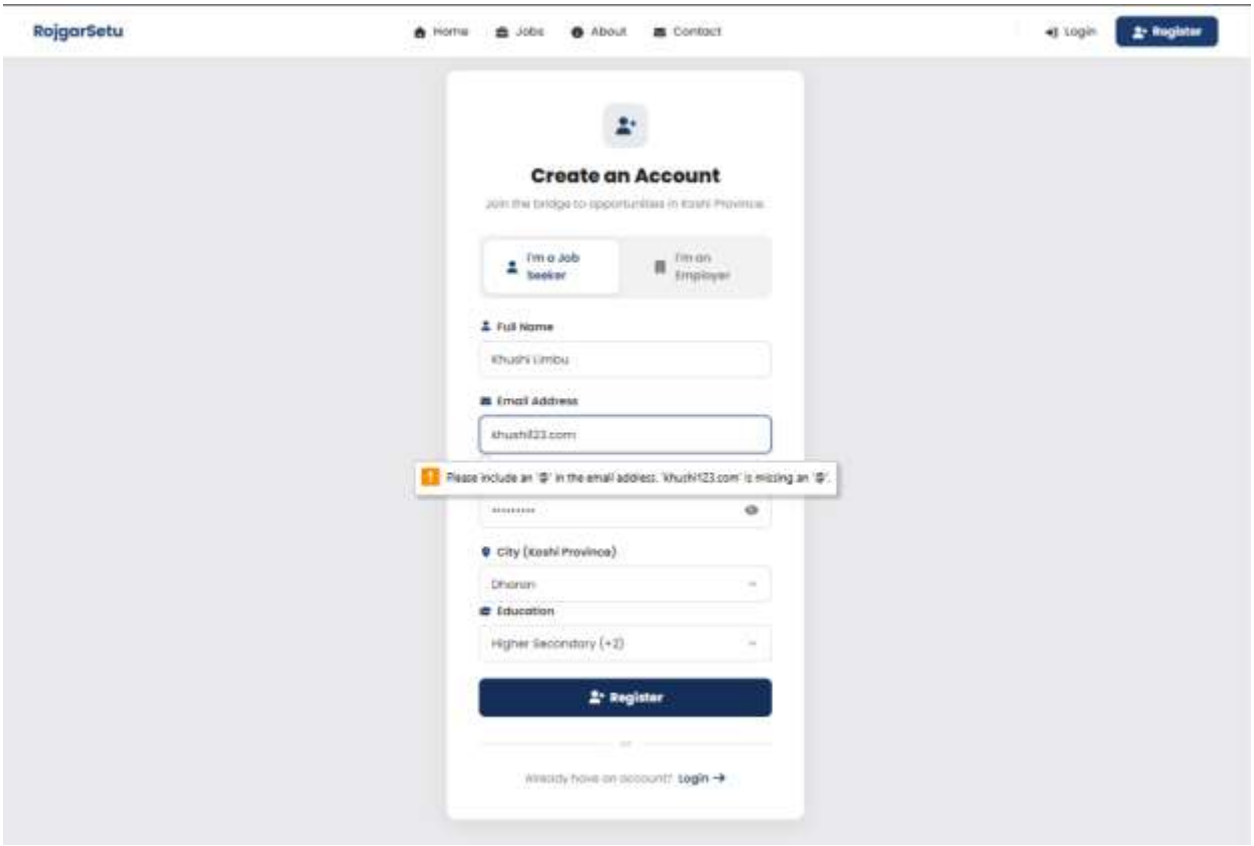


Figure 96 Invalid Email Validation

8.2.3 Seeker Registration

Table 9 Testing Seeker Registration

Test Case ID	1.3
Test Description	Enter a valid registration detail for seeker account registration
Expected Result	Seeker account should be successfully created and directed to login page.
Actual Result	Seeker account was successfully and properly redirected.
Status	Test was successful.

Evidence:

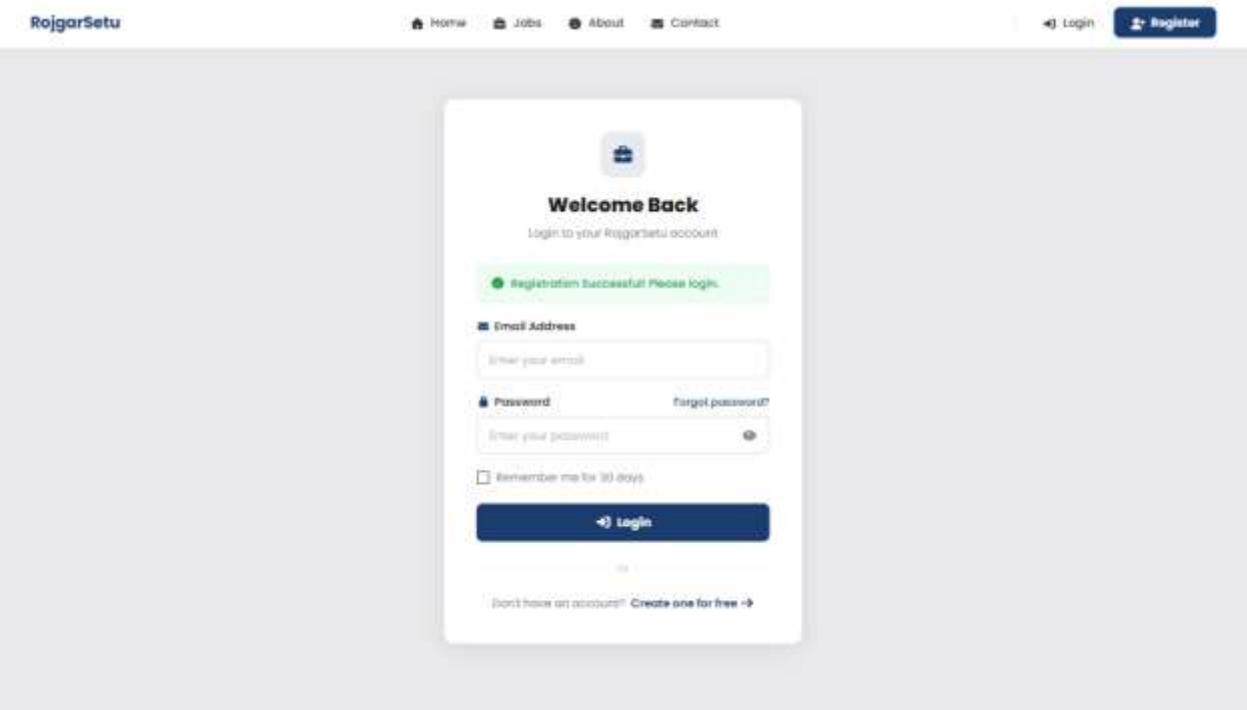


Figure 97 Successful Seeker Registration

8.2.4 Employer Registration

Table 10 Testing Employer Registration

Test Case ID	1.4
Test Description	Enter a valid registration detail for employer account registration
Expected Result	Employer account should be successfully created.
Actual Result	Employer account was successfully.
Status	Test was successful.

Evidence:

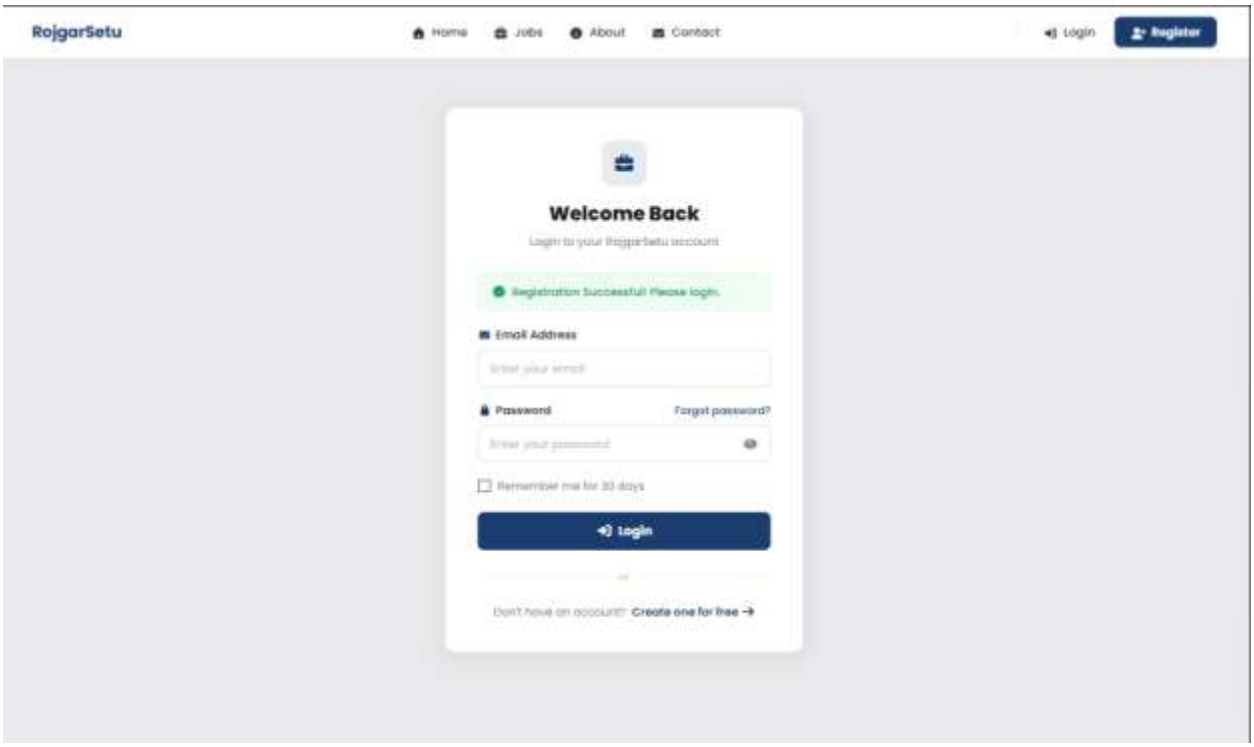


Figure 98 Successful Employer Registration

8.3 Test ID 2 Name: User Login Authentication

Tests login authentication for valid and invalid credentials, dashboard redirection based on roles and creation of a session.

The testing process for this feature is given below:

1. Go to the Login page.
2. Enter incorrect email and password.
3. Check invalid login error message.
4. Use seeker's login credentials to log in.
5. Make sure seeker is redirected to seeker dashboard.
6. Use employer login details to log in.
7. Make sure employer is redirected to employer dashboard.
8. Log in with Admin account.
9. Make sure admin is redirected to admin dashboard.
10. Try to access protected routes without logging in.

8.3.1 Invalid Login

Table 11 Testing Invalid Login

Test Case ID	2.1
Test Description	Enter incorrect email and password
Expected Result	The system to show invalid login credentials message.
Actual Result	Invalid login message was successfully shown.
Status	Test was successful.

Evidence:

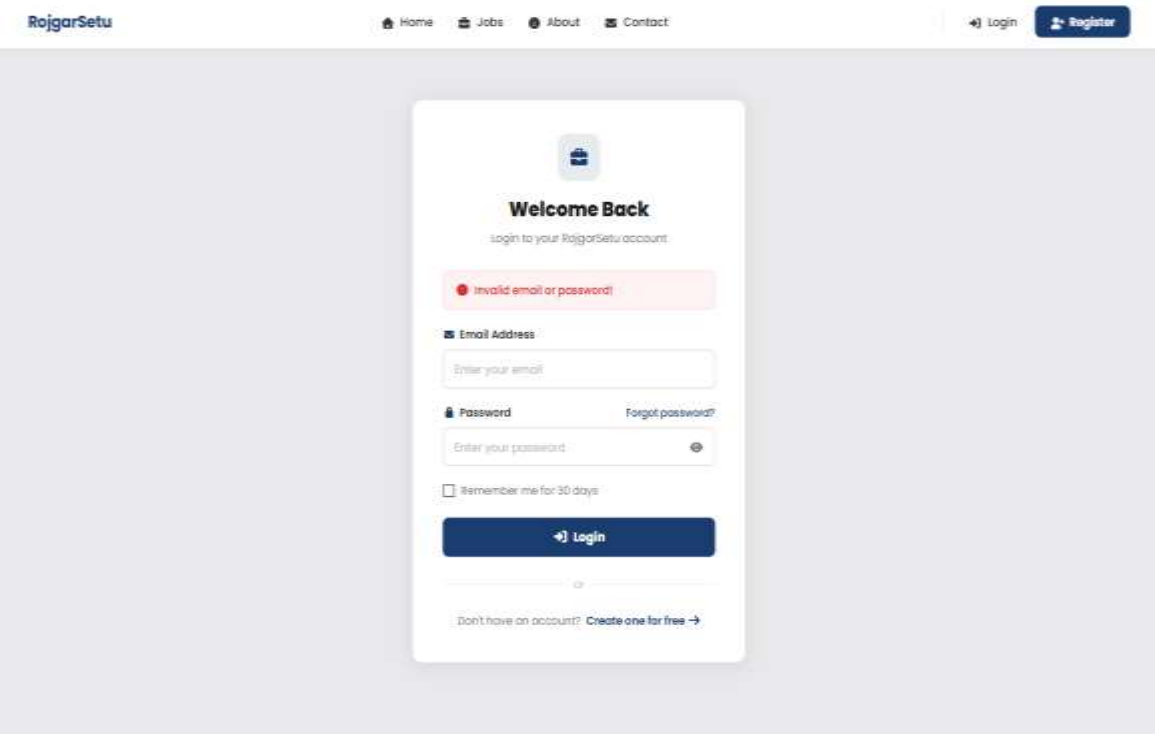


Figure 99 Invalid Login

8.3.2 Admin Login

Table 12 Testing Admin Login

Test Case ID	2.2
Test Description	Enter valid admin login credentials
Expected Result	User should be directed to admin dashboard.
Actual Result	User successfully redirected to admin dashboard.
Status	Test was successful.

Evidence:

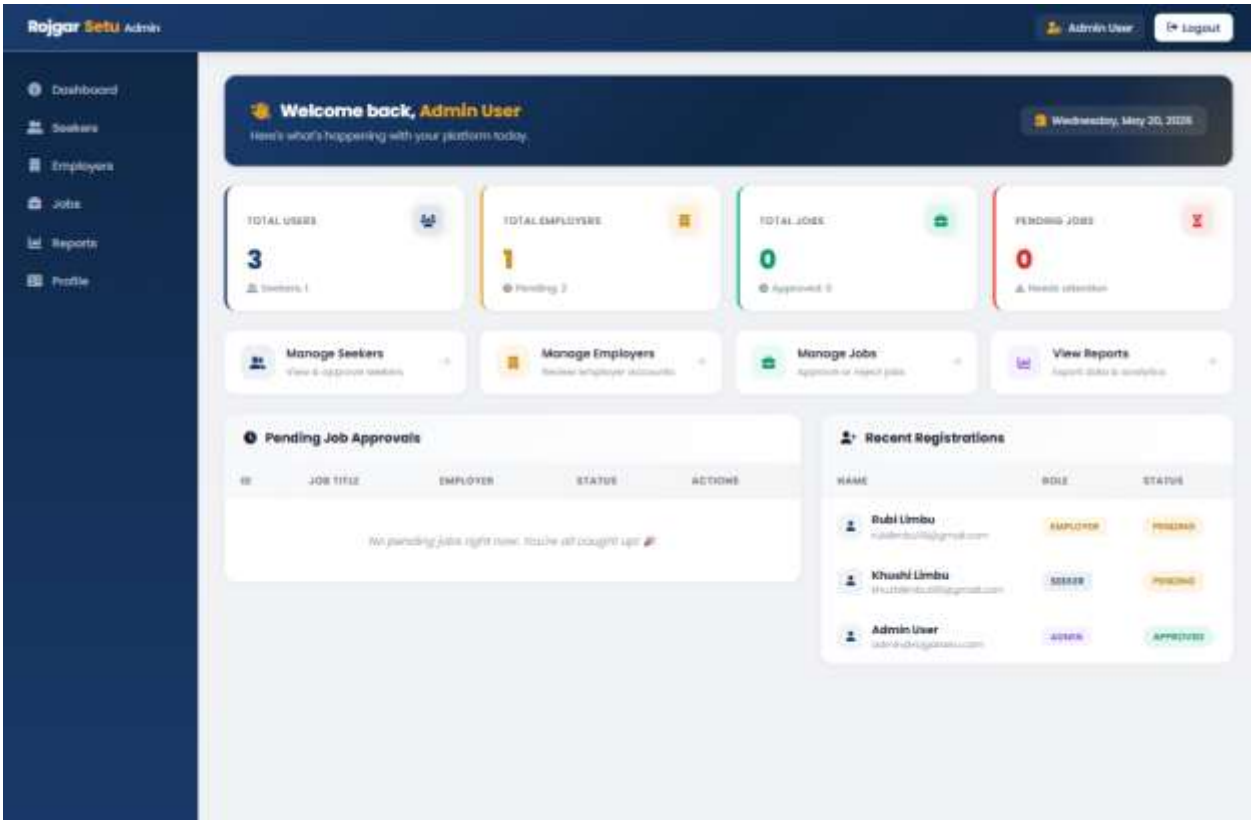


Figure 100 Successful Admin Login

8.3.3 Employer Login

Table 13 Testing Employer Login

Test Case ID	2.3
Test Description	Enter valid employer login credentials
Expected Result	User should be directed to the employer dashboard.
Actual Result	User successfully redirected to employer dashboard.
Status	Test was successful.

Evidence:

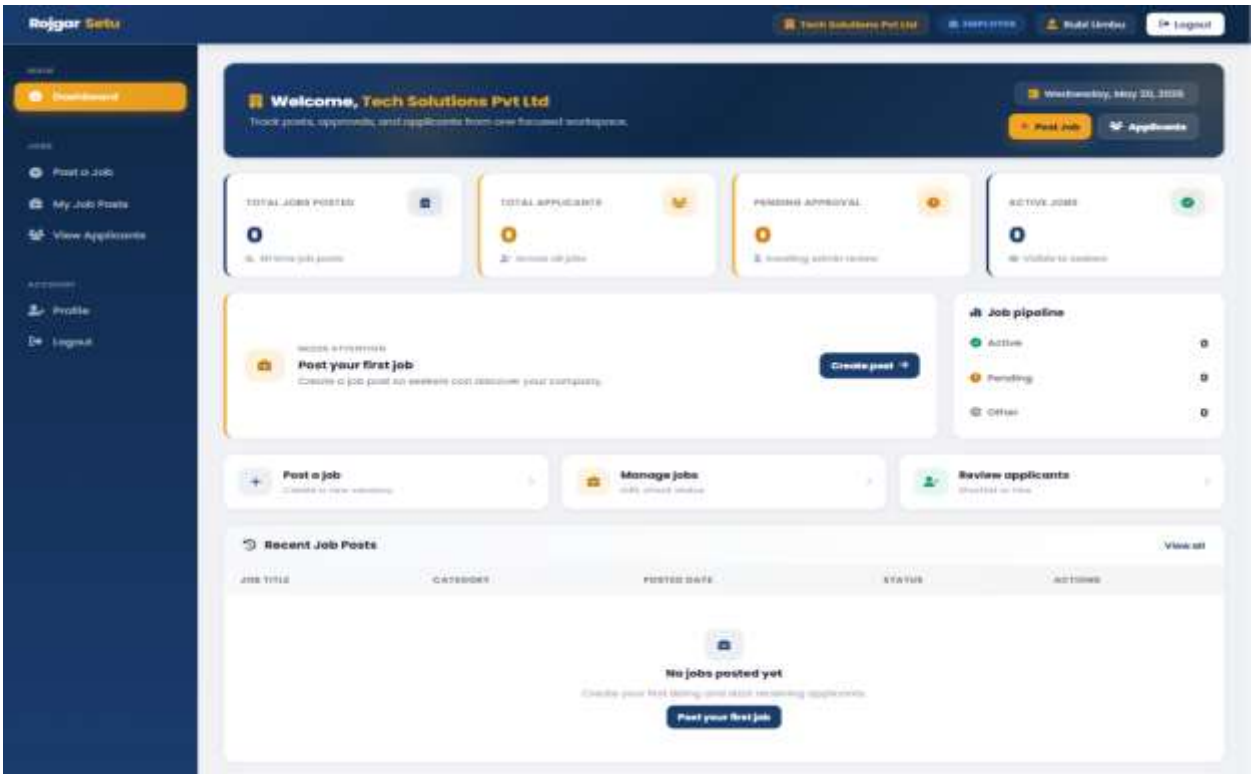


Figure 101 Successful Employer Login

8.3.4 Seeker Login

Table 14 Testing Seeker Login

Test Case ID	2.4
Test Description	Enter valid seeker login credentials
Expected Result	User should be directed to the seeker dashboard.
Actual Result	User successfully redirected to seeker dashboard.
Status	Test was successful.

Evidence:

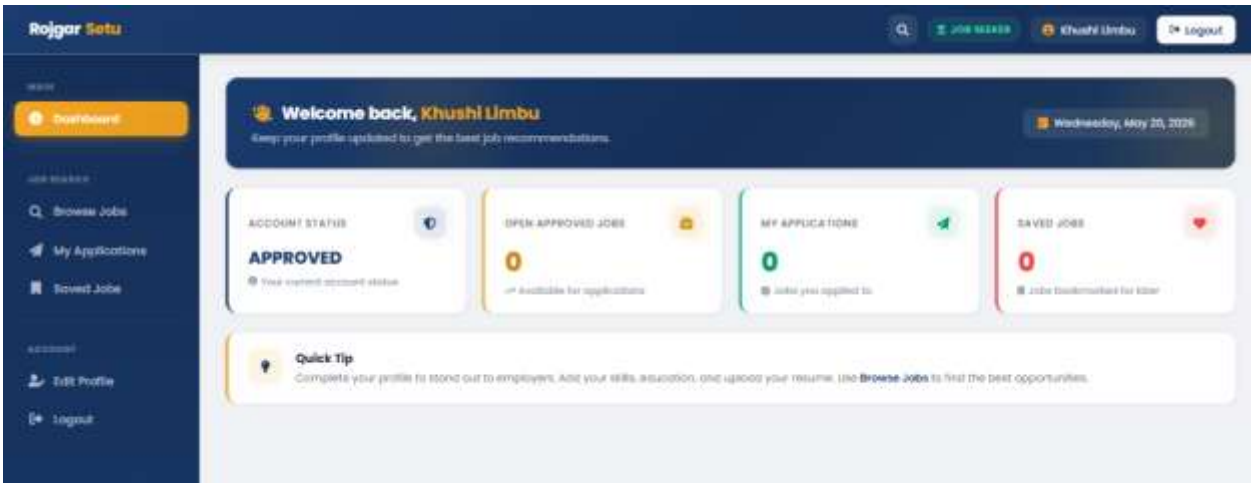


Figure 102 Successful Seeker Login

8.3.5 Unauthorized Access Prevention

Table 15 Unauthorized Access Prevention

Test Case ID	2.5
Test Description	Try accessing protected dashboard routes without logging in
Expected Result	The system should deny access to any unauthorized users and should log them back to the login page.
Actual Result	Unauthorized access was successfully denied and the user was sent to the login page.
Status	Test was successful.

Evidence:

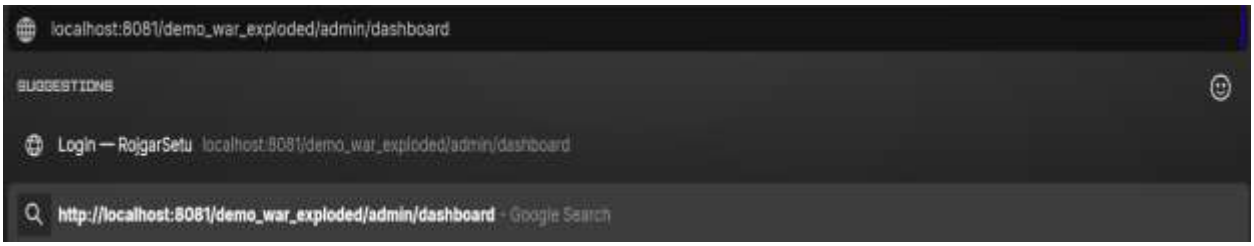


Figure 103 Trying to Access Unauthorized

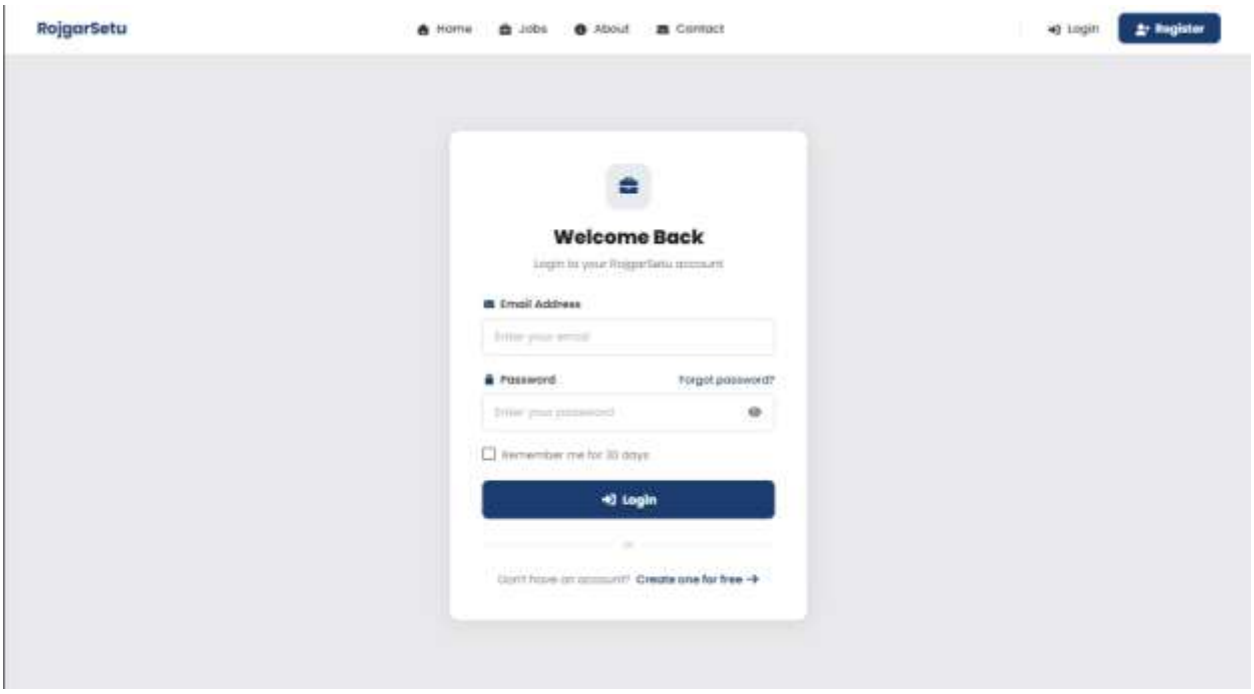


Figure 104 Unauthorized Access Prevention

8.4 Test ID 3 Name: Employer Job Posting

Validates the job posting process, such as creating jobs, editing jobs, deleting jobs, and performing validation checks and awaiting approval process.

The testing process for this feature is given below:

1. Login with employer's account.
2. Open up the create job page.
3. Fill out the form with blank job title.
4. Complete all the necessary entries properly.
5. Submit new job posting.
6. Make sure job is listed in the employer dashboard.
7. Edit the details of existing jobs.
8. Update job information and save it.
9. Delete posted job.

8.4.1 Empty Job Title Validation

Table 16 Testing Empty Job Title Validation

Test Case ID	3.1
Test Description	Leave the job title field blank
Expected Result	The system should show the message about validation of the required field.
Actual Result	Validation message was shown successfully.
Status	Test was successful.

Evidence:

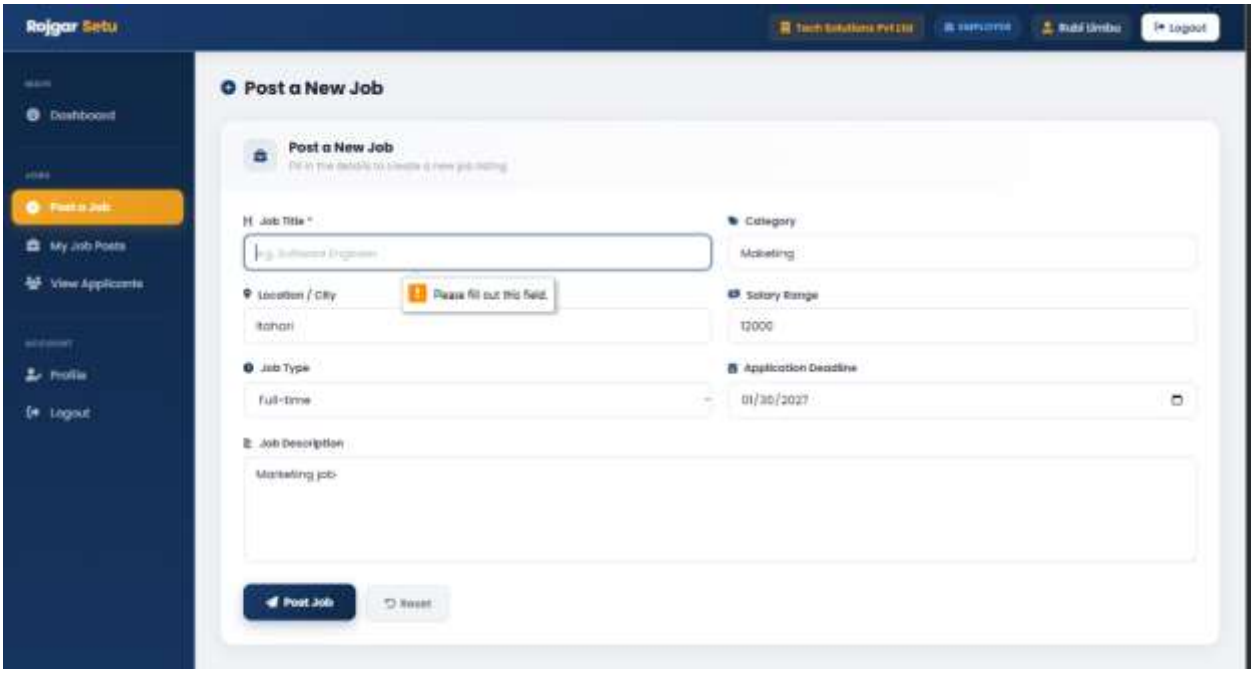


Figure 105 Empty Job Title Validation

8.4.2 Create Job

Table 17 Testing Create Job

Test Case ID	3.2
Test Description	Fill up the required fields
Expected Result	Job posting should be successfully created and be pending approval.
Actual Result	Job posting was successfully created.
Status	Test was successful.

Evidence:

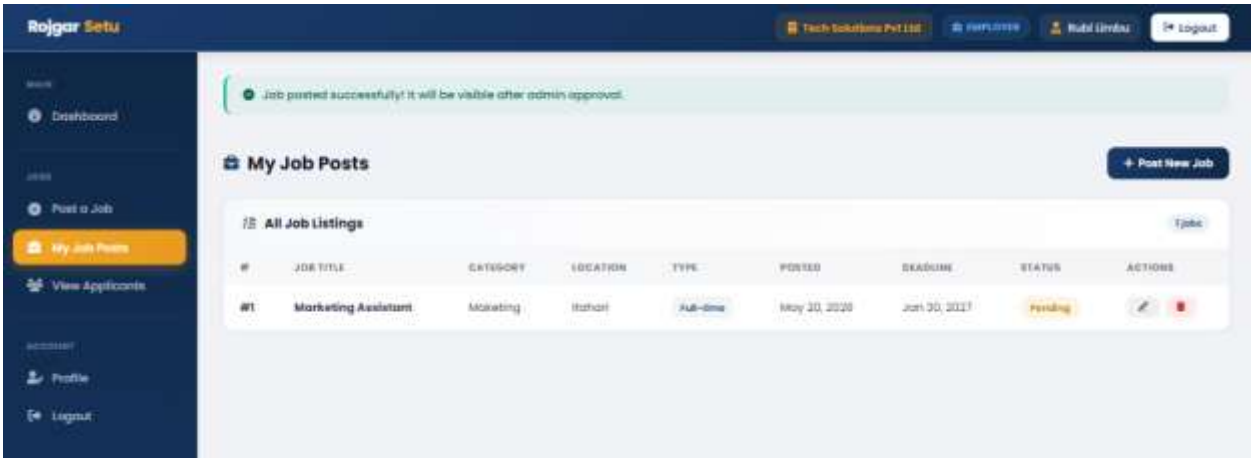


Figure 106 Successful Job Creation

8.4.3 Edit Job

Table 18 Testing Edit Job

Test Case ID	3.3
Test Description	Edit the details of existing jobs
Expected Result	The changes to the job details should be saved successfully.
Actual Result	Job was updated successfully.
Status	Test was successful.

Evidence:

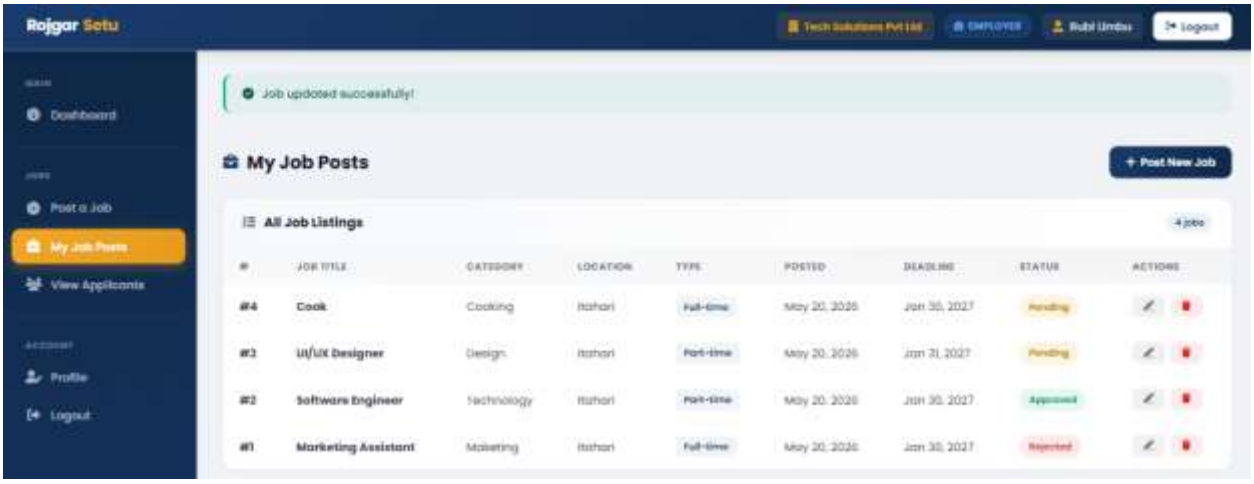


Figure 107 Successful Job Update

8.4.4 Delete Job

Table 19 Testing Delete Job

Test Case ID	3.4
Test Description	Delete an existing job
Expected Result	Job posting should be deleted successfully.
Actual Result	Successfully removed job posting.
Status	Test was successful.

Evidence:

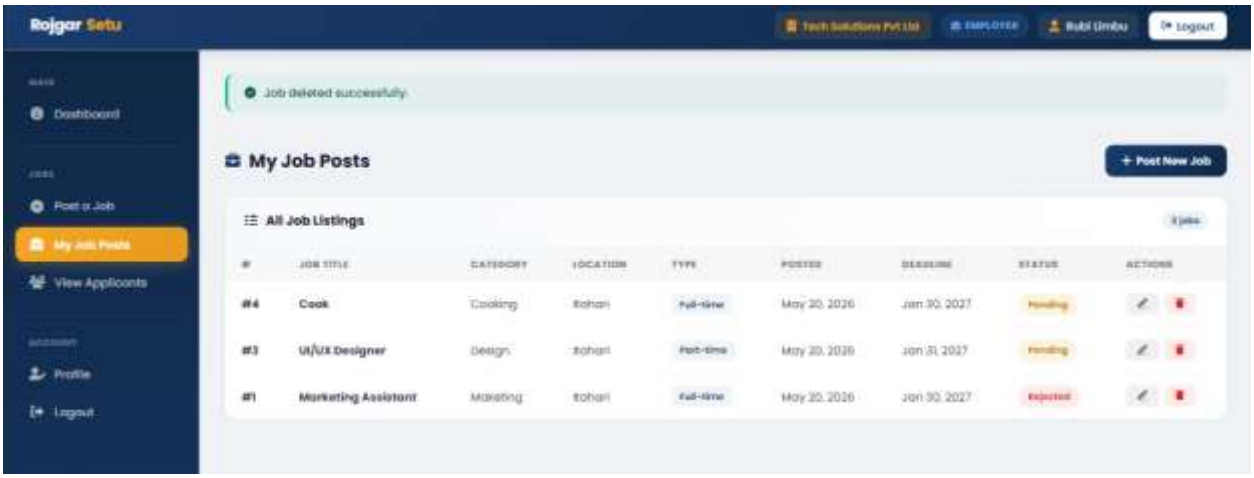


Figure 108 Successful Removal of Job

8.5 Test ID 4 Name: Job Search and Filtering

Advanced search and filtering tools that allow to search by keyword, category, district and location to show relevant jobs.

The testing process for this feature is given below:

1. Click on Browse Jobs.
2. Use keywords to search for job opportunities.
3. Use category to narrow down search results.
4. Search for jobs based on district.
5. Search for jobs by location.
6. Use more than one filter at a time.
7. Check if there are any jobs displayed that don’t correspond to the filters.
8. Clear filters and ensure that all jobs are there.

8.5.1 Search by Keyword

Table 20 Testing Search by Keyword

Test Case ID	4.1
Test Description	Use keywords in search bar to search for jobs
Expected Result	Matching job listings should be displayed.
Actual Result	The relevant job listings were displayed successfully.
Status	Test was successful.

Evidence:

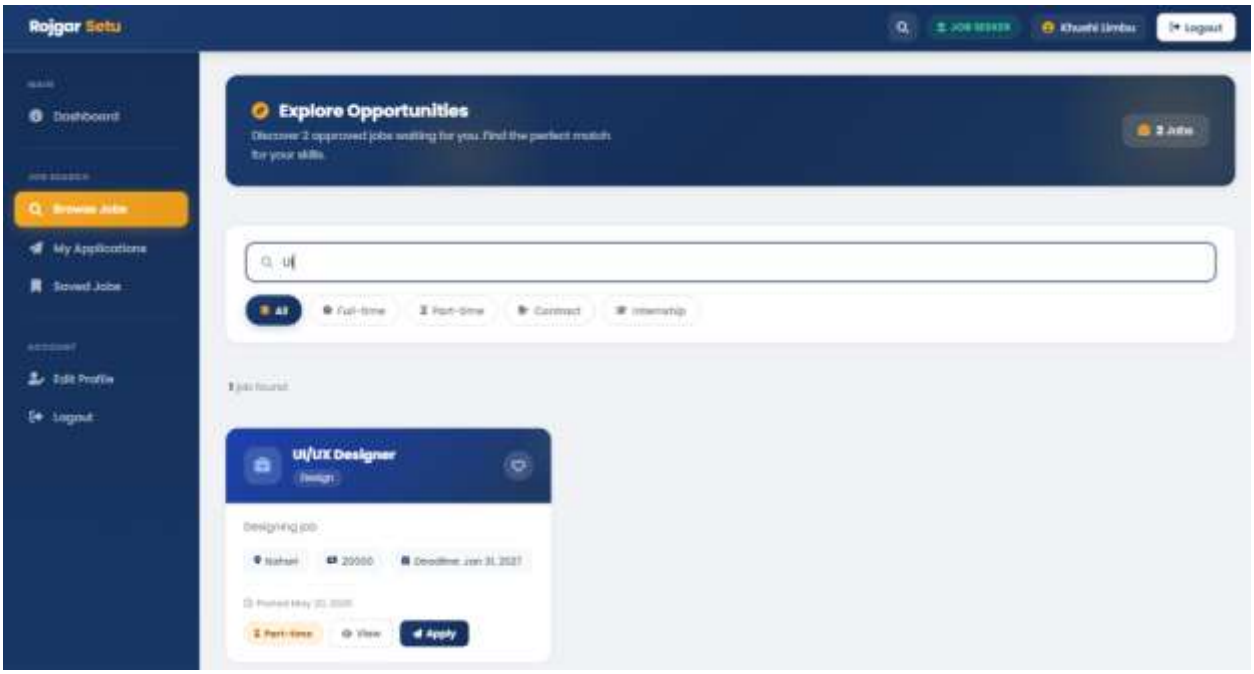


Figure 109 Search by Keyword

8.5.2 Search by Category

Table 21 Testing Search by Category

Test Case ID	4.2
Test Description	Use category selection to filter jobs
Expected Result	Only the jobs that belong to the selected category should be displayed.
Actual Result	Successfully displayed filtered jobs.
Status	Test was successful.

Evidence:

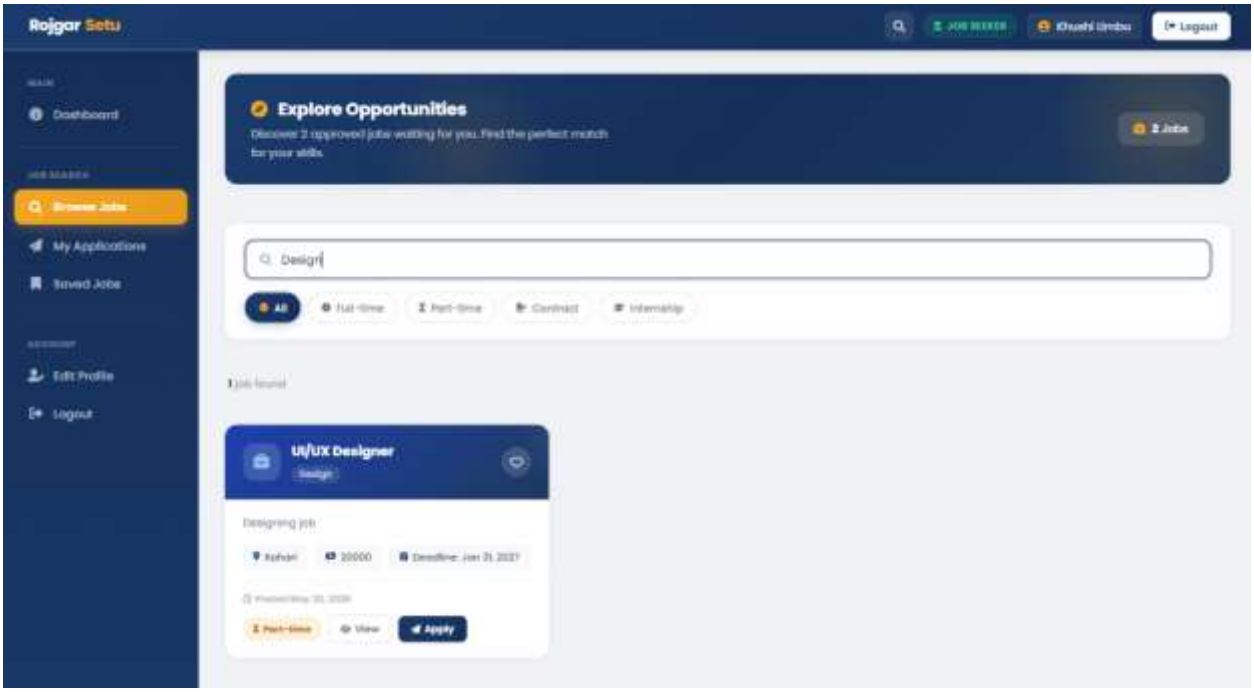


Figure 110 Search by Category

8.5.3 Search by Location

Table 22 Testing Search by Location

Test Case ID	4.3
Test Description	Select location to filter and search for jobs
Expected Result	Only jobs that match the selected location should be displayed in the system.
Actual Result	The jobs for the selected location were successfully displayed.
Status	Test was successful.

Evidence:

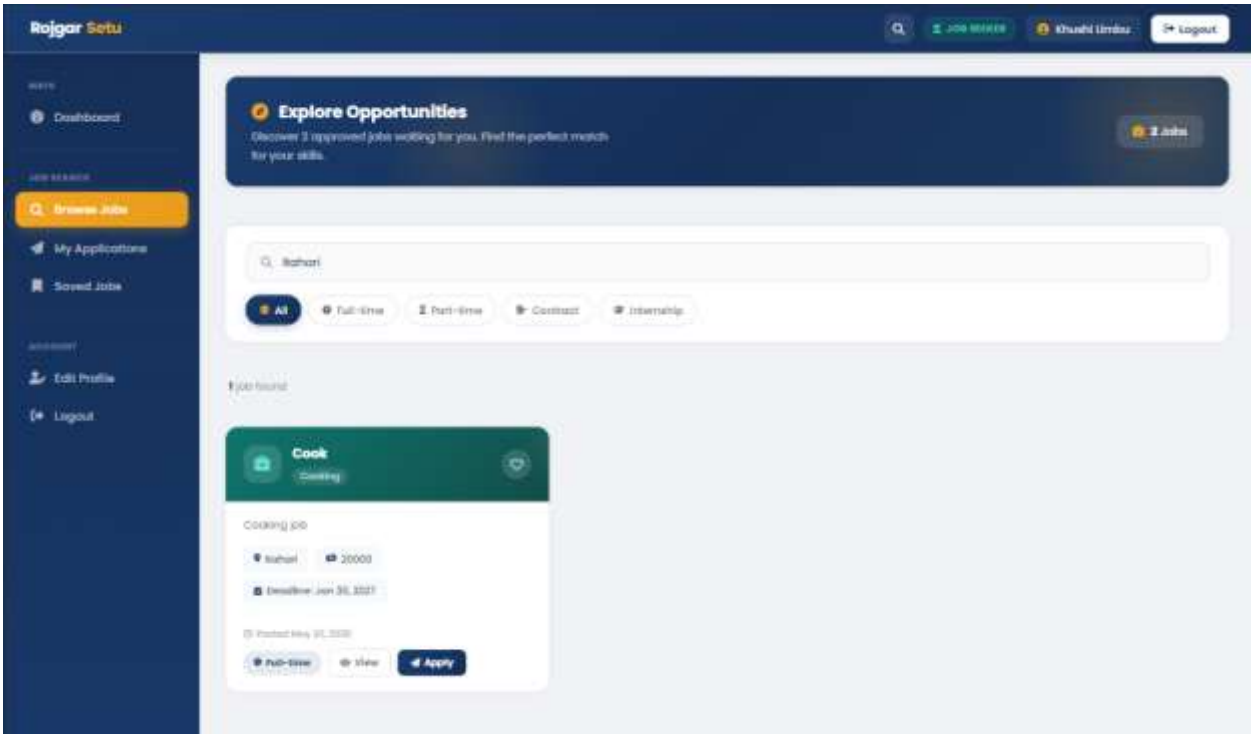


Figure 111 Search by Location

8.6 Test ID 5 Name: Job Application System

Verifies the submission of job applications, duplicate application avoidance, cover letter submission, and job application status.

The testing process for this process is given below:

1. Log in with seeker account.
2. Open job listings (if available).
3. Click apply button.
4. Enter cover letter.
5. Submit job application.
6. Confirm application success message.
7. Try again applying to the same job.
8. Check duplicate application prevention.
9. A seeker can see the status of his/her application in seeker dashboard.

8.6.1 Apply for Job

Table 23 Testing Apply for Job

Test Case ID	5.1
Test Description	Use valid application details to apply for a job
Expected Result	Applications for the job must be submitted successfully.
Actual Result	Job application was successfully submitted.
Status	Test was successful.

Evidence:

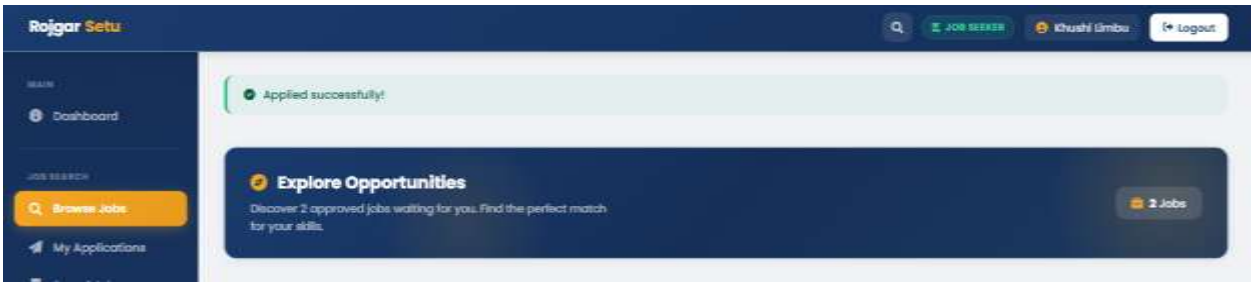


Figure 112 Successful Job Application

8.6.2 Duplicate Application Prevention

Table 24 Testing Duplicate Application Prevention

Test Case ID	5.2
Test Description	Apply for one job various times
Expected Result	The system should not allow to re-apply the same application.
Actual Result	The duplicate application was successfully prevented.
Status	Test was successful.

Evidence:

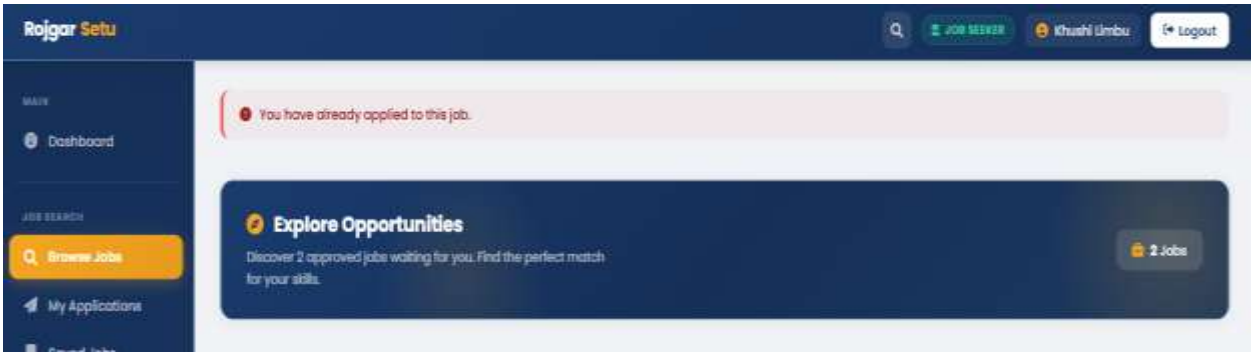


Figure 113 Duplicate Application Prevention

8.6.3 Cover Letter Submission

Table 25 Testing Cover Letter Submission

Test Case ID	5.3
Test Description	Submit cover letter content with job application
Expected Result	The system should be able to submit the cover letter, plus the application.
Actual Result	The cover letter was sent with the job application successfully.
Status	Test was successful.

Evidence:

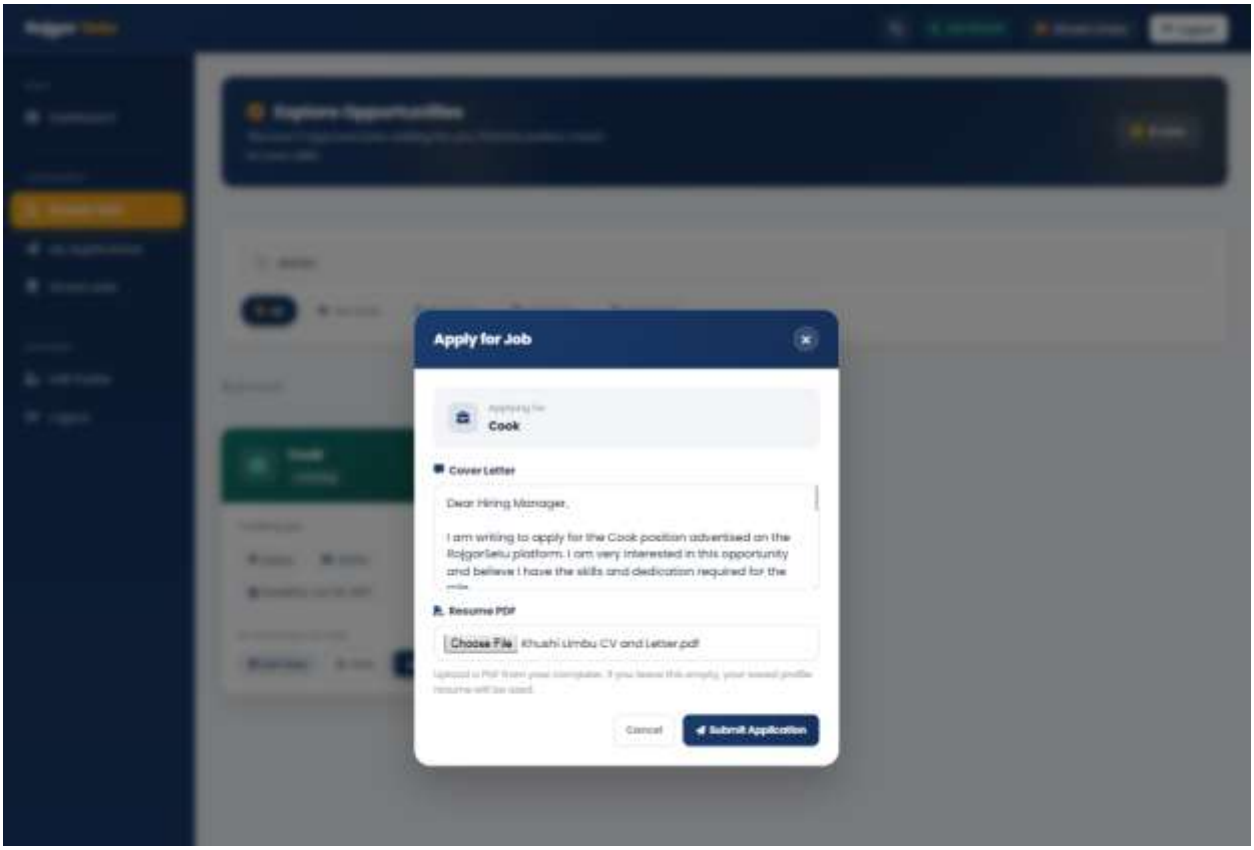


Figure 114 Submitting Cover Letter with Job Application

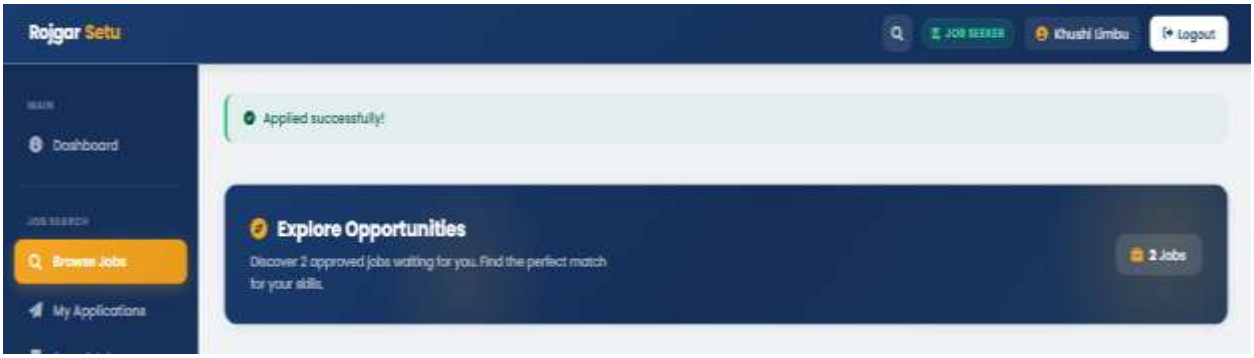


Figure 115 Successful Cover Letter Submission

8.6.4 Application Status Tracking

Table 26 Testing Application Status Tracking

Test Case ID	5.4
Test Description	Check the status of the application after the submission of job application
Expected Result	System should show the current state of the application properly.
Actual Result	Successfully displayed application status.
Status	Test was successful.

Evidence:

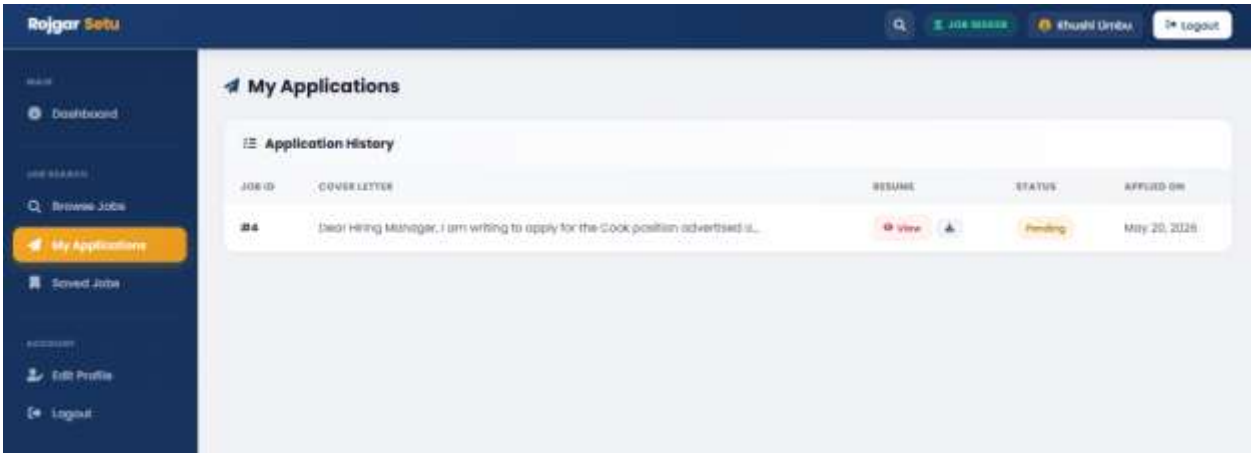


Figure 116 Application Status Tracking

8.7 Test ID 6 Name: Resume Upload System

Tests ensure that the upload resume functionality is supported, resume is validated with PDF and is properly stored and retrieved from the tests.

The testing process for this feature is given below:

1. Log in with seeker account.
2. Open profile or application page.
3. Choose an appropriate PDF resume.
4. Upload Resume.
5. Check the message to see if it has been uploaded successfully.
6. Try uploading invalid file format.
7. Verify the rejection of invalid file.
8. Check resume path in database.
9. Successfully download uploaded resume.

8.7.1 Upload PDF Resume

Table 27 Testing Upload PDF Resume

Test Case ID	6.1
Test Description	Submit PDF resume file with job application
Expected Result	The system should upload and store the PDF resume successfully.
Actual Result	Successfully uploaded and stored PDF resume.
Status	Test was successful.

Evidence:

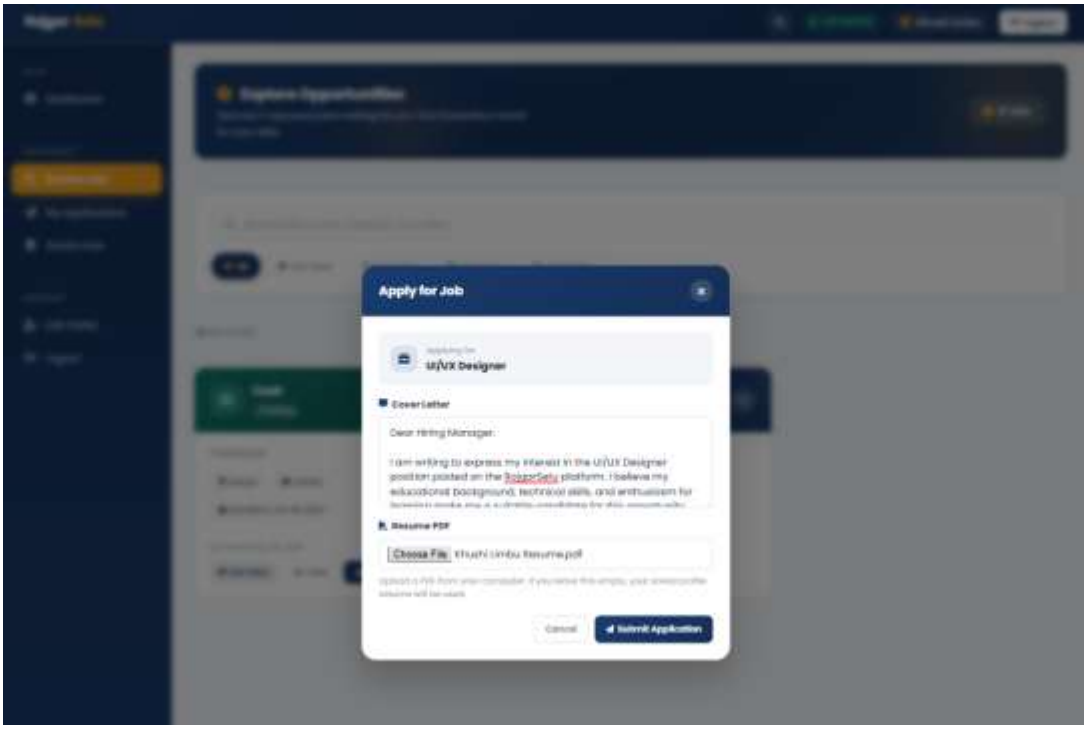


Figure 117 Uploading PDF Resume

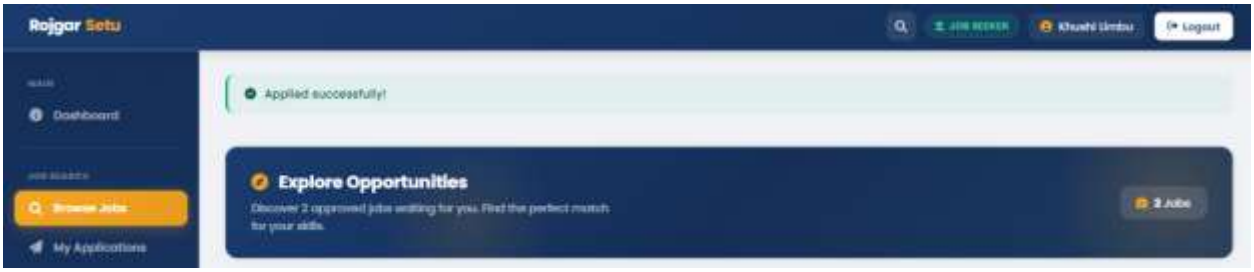


Figure 118 Successful PDF Resume Submission

8.7.2 Invalid File Upload Handling

Table 28 Testing Invalid File Upload Handling

Test Case ID	6.2
Test Description	Try uploading invalid file format
Expected Result	It should not accept any invalid file formats and show a validation message.
Actual Result	File upload was rejected due to not being a valid file.
Status	Test was successful.

Evidence:

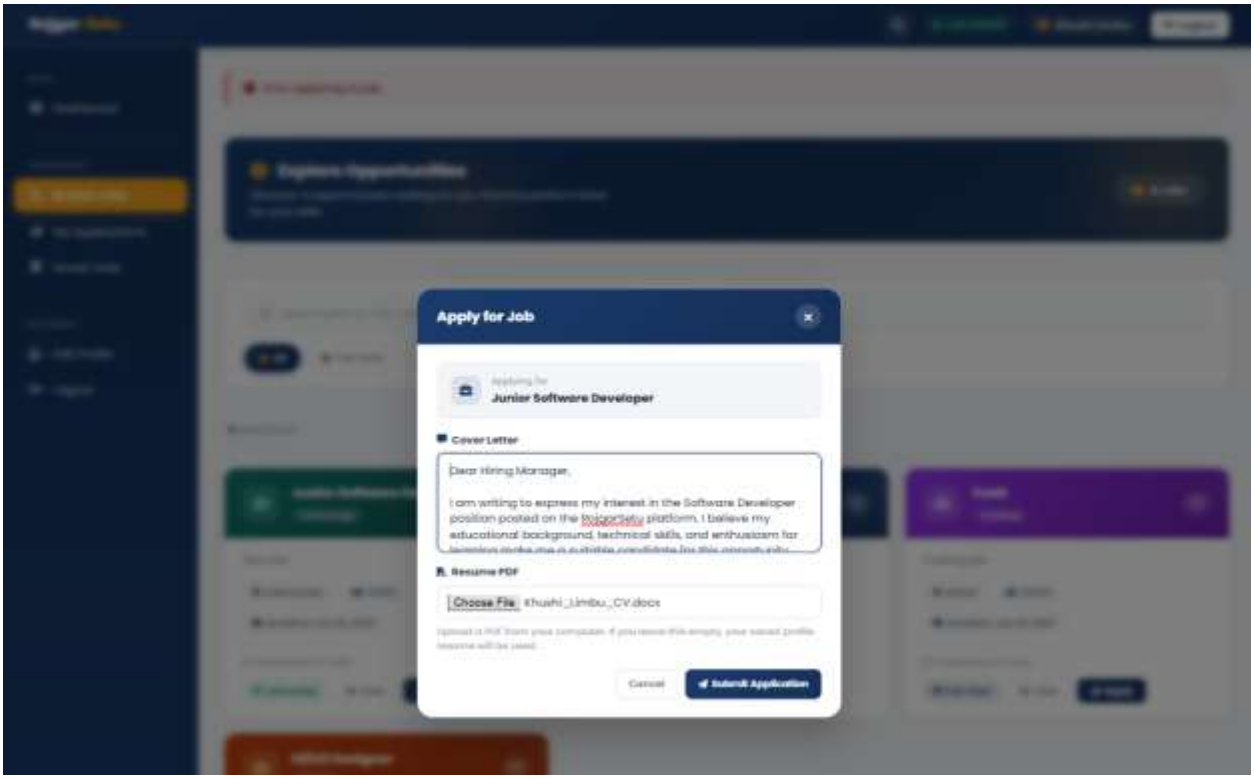


Figure 119 Submitting Invalid File Format

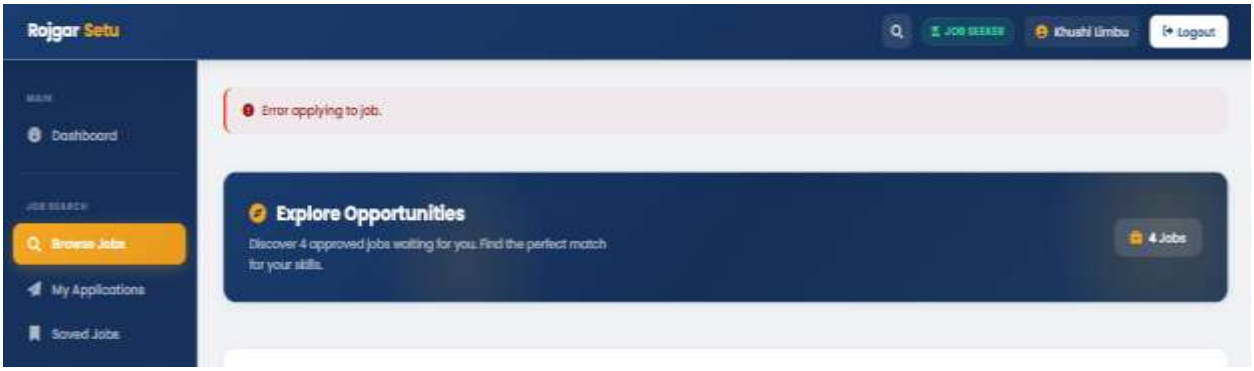


Figure 120 Invalid File Upload Handling

8.7.3 Resume Storage Verification

Table 29 Testing Resume Storage Verification

Test Case ID	6.3
Test Description	Verify correct storage of uploaded resume file path
Expected Result	System shall successfully store the path of the resume that is uploaded into the database.
Actual Result	Resume was successfully stored in database.
Status	Test was successful.

Evidence:

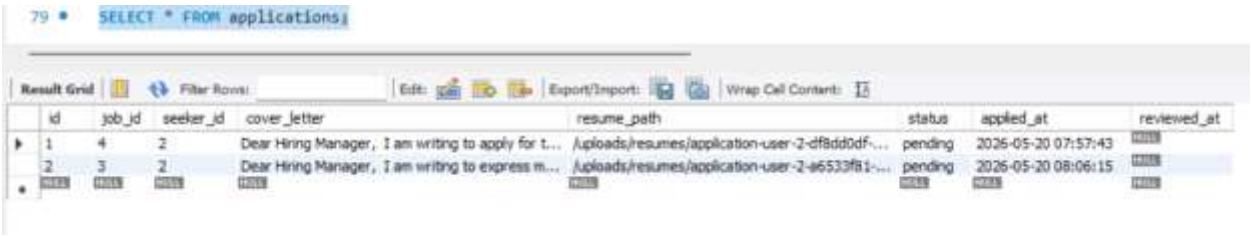


Figure 121 Resume Storage Verification

8.7.4 Resume Retrieval Validation

Table 30 Testing Resume Retrieval Validation

Test Case ID	6.4
Test Description	Retrieve uploaded resume from seeker
Expected Result	The system should be able to properly fetch and show the uploaded resume.
Actual Result	Successfully retrieved the resumes that were uploaded.
Status	Test was successful.

Evidence:

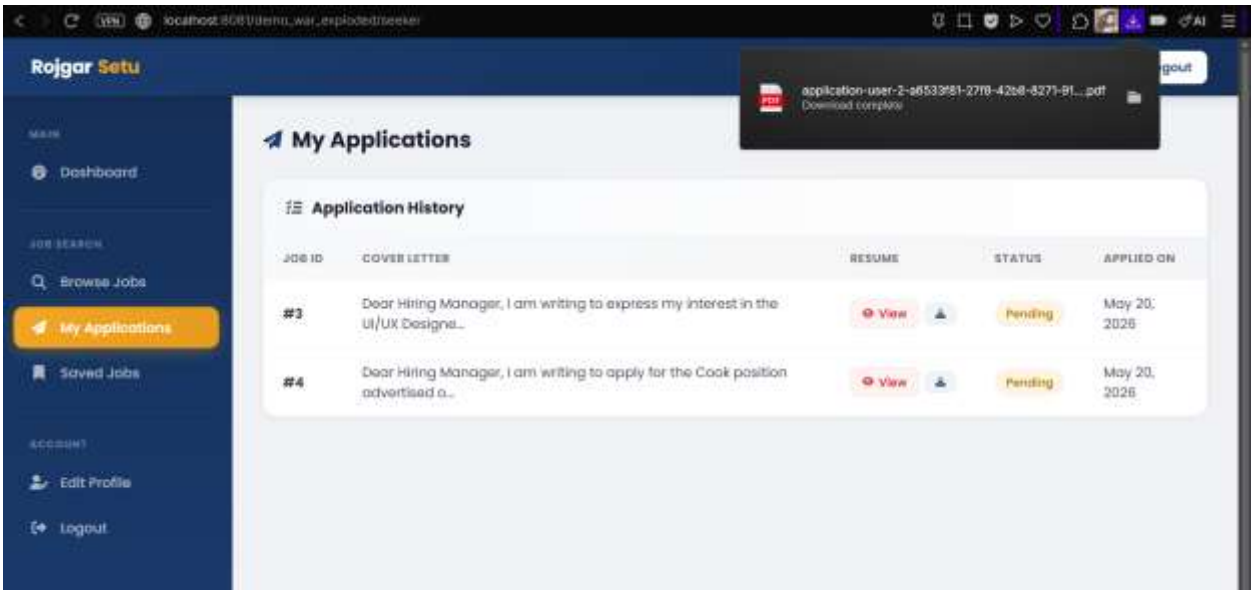


Figure 122 Successful Resume Retrieval

8.8 Test ID 7 Name: Admin User Management

Checks administrator functionality to approve, reject, delete, and manage seeker and employer accounts.

The testing process for this feature is given below:

1. Log in using admin account.
2. Open manage user's page.
3. See pending employer accounts.
4. Approve employer account.
5. Reject employer account.
6. Delete selected user account.
7. Attempt deleting admin account.
8. Verify that admin deletion is prevented.
9. Confirm the status of users.

8.8.1 Approve Employer

Table 31 Testing Approve Employer

Test Case ID	7.1
Test Description	Approve of pending employer account via admin dashboard
Expected Result	The status of the employer's account should change to "approved" successfully.
Actual Result	Employer account was successfully approved.
Status	Test was successful.

Evidence:

Figure 123 Successfully Approved Employer

8.8.2 Reject Employer

Table 32 Testing Reject Employer

Test Case ID	7.2
Test Description	Turn down an employer's account from the admin dashboard
Expected Result	The status of the employer's account should change to "rejected" successfully.
Actual Result	Successfully rejected an employer account.
Status	Test was successful.

Evidence:

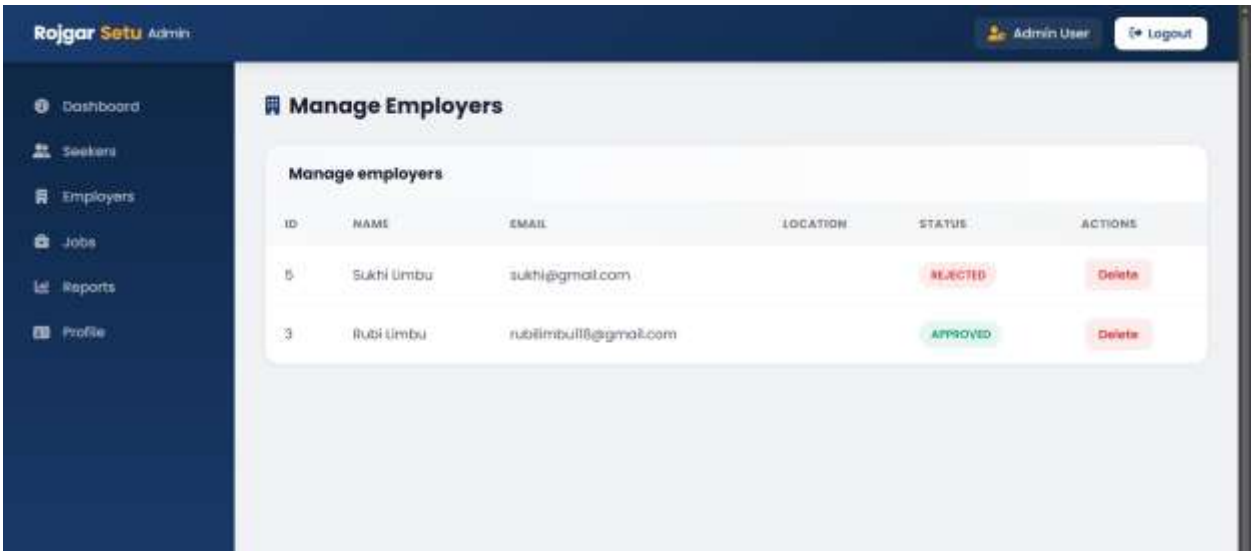


Figure 124 Successfully Rejected Employer

8.8.3 Approve Seeker

Table 33 Testing Approve Seeker

Test Case ID	7.3
Test Description	Accept pending seeker account from admin dashboard
Expected Result	The account status of seeker should change to “approved” successfully.
Actual Result	Seeker account successfully approved.
Status	Test was successful.

Evidence:

Figure 125 Successfully Approved Seeker

8.8.4 Delete User

Table 34 Testing Delete User

Test Case ID	7.4
Test Description	Remove account from admin management page
Expected Result	Selected user account should be successfully deleted.
Actual Result	Successfully deleted the user account.
Status	Test was successful.

Evidence:

Figure 126 Deleting a User

Figure 127 Successful User Removal

8.9 Test ID 8 Name: Admin Job Approval System

Admin approve/reject job postings, moderation and pending job management.

The testing process for this feature is given below:

1. Log in as the admin.
2. Display the list of available open jobs.
3. Check out new jobs that have been posted.
4. Approve pending job.
5. Reject inappropriate job.
6. Remove incorrect job offers.
7. Ensure that jobs appear in the public listing upon approval.
8. Check that rejected jobs are not visible.

8.9.1 Approve Job

Table 35 Testing Approve Job

Test Case ID	8.1
Test Description	Approve pending job posting using admin dashboard
Expected Result	The job posting selected should be approved and posted to the public.
Actual Result	Job posting was successfully posted and made visible to the public.
Status	Test was successful.

Evidence:

Figure 128 Successful Job Approval

8.9.2 Reject Job

Table 36 Testing Reject Job

Test Case ID	8.2
Test Description	Reject a pending job posting using admin dashboard
Expected Result	The job posting selected should be rejected and prevented from being posted to the public.
Actual Result	Successfully rejected job posting.
Status	Test was successful.

Evidence:

Figure 129 Successful Job Rejection

8.9.3 Delete Inappropriate Job

Table 37 Testing Delete Inappropriate Job

Test Case ID	8.3
Test Description	Remove the unsuitable or wrong job posting from admin panel
Expected Result	The chosen job posting should be removed from the system.
Actual Result	Successfully deleted inappropriate job.
Status	Test was successful.

Evidence:

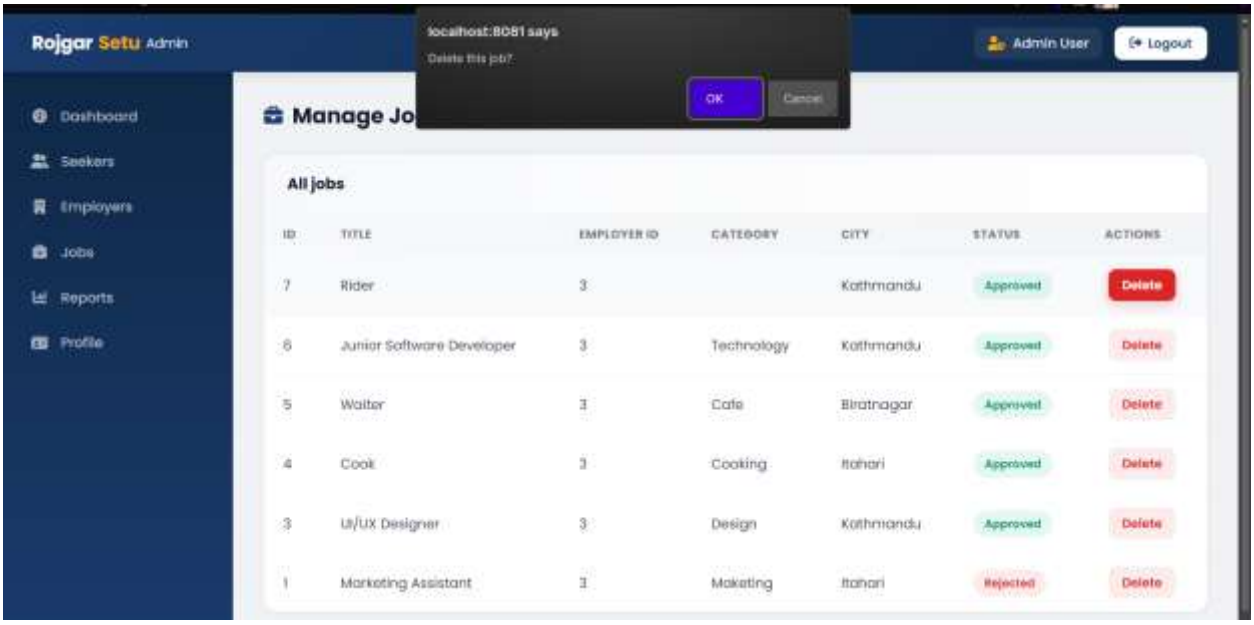


Figure 130 Removing Unsuitable Job Posting

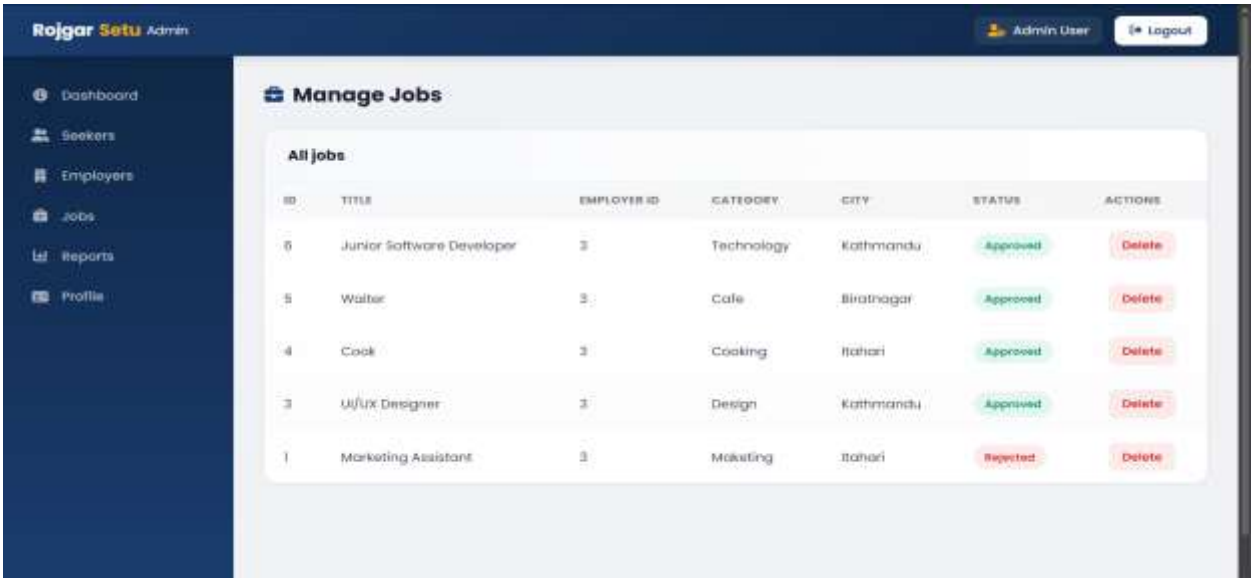


Figure 131 Successful Removal of Job Posting

8.9.4 View Pending Jobs

Table 38 Testing View Pending Jobs

Test Case ID	8.4
Test Description	View all the pending job postings that need to be approved
Expected Result	All jobs that are pending should be shown in the system.
Actual Result	Successfully displayed pending job postings.
Status	Test was successful.

Evidence:

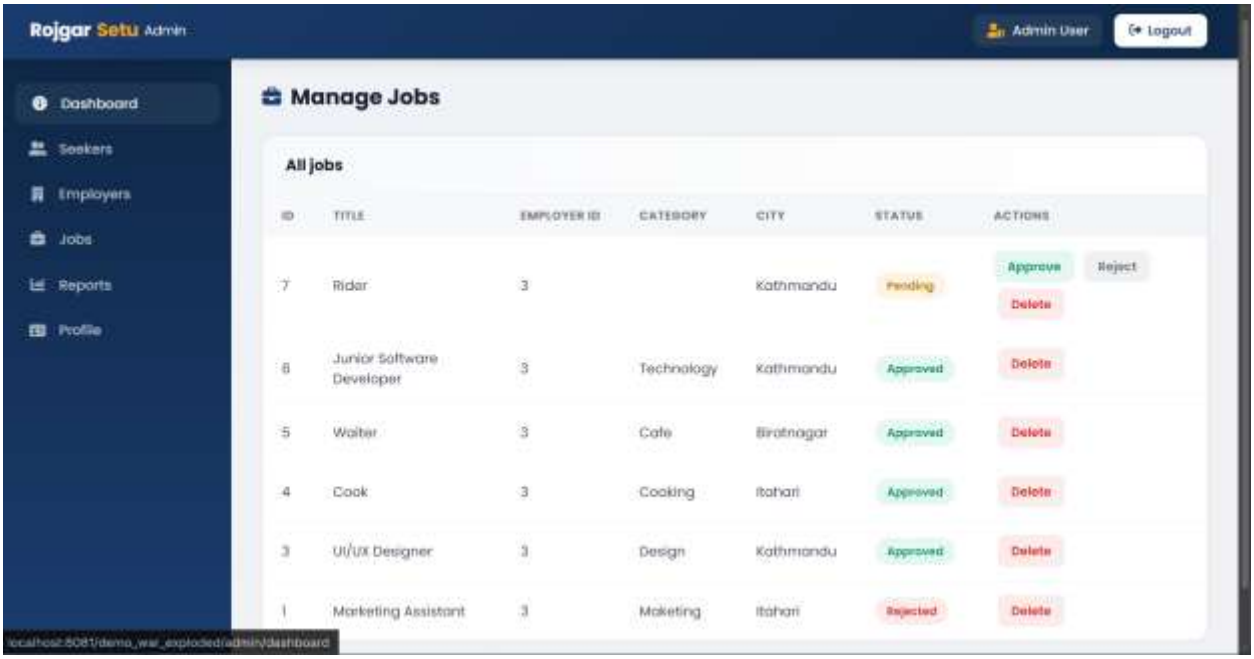


Figure 132 View Pending Jobs

8.10 Test ID 9 Name: Session and Security Management

Verifies session management, unauthorized route restriction, logout, cache prevention, and role-based access control.

The testing process for this process is given below:

1. Log in using valid account.
2. View protected dashboard pages.
3. Log off the computer system.
4. Tap the browser back button.
5. Check that restricted pages can’t be accessed.
6. Try to log on to admin via a seeker account.
7. Try to accessing employer route without login.
8. Verify login page redirection.
9. Check session expiration handling.

8.10.1 Unauthorized Route Access

Table 39 Testing Unauthorized Route Access

Test Case ID	9.1
Test Description	Try to access a dashboard route with no authentication
Expected Result	No unauthorized access to system should be allowed and login page should be displayed.
Actual Result	Unauthorized access was denied successfully.
Status	Test was successful.

Evidence:

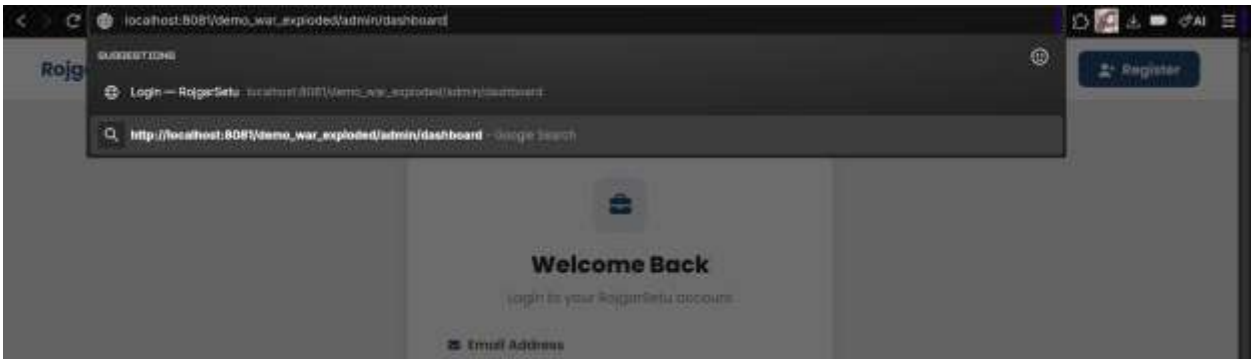


Figure 133 Trying to Access Protected Routes Unauthorized

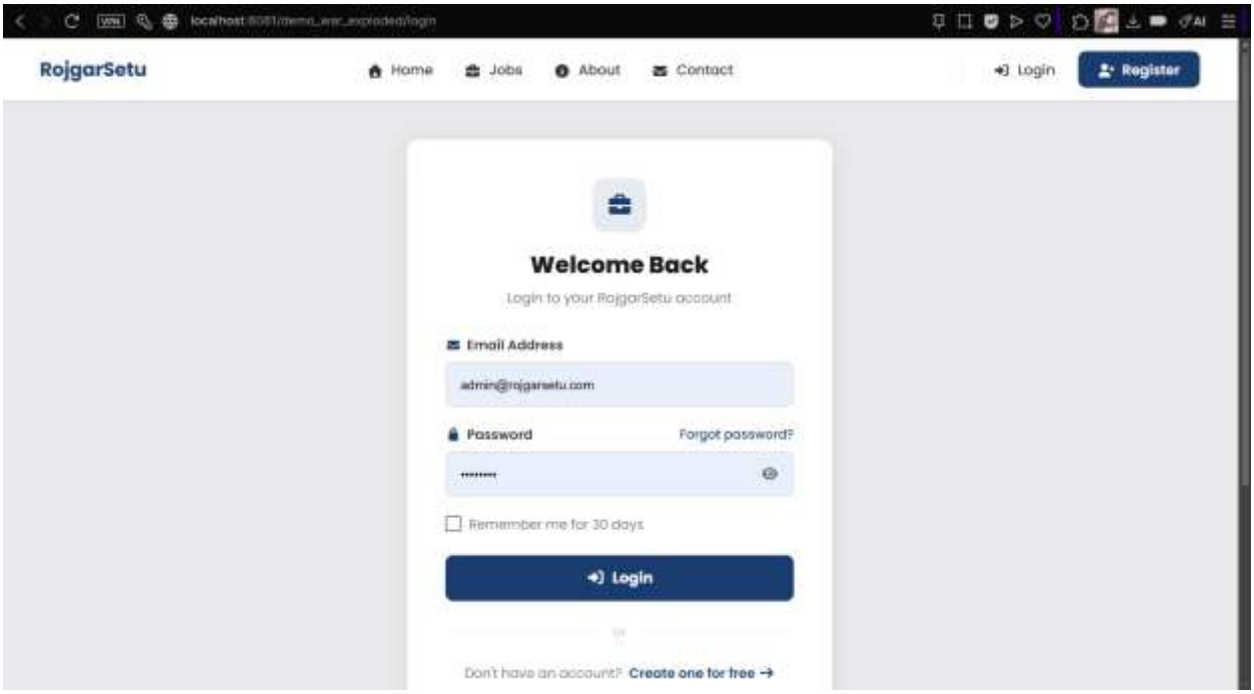


Figure 134 Unauthorized Access Prevented Successfully

8.10.2 Logout Functionality

Table 40 Testing Logout Functionality

Test Case ID	9.2
Test Description	After logging in to the system, log out of the system
Expected Result	User session should end and user redirected to login page.
Actual Result	Successfully logged out and redirected.
Status	Test was successful.

Evidence:

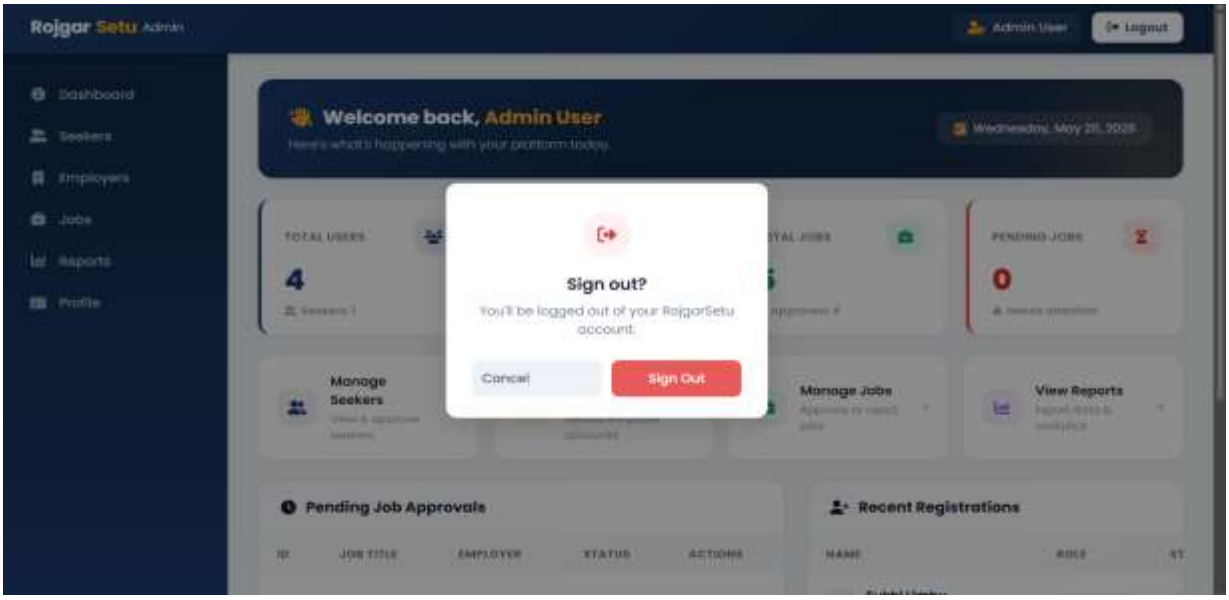


Figure 135 Logging Out

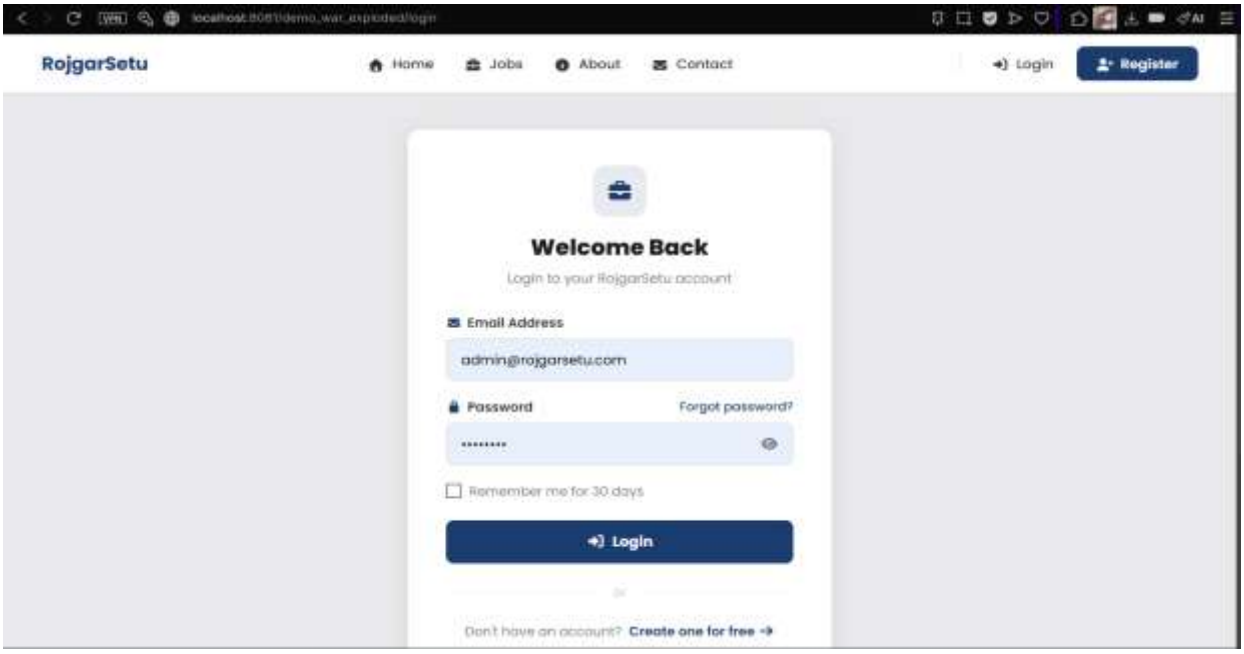


Figure 136 Successfully Logged Out

8.10.3 Browser Back-button Prevention

Table 41 Testing Browser Back-button Prevention

Test Case ID	9.3
Test Description	Click back button of browser after logging out from the system
Expected Result	The pages that were previously access should not be accessible.
Actual Result	Protected pages not accessible after logging out.
Status	Test was successful.

Evidence:

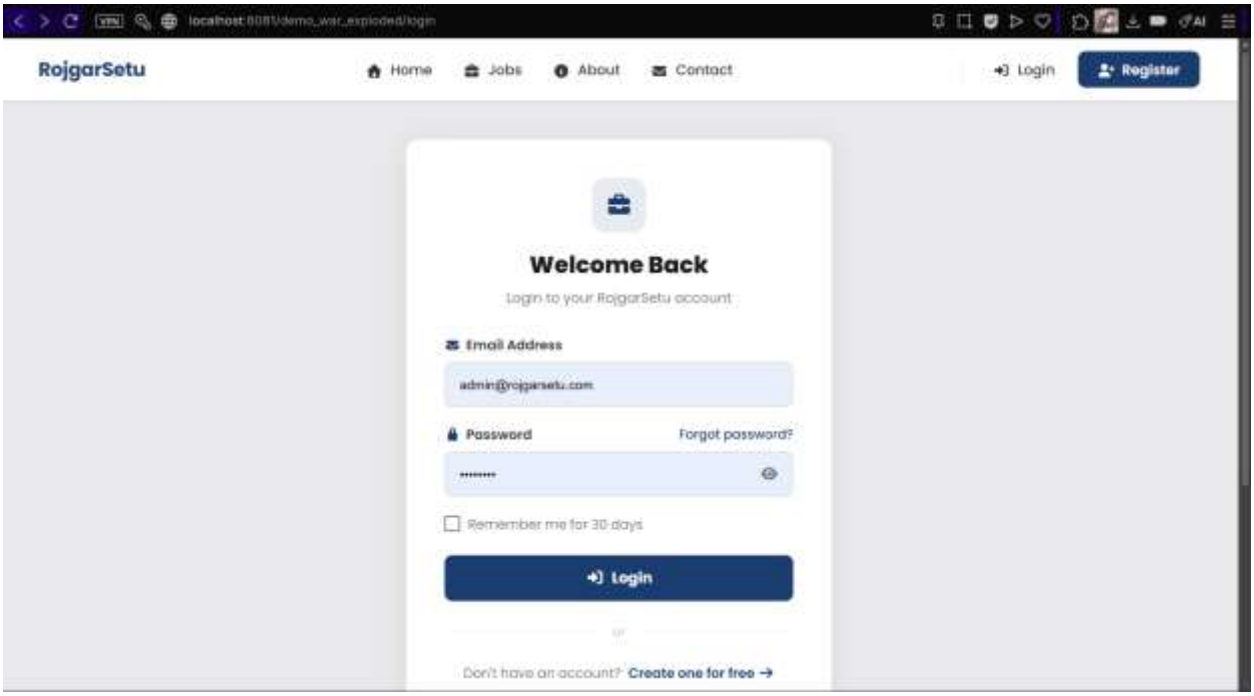


Figure 137 Browser Back-button Prevention Successful

8.10.4 Role-based Route Restriction

Table 42 Testing Role-based Route Restriction

Test Case ID	9.4
Test Description	Try to use an unauthorized role to access employer or admin routes
Expected Result	The unauthorized users should not be given access by the system and should be redirected.
Actual Result	Restriction on unauthorized access to routes was successful.
Status	Test was successful.

Evidence:

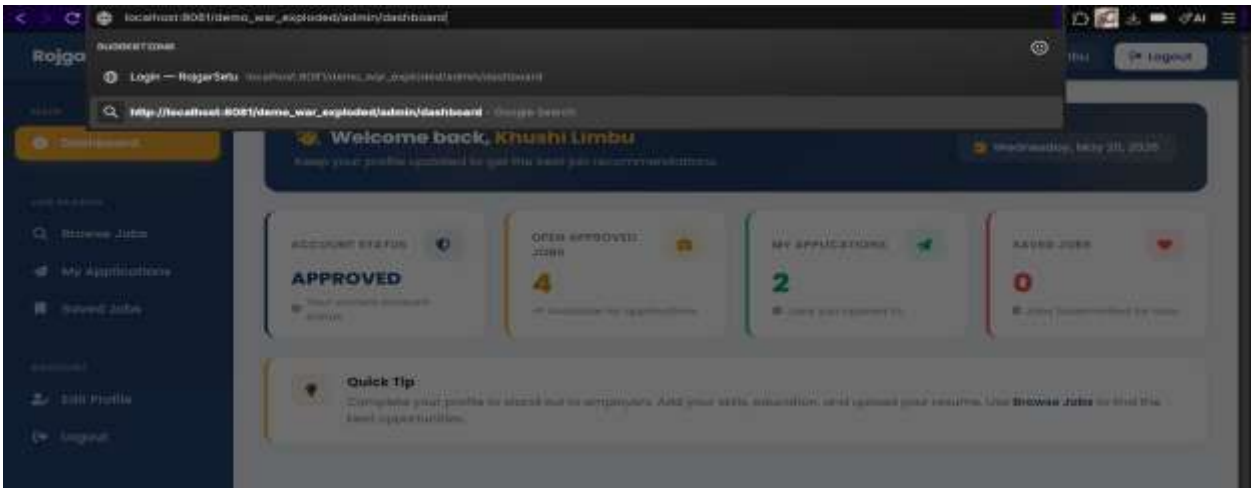


Figure 138 Trying to Access Admin Route with Unauthorized Role

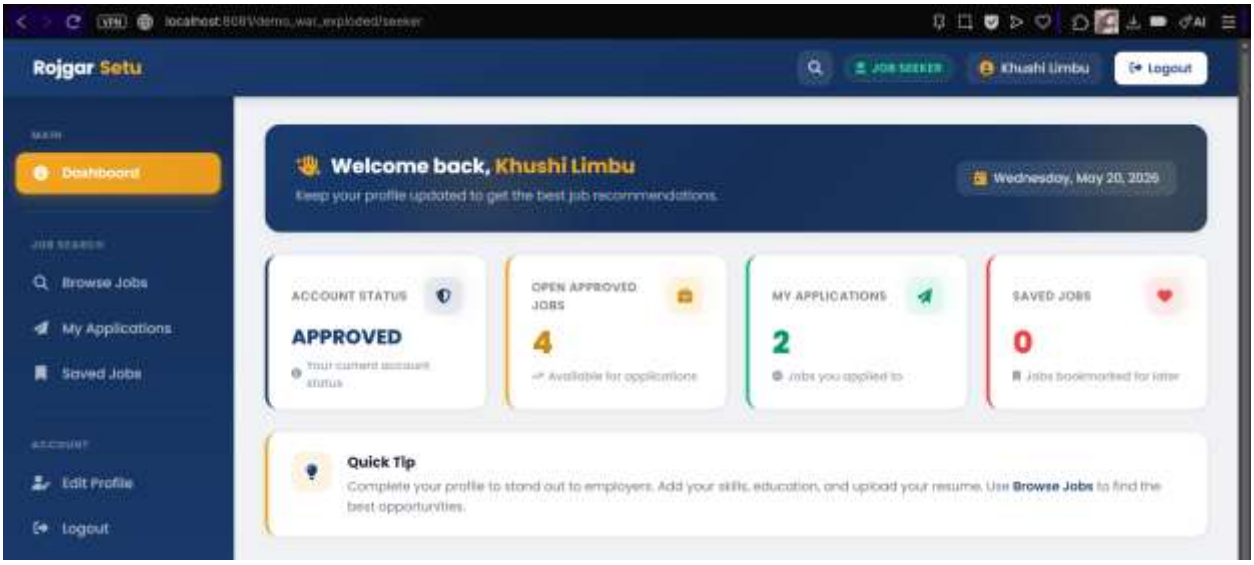


Figure 139 Successful Role-Based Route Restriction

8.11 Test ID 10 Name: Reports and Statistics Dashboard

Evaluate dashboard statistics, aggregate reporting, count of totals, count of pending and export to CSV.

The testing process for this feature is given below:

1. Use admin account to log in.
2. Go to the dashboard statistics page.
3. Check count of total users.
4. Check the number of jobs is accurate.
5. Check the number of jobs pending.
6. Check total count of applications.
7. Generate CSV reports.
8. Download exported reports.
9. Check the accuracy of data exported from a report.

8.11.1 Total Users Count

Table 43 Testing Total Users Count

Test Case ID	10.1
Test Description	Recheck the total number of users registered on admin dashboard.
Expected Result	The system should correctly show the number of users registered.
Actual Result	The total users count was displayed successfully.
Status	Test was successful.

Evidence:

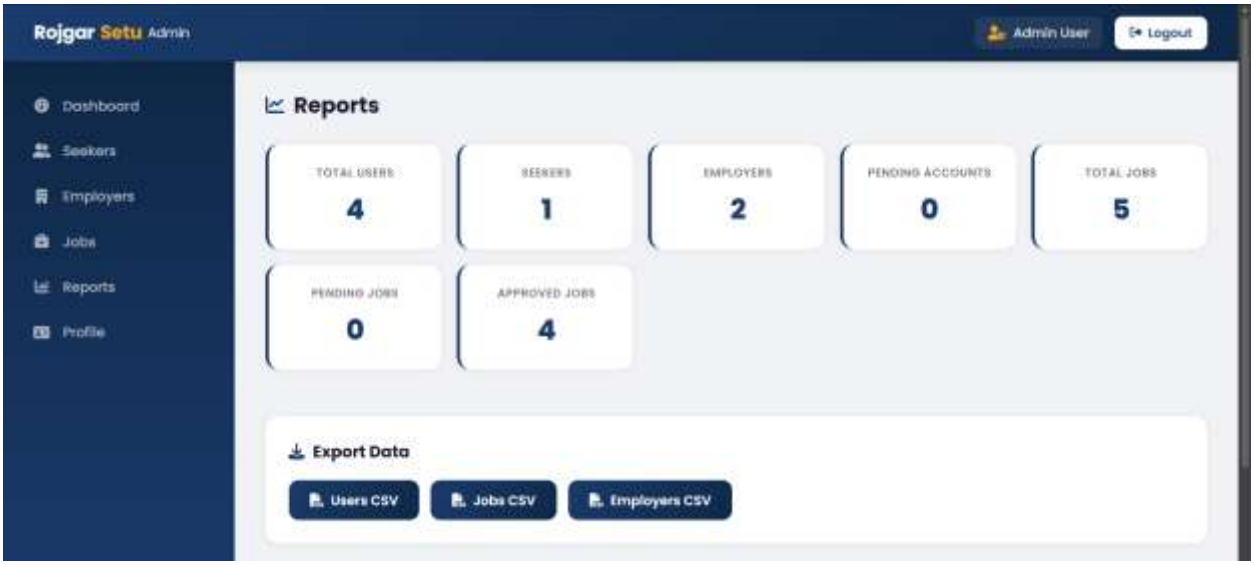


Figure 140 Total Seekers and Employers Count

8.11.2 Total Jobs Count

Table 44 Testing Total Jobs Count

Test Case ID	10.2
Test Description	Recheck the total number of jobs registered on admin dashboard.
Expected Result	The system should correctly show the number of jobs registered.
Actual Result	The total jobs count was displayed successfully.
Status	Test was successful.

Evidence:

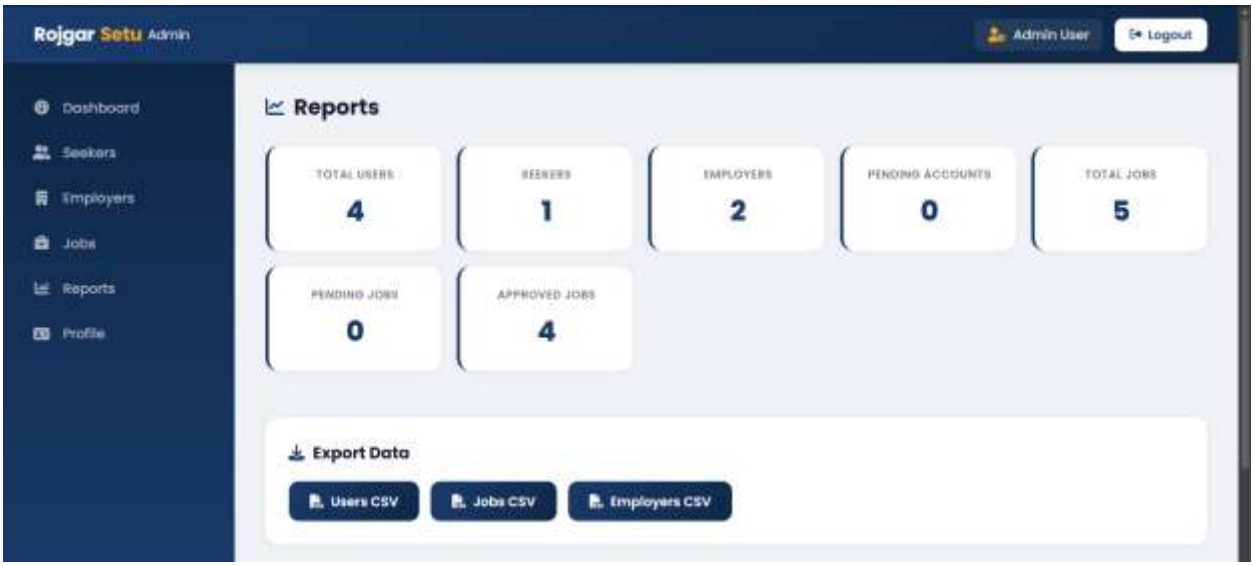


Figure 141 Total Jobs Count

8.11.3 CSV Export Verification

Table 45 Testing CSV Export Verification

Test Case ID	10.3
Test Description	Generate system reports and export to CSV report file
Expected Result	System should be able to create and download a CSV report.
Actual Result	Report exported to CSV was successfully created and downloaded.
Status	Test was successful.

Evidence:

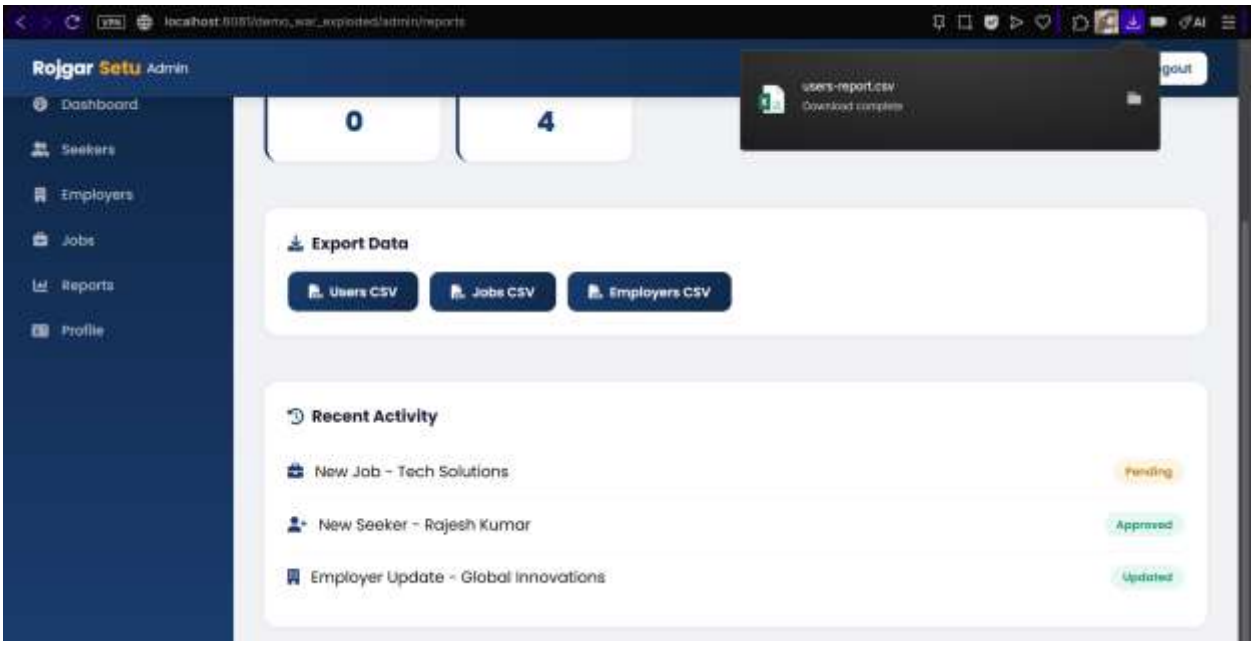


Figure 142 CSV Export Verification

9 Coursework Development with Analysis

This section will explain how the roles played by each team member in the development of the RojgarSetu system. It also explains about the finished work, UI/UX implementation, issues faced while developing, solutions for solving technical issues. The system is developed with Java, JSP, Servlet, JDBC, MySQL, HTML and CSS language based on the MVC design pattern.

9.1 Team Contribution

Table 46 Team Contribution

Member Name	Member Task
Manoj Katuwal	• Admin Dashboard Development • Employer Module Development • Reports and Statistics Module • Database Integration • Documentation
Ayush Chaudhari	• Seeker Module Development • Job Application System • Resume Upload Functionality • Job Search and Filtering
Biman Hang Limbu Subba	• Login Authentication System • Session Management • Role-Based Access Control • Logout and Security Handling
Kristina Gurung	• Registration Module • UI/UX Design • Frontend Styling • Responsive Layout Design
Khushi Limbu	• Public Pages Development • Black-Box Testing • Validation Testing • Debugging and Error Handling

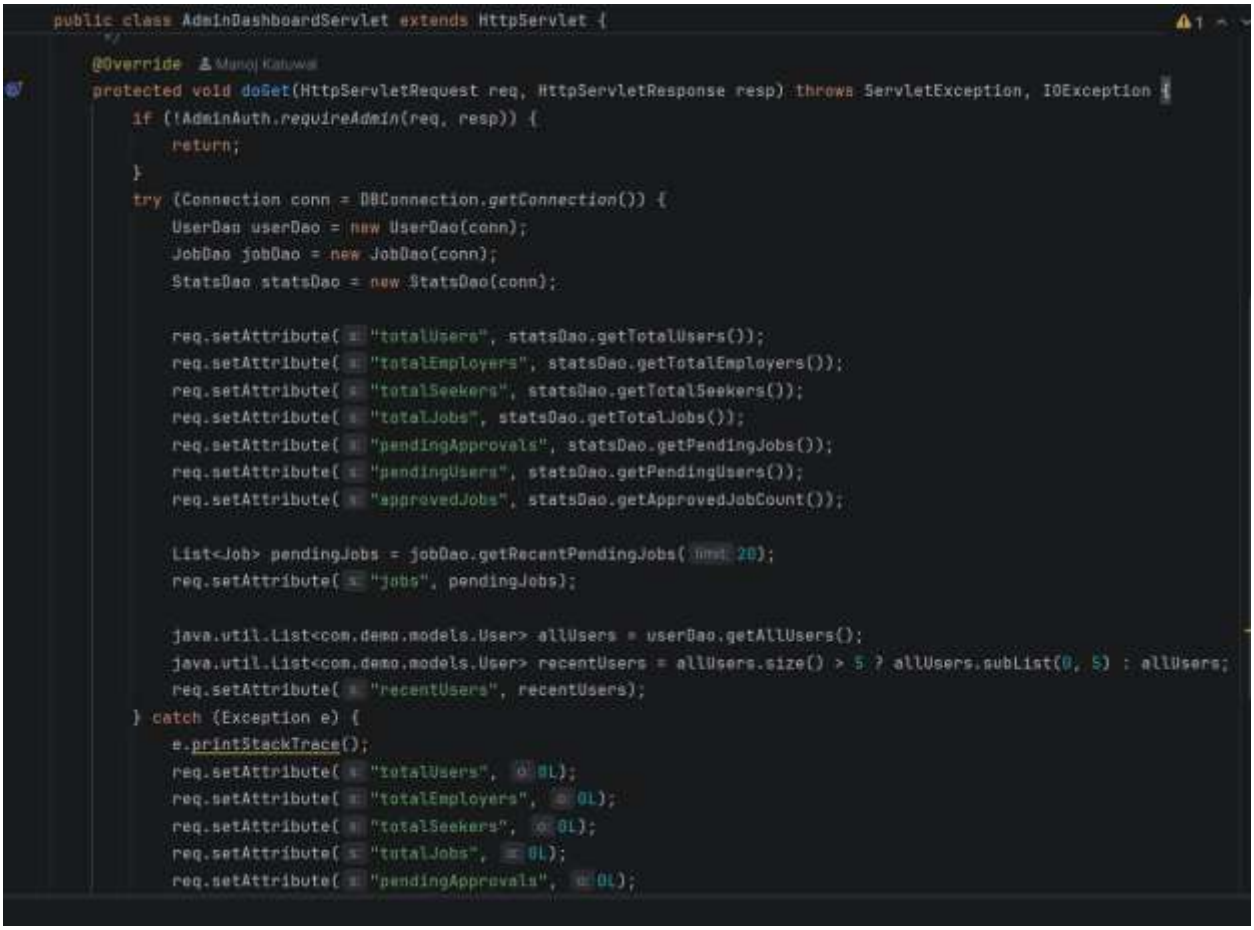
9.1.1 Manoj Katuwal

a. Completed Admin Dashboard Feature

The admin dashboard has been created to give an all-in-one user management system of RojgarSetu. All of the user, employer, job postings, pending approvals, and system reports are managed from a single dashboard by the administrator. The dashboard shows critical information like total users, jobs, pending jobs, and applications.

Java Servlets, JSP, JDBC and MySQL were used to implement the module. In order to safeguard the administrator routes from unauthorized access, session validation, and role-based access control were implemented.

Evidence:



```
public class AdminDashboardServlet extends HttpServlet {

    @Override  Manoj Katuwal
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        if (!AdminAuth.requireAdmin(req, resp)) {
            return;
        }
        try (Connection conn = DBConnection.getConnection()) {
            UserDao userDao = new UserDao(conn);
            JobDao jobDao = new JobDao(conn);
            StatsDao statsDao = new StatsDao(conn);

            req.setAttribute( s: "totalUsers", statsDao.getTotalUsers());
            req.setAttribute( s: "totalEmployers", statsDao.getTotalEmployers());
            req.setAttribute( s: "totalSeekers", statsDao.getTotalSeekers());
            req.setAttribute( s: "totalJobs", statsDao.getTotalJobs());
            req.setAttribute( s: "pendingApprovals", statsDao.getPendingJobs());
            req.setAttribute( s: "pendingUsers", statsDao.getPendingUsers());
            req.setAttribute( s: "approvedJobs", statsDao.getApprovedJobCount());

            List<Job> pendingJobs = jobDao.getRecentPendingJobs( limit: 20);
            req.setAttribute( s: "jobs", pendingJobs);

            java.util.List<com.demo.models.User> allUsers = userDao.getAllUsers();
            java.util.List<com.demo.models.User> recentUsers = allUsers.size() > 5 ? allUsers.subList(0, 5) : allUsers;
            req.setAttribute( s: "recentUsers", recentUsers);
        } catch (Exception e) {
            e.printStackTrace();
            req.setAttribute( s: "totalUsers",  o: 0L);
            req.setAttribute( s: "totalEmployers",  o: 0L);
            req.setAttribute( s: "totalSeekers",  o: 0L);
            req.setAttribute( s: "totalJobs",  o: 0L);
            req.setAttribute( s: "pendingApprovals",  o: 0L);
```

b. Employer Module Development

The employer module was included to enable the employer to post, edit, update and delete jobs. Employers are also able to open applications by seekers and control job statuses.

The module includes:

- Job posting feature
- Job editing and removal
- Pending approval workflow
- Employer profile management
- Application management

In order to prevent invalid or incomplete job postings, validation checks were implemented.

Evidence:

```
if (!EmployerAuth.requireEmployer(req, resp))
    return;
```

```
req.setAttribute( s: "employerUser", userDao.getUserById(userId));
req.setAttribute( s: "employerProfile", employerDao.getEmployerProfile(userId));
req.setAttribute( s: "totalJobsPosted", statsDao.getJobCountByEmployer(userId));
req.setAttribute( s: "totalApplicants", appDao.getApplicationCountByEmployer(userId))
req.setAttribute( s: "pendingJobs", statsDao.getPendingJobCountByEmployer(userId));
req.setAttribute( s: "activeJobs", statsDao.getActiveJobCountByEmployer(userId));
req.setAttribute( s: "recentJobs", jobDao.getRecentJobsByEmployer(userId, limit: 5));
```

```
if ("postjob".equals(action)) {
    handlePostJob(req, resp, session, userId);
    return;
```

c. Reports and Statistics Module

The reports and statistics module were created to provide system level summaries and CSV downloading reports. Total Users, Job, Applications and Pending Approvals were calculated using aggregate SQL queries.

The functionality of exporting data in CSV format in Java was realized with the help of the Java PrintWriter and file streaming.

Evidence:

```
req.setAttribute( s: "totalUsers", statsDao.getTotalUsers());
req.setAttribute( s: "totalEmployers", statsDao.getTotalEmployers());
req.setAttribute( s: "totalSeekers", statsDao.getTotalSeekers());
req.setAttribute( s: "totalJobs", statsDao.getTotalJobs());
req.setAttribute( s: "pendingApprovals", statsDao.getPendingJobs());
req.setAttribute( s: "pendingUsers", statsDao.getPendingUsers());
req.setAttribute( s: "approvedJobs", statsDao.getApprovedJobCount());
```

d. Critical Analysis

Challenges

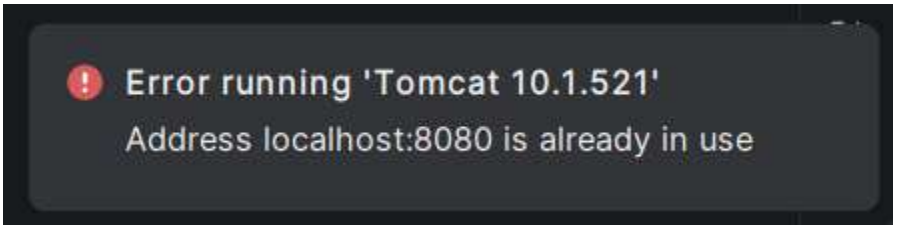
Challenge 1: Apache Tomcat Deployment Conflict

The Apache Tomcat server started, but was unable to start up because port 8080 and 1090 were already in use by another process.

Solution:

The Tomcat configuration was changed to the other available port (e.g., 8082). The conflicting processes were found and killed with the command line tools.

Evidence:



```
20-May-2026 19:18:01.665 INFO [main] org.apache.catalina.core.StandardEngine.startInternal Starting Servlet engine:
20-May-2026 19:18:01.686 INFO [main] org.apache.coyote.AbstractProtocol.start Starting ProtocolHandler ["http-nio-8
20-May-2026 19:18:01.739 INFO [main] org.apache.catalina.startup.Catalina.start Server startup in [139] milliseconds
Connected to server
[2026-05-20 07:18:01,843] Artifact demo:war exploded: Artifact is being deployed, please wait…
20-May-2026 19:18:02.824 INFO [RMI TCP Connection(2)-127.0.0.1] org.apache.jasper.servlet.TldScanner.scanJars At le
[2026-05-20 07:18:02,875] Artifact demo:war exploded: Artifact is deployed successfully
[2026-05-20 07:18:02,875] Artifact demo:war exploded: Deploy took 1,032 milliseconds
20-May-2026 19:18:11.699 INFO [Catalina-utility-2] org.apache.catalina.startup.HostConfig.deployDirectory Deploying
20-May-2026 19:18:11.783 INFO [Catalina-utility-2] org.apache.catalina.startup.HostConfig.deployDirectory Deployment
```

Challenge 2: Unauthorized Dashboard Access

Users could access protected routes on the dashboard without the proper authentication code in the beginning by direct URLs.

Solution:

Restricting access and redirecting to the login page, role-based authentication filters and session validation checks were put in place.

Evidence:

```
    public static boolean requireAdmin(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        HttpSession session = req.getSession( b: false);
        if (session == null) {
            resp.sendRedirect( location: req.getContextPath() + "/login");
            return false;
        }
        Object role = session.getAttribute( s: "userRole");
        if (role == null || !"ADMIN".equalsIgnoreCase(String.valueOf(role))) {
            resp.sendRedirect( location: req.getContextPath() + "/login");
            return false;
        }
        return true;
    }
}
```

9.1.2 Ayush Chaudhari

a. Completed Seeker Module Feature

The seeker module was created to enable seeker to browse, search, filter, apply for and manage jobs. The module offers a simple and convenient interface for the interaction of job-seekers with job opportunities available in the system.

The module includes:

- Job browsing
- Job filtering
- Job application submission
- Cover letter submission
- Application tracking

Evidence:

```
switch (page) {
    case "profile":
        handleProfilePage(req, resp, userId);
        break;
    case "applications":
        handleApplicationsPage(req, resp, userId);
        break;
    case "saved":
        handleSavedPage(req, resp, userId);
        break;
    case "browse":
        handleBrowsePage(req, resp, userId);
        break;
    default:
        handleDashboardPage(req, resp, userId);
        break;
```

```
req.setAttribute( s: "applicationCount", appDao.getApplicationCountBySeeker(userId));

req.setAttribute( s: "savedJobsCount", seekerDao.getSavedJobCount(userId));
```

b. Resume Upload Functionality

Multipart form handling technique has been employed to implement the resume upload system. PDF resumes uploaded are securely stored and associated with seekers profile and job applications.

The module checks the uploaded files and blocks illegal file extensions.

Evidence:

```
public class ApplicationDao {  14 usages  Manoj Katuwal                                    25
    public boolean createApplication(Application app) {  1 usage  Manoj Katuwal
        try {
            ensureResumePathColumn();
            String sql = "INSERT INTO applications(job_id, seeker_id, cover_letter, resume_path, status) VALUES(?,?,?,?,?)";
            try (PreparedStatement ps = conn.prepareStatement(sql)) {
                ps.setInt( parameterIndex: 1, app.getJobId());
                ps.setInt( parameterIndex: 2, app.getSeekerId());
                ps.setString( parameterIndex: 3, app.getCoverLetter());
                ps.setString( parameterIndex: 4, app.getResumePath());
                ps.setString( parameterIndex: 5, app.getStatus() != null ? app.getStatus() : "pending");
                return ps.executeUpdate() > 0;
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return false;
    }
```

c. Critical Analysis

Challenges

Challenge 1: Duplicate Job Applications

At first, each one of the seekers were allowed to submit several times to the same job posting, which resulted in multiple entries in the database.

Solution:

Prior to the application's records being inserted into the database, duplicate validation logic was added. System verifies if a seeker has applied to a particular job.

Evidence:

```
/** Check if seeker already applied to a job. */
public boolean hasApplied(int seekerId, int jobId) {  1 usage   Manoj Katuwal
    try {
        String sql = "SELECT id FROM applications WHERE seeker_id = ? AND job_id = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt( parameterIndex: 1, seekerId);
            ps.setInt( parameterIndex: 2, jobId);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next();
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return false;
}
```

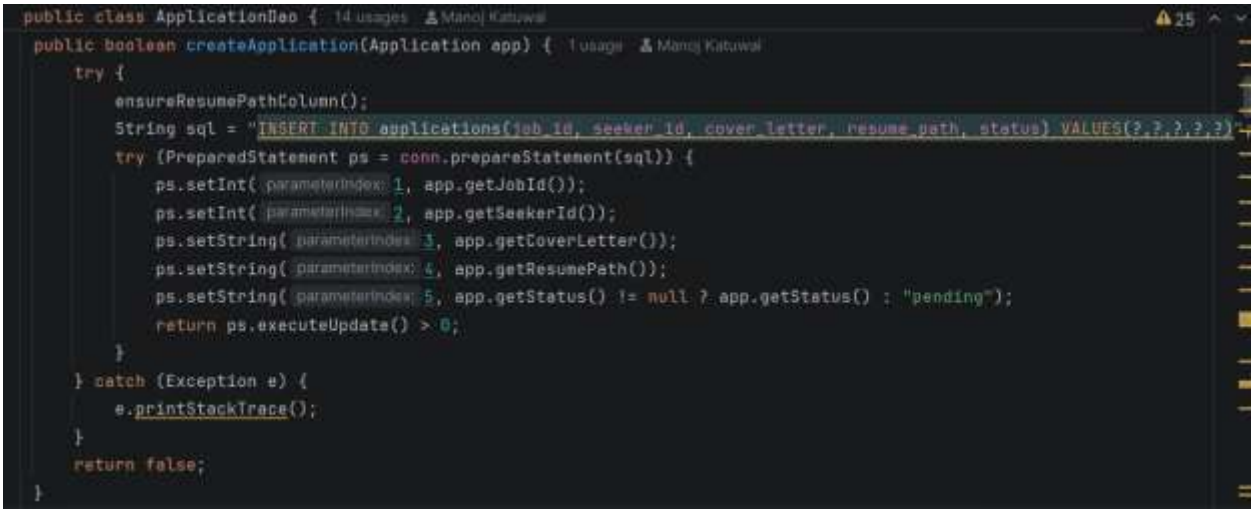
Challenge 2: Resume Storage Handling

During development, it wasn't easy to handle with uploaded resume paths or to retrieve uploaded files correctly.

Solution:

The file paths of the resumes were stored and retrieved dynamically by seeker profile and application modules.

Evidence:



```
public class ApplicationDao {  14 usages  Manoj Katuwal                                    25
    public boolean createApplication(Application app) {  1 usage  Manoj Katuwal
        try {
            ensureResumePathColumn();
            String sql = "INSERT INTO applications(job_id, seeker_id, cover_letter, resume_path, status) VALUES(?,?,?,?,?)";
            try (PreparedStatement ps = conn.prepareStatement(sql)) {
                ps.setInt( parameterIndex: 1, app.getJobId());
                ps.setInt( parameterIndex: 2, app.getSeekerId());
                ps.setString( parameterIndex: 3, app.getCoverLetter());
                ps.setString( parameterIndex: 4, app.getResumePath());
                ps.setString( parameterIndex: 5, app.getStatus() != null ? app.getStatus() : "pending");
                return ps.executeUpdate() > 0;
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return false;
    }
```

9.1.3 Biman Hang Limbu Subba

a. Completed Login Feature

Java Servlets and HttpSession management were used to create the login authentication system. Access is based on email and password credentials and access is redirected based on the role.

The module includes:

- Login validation
- Session creation
- Role-based redirection
- Unauthorized access prevention
- Logout feature

Evidence:

```
HttpSession session = req.getSession();
session.setAttribute( s: "user", user);
session.setAttribute( s: "userId", user.getId());
session.setAttribute( s: "userRole", user.getRole());
```

```
} else if ("SEEKER".equalsIgnoreCase(user.getRole())) {
    if ("browse".equalsIgnoreCase(next)) {
        resp.sendRedirect( location: req.getContextPath() + "/seeker?page=browse");
    } else {
        resp.sendRedirect( location: req.getContextPath() + "/seeker");
```

b. Session and Security Management

Mechanisms to prevent access to protected routes have been added to the session. Employer/Seeker Dashboard routes were secured with Authentication filters.

Prevention methods were also put in place to prevent previously accessed pages from opening after logging out, through browser cache prevention techniques.

Evidence:

```
Object role = session.getAttribute( s: "userRole");
if (role == null || !"EMPLOYER".equalsIgnoreCase(String.valueOf(role))) {
    resp.sendRedirect( location: req.getContextPath() + "/login");
    return false;
}
```

c. Critical Analysis

Challenges

Challenge 1: Session Expiration Problem

In the beginning, after logging out, protected pages could still be accessed from the browser's history.

Solution:

A NoCacheFilter was added to prevent browser caching and allow protected pages to no longer be re-opened once the user has logged out.

Evidence:

```
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {

        HttpServletResponse httpResp = (HttpServletResponse) response;
        httpResp.setHeader( s: "Cache-Control", s1: "no-cache, no-store, must-revalidate");
        httpResp.setHeader( s: "Pragma", s1: "no-cache");
        httpResp.setDateHeader( s: "Expires", l: 0);

        chain.doFilter(request, response);
    }
}
```

Challenge 2: Role-Based Route Restriction

Unauthorized users could be allowed to try to access restricted routes by entering the dashboard urls by hand.

Solution:

Restriction of access based on the user roles was implemented using authentication guard utilities and session validation logic.

Evidence:

```
Object role = session.getAttribute( s: "userRole");
if (role == null || !"EMPLOYER".equalsIgnoreCase(String.valueOf(role))) {
    resp.sendRedirect( location: req.getContextPath() + "/login");
    return false;
}
return true;
```

```
Object role = session.getAttribute( s: "userRole");
if (role == null || !"SEEKER".equalsIgnoreCase(String.valueOf(role))) {
    resp.sendRedirect( location: req.getContextPath() + "/login");
    return false;
}
return true;
```

9.1.4 **Kristina Gurung**

a. Completed Registration Module

The registration system is designed for both parties (seekers and employers). Email formats, duplicate accounts, all required fields were validated.

During the registration process, user accounts are successfully created and after a successful registration, users are redirected properly.

Evidence:

```
User user = new User();
user.setFullName(req.getParameter( s: "fullName"));
user.setEmail(req.getParameter( s: "email"));
user.setPassword(req.getParameter( s: "password"));
user.setRole(role);
user.setLocation(req.getParameter( s: "city"));
```

```
if ("SEEKER".equalsIgnoreCase(role)) {
    seeker = new SeekerProfile();
    seeker.setEducation(req.getParameter( s: "education"));
    seeker.setSkills("");
} else if ("EMPLOYER".equalsIgnoreCase(role)) {
    employer = new EmployerProfile();
    employer.setCompanyName(req.getParameter( s: "companyName"));
    employer.setCompanyCategory(req.getParameter( s: "industry"));
}
```

b. UI/UX Design for System Pages

The frontend application was created with the use of JSP, HTML and CSS. User experience was enhanced with responsive layouts, flexbox, spacing and consistent colour schemes.

The following pages were designed:

- Home page
- Login page
- Registration page
- Dashboard pages
- Public pages

c. Critical Analysis

Challenges

Challenge 1: Responsive Layout Issues

At first, some pages had different layouts when viewed on the smaller screen sizes.

Solution:

To ensure compatibility across devices, responsive CSS techniques were used such as flexbox and media queries.

Evidence:

```
.mission-vision {
    display: flex;
    gap: 30px;
    margin-bottom: 60px;
    animation: fadeInUp 0.6s ease-out 0.2s both;
}
```

```
@media (max-width: 768px) {
    .mission-vision {
        flex-direction: column;
        gap: 20px;
    }
```

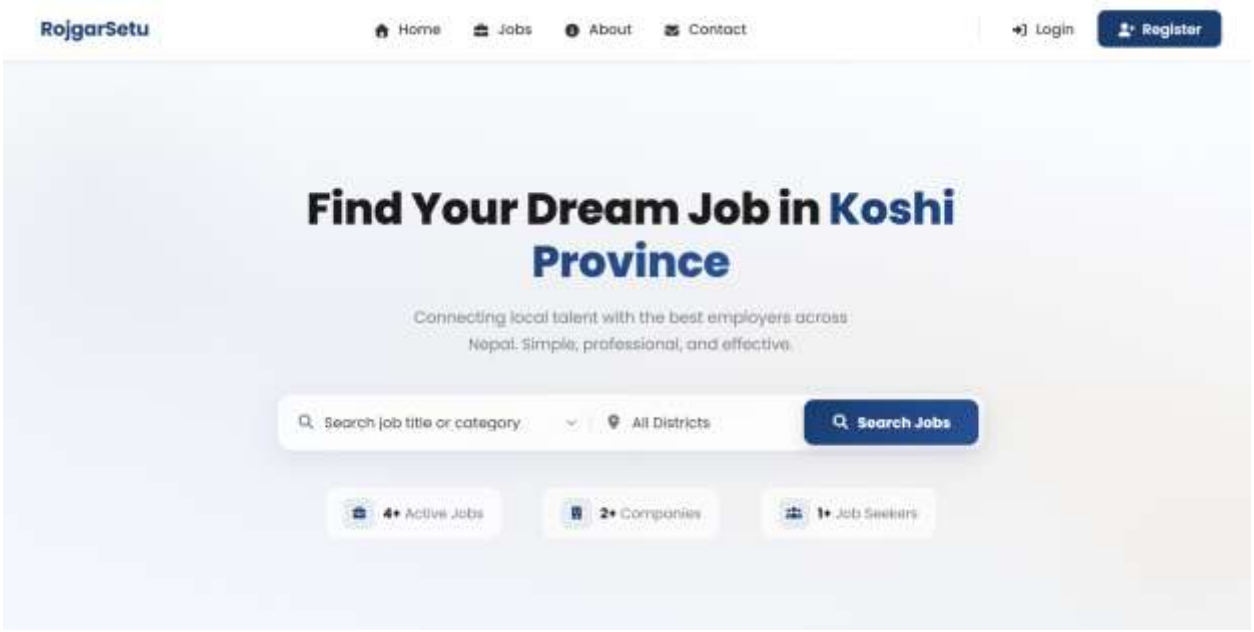
```
@media (max-width: 480px) {
```

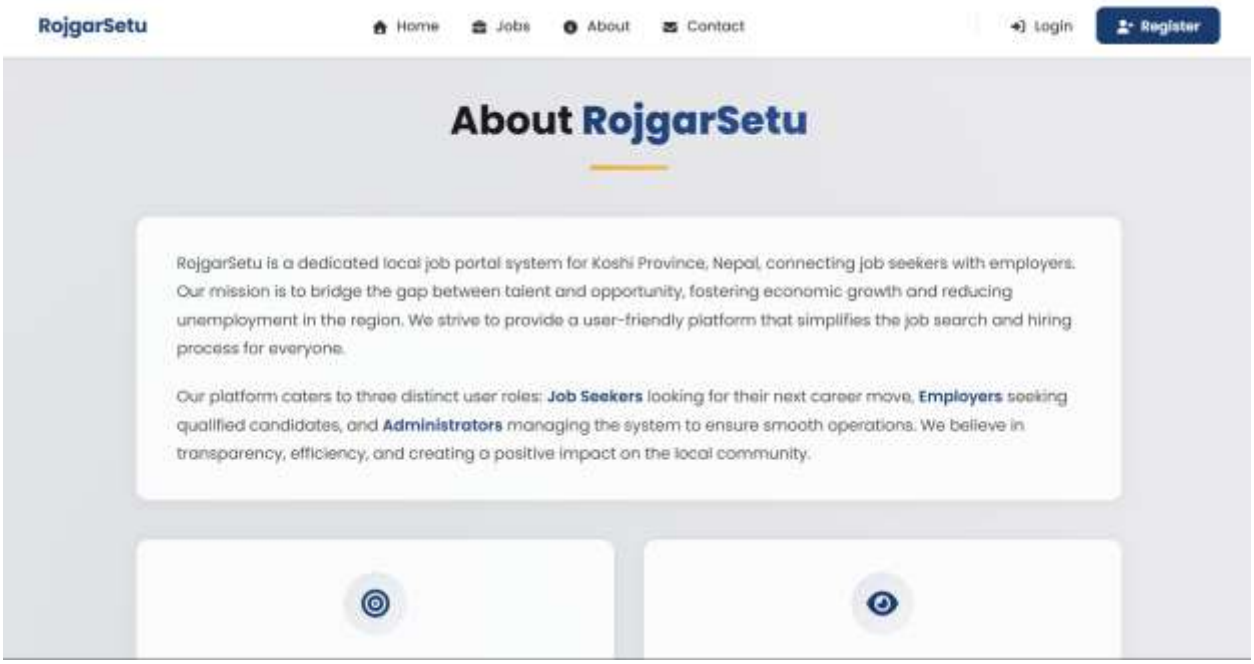
9.1.5 Khushi Limbu

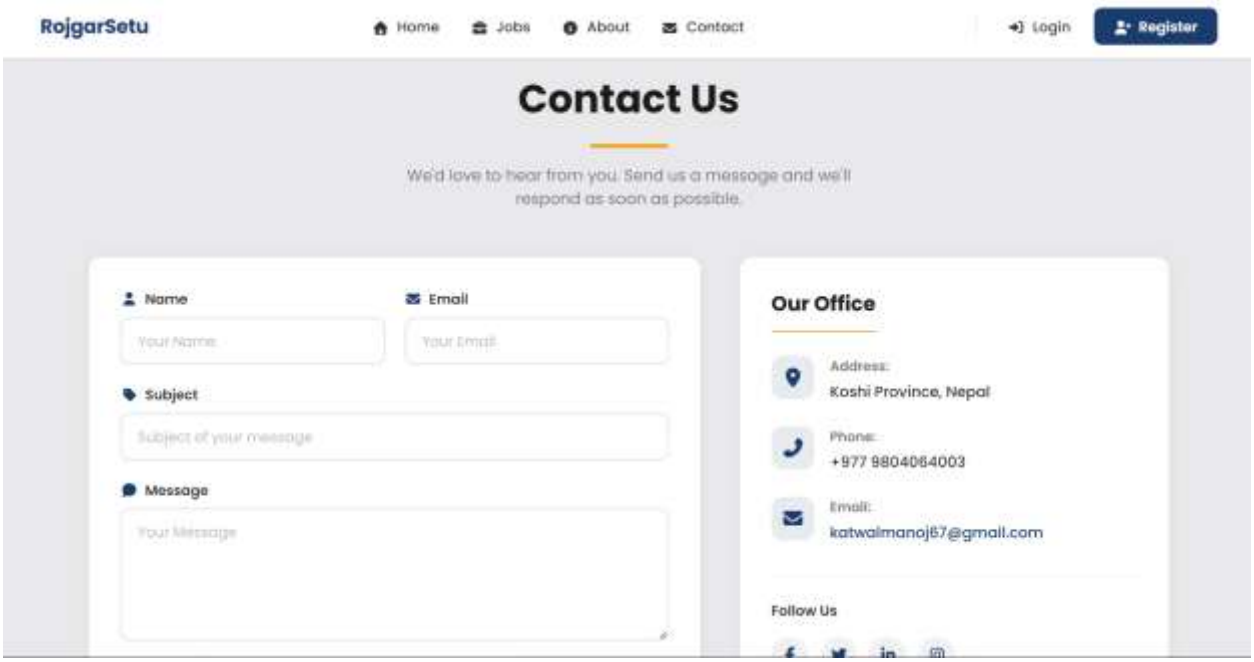
a. Public Pages Development

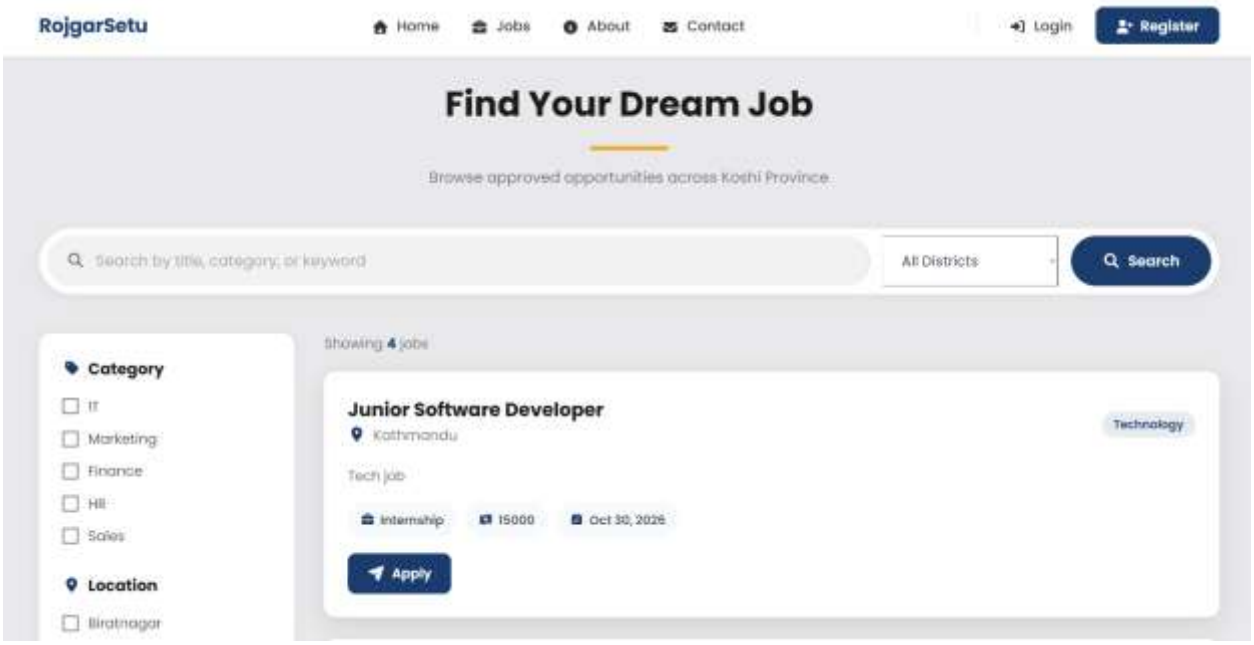
Home, About, Contact and Browse Jobs pages have been created for visitors to view information about the system and some jobs made public.

Evidence:






RojgarSetu
Home
Jobs
About
Contact
Login
Register
Contact Us
We'd love to hear from you. Send us a message and we'll respond as soon as possible.
Name
Email
Subject
Message
Our Office
Address:
Koshi Province, Nepal
Phone:
+977 9804064003
Email:
katwalmanoj67@gmail.com
Follow Us


RojgarSetu
Home
Jobs
About
Contact
Login
Register
Find Your Dream Job
Browse approved opportunities across Koshi Province
Search by title, category, or keyword
All Districts
Search
Category
IT
Marketing
Finance
HR
Sales
Location
Biratnagar
Showing 4 jobs
Junior Software Developer
Kathmandu
Technology
Tech job
Internship
15000
Oct 30, 2026
Apply

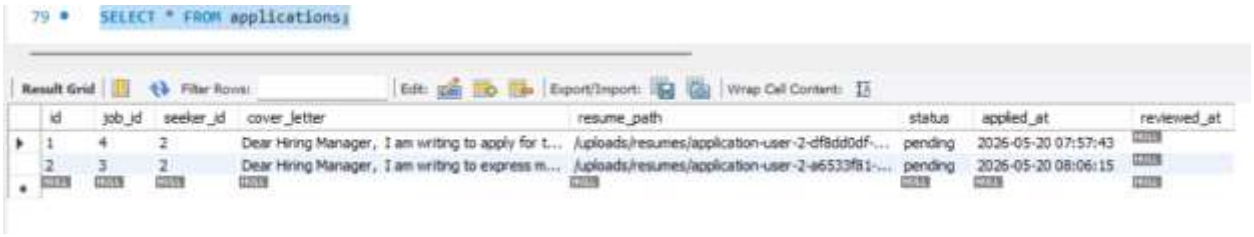
b. System Testing and Validation

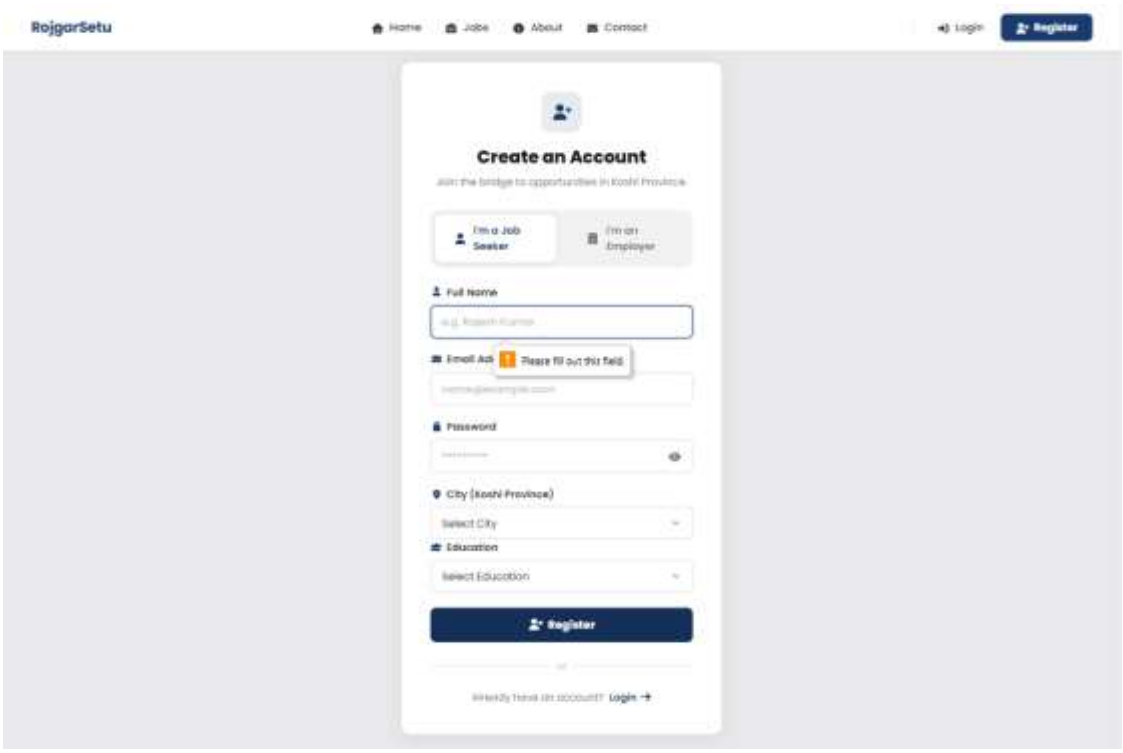
All the significant features of the RojgarSetu system have been tested using the black-box testing methodology.

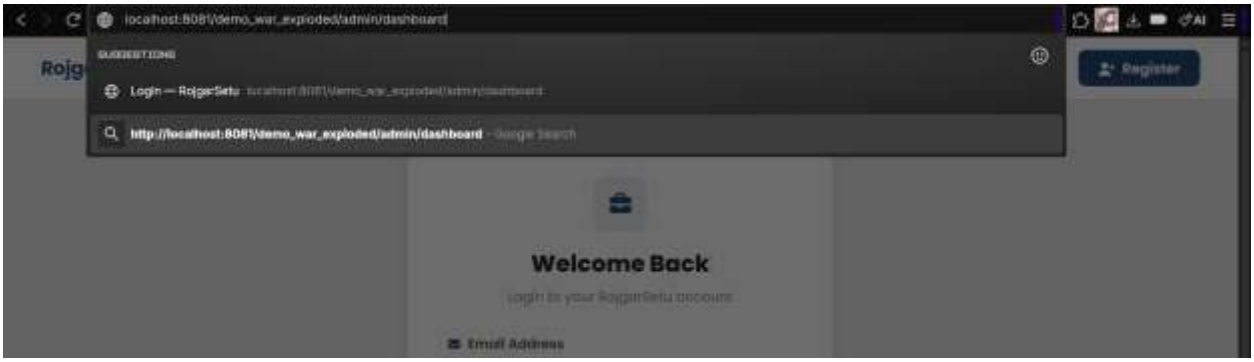
Testing covered:

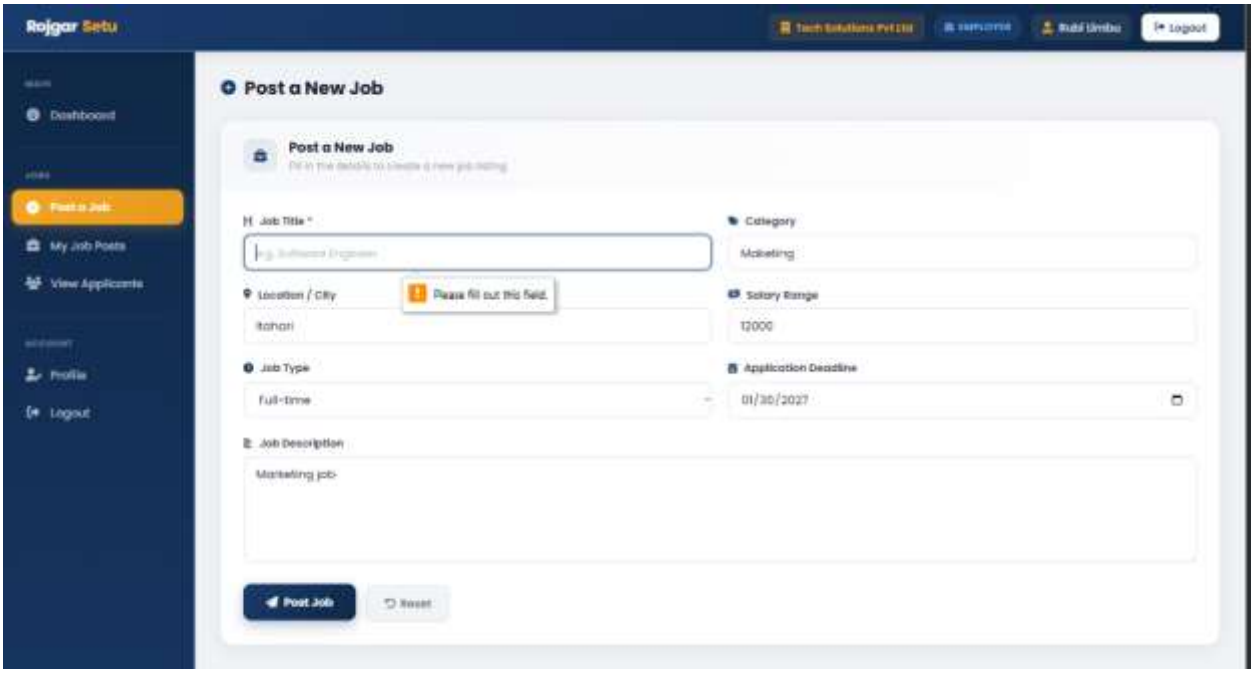
- Registration validation
- Login authentication
- Job posting validation
- Resume upload handling
- Session management
- Security testing
- Reports verification

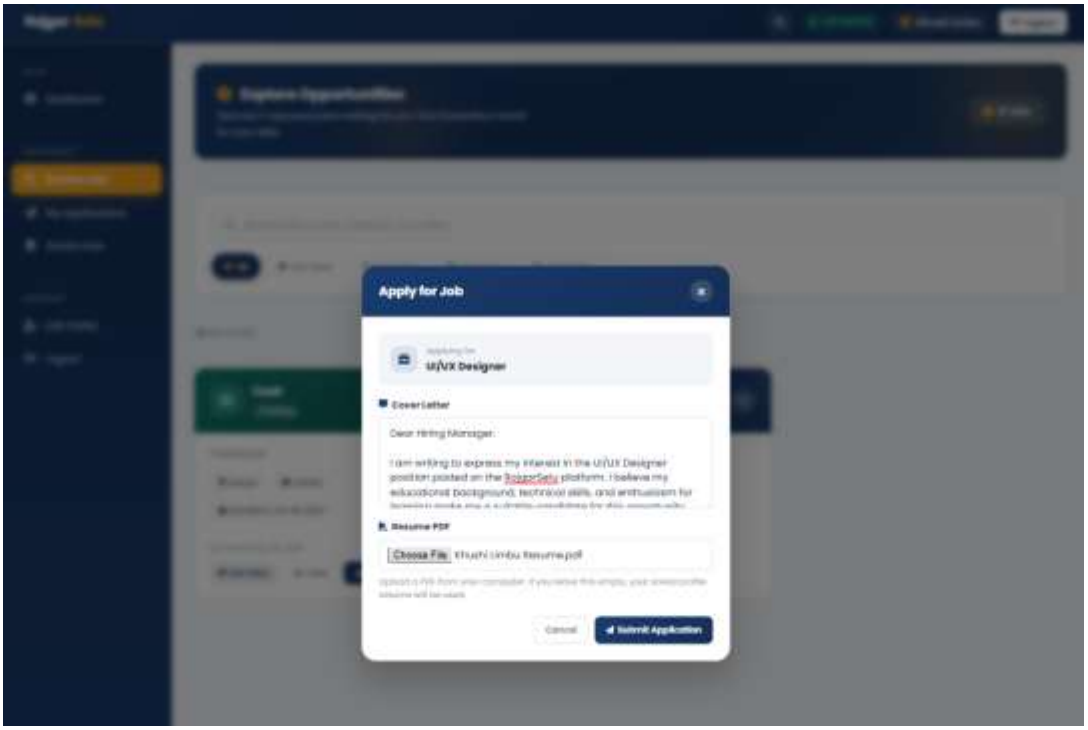
Evidence:




RojgarSetu
Home
Jobs
About
Contact
Login
Register
Create an Account
I'm a Job Seeker
I'm an Employer
Full Name
Email Address
Please fill out this field.
Password
City (Koshi Province)
Select City
Education
Select Education
Register
Already have an account? Login


localhost:8081/demo_war_exploded/admin/dashboard
Login — RojgarSetu
http://localhost:8081/demo_war_exploded/admin/dashboard
Register
Welcome Back
Email Address


Rojgar Setu
Logout
Dashboard
Post a Job
My Job Posts
View Applicants
Profile
Logout
Post a New Job
Post a New Job
Job Title *
Category
Marketing
Location / City
Please fill out this field.
Itahari
Salary Range
12000
Job Type
Full-time
Application Deadline
01/30/2027
Job Description
Marketing job
Post Job
Reset


Apply for Job
UI/UX Designer
Cover Letter
Dear Hiring Manager,
Resume PDF
Choose File
Cancel
Submit Application

c. Critical Analysis

Challenges

Challenge 1: Validation Message Handling

A lot of validation messages and redirections were failing to be displayed properly initially during testing.

Solution:

In order to improve system reliability, more validation checks and exception handling logic were integrated in multiple forms and modules.

Evidence:

```
        HttpSession session = req.getSession();
        if (result) {
            session.setAttribute( s: "success",  o: "Registration Successful! Please login.");
            resp.sendRedirect( location: req.getContextPath() + "/login");
        } else {
            session.setAttribute( s: "error",  o: "Registration Failed!");
            resp.sendRedirect( location: req.getContextPath() + "/register");
        }
```

```
} catch (Exception e) {
    e.printStackTrace();
    resp.sendRedirect( location: req.getContextPath() + "/register?error=2");
}
```

```
try {
    user.setDob(java.sql.Date.valueOf(req.getParameter( s: "dob")));
} catch (Exception e) {
    user.setDob(null);
}
```

10 Conclusion

The system **RojgarSetu** was successfully designed and developed as web-based job portal in Nepal which links job seekers with employers. The aim of this project was to create a more simplified recruitment system by providing an online system where employers can search and post jobs, and job seekers can search and apply for them from an easily accessible location.

Throughout the development of the system, a variety of important concepts of advanced programming and database management were utilized. The system was designed with normalized design using relational database structures, ERDs, and attractive user-friendly UI/UX wireframes, and separate module like user login and registration, post a new job, manage jobs, apply to a job, upload resume, and administrator module with role-based access control were implemented and tested well. The importance of security was also kept in mind. PreparedStatement to prevent SQL injection attacks, password hashing with BCrypt, and a secure session management, along with validation techniques and transactions, helped protect data and system integrity. Test cases were conducted and evaluated to determine how well the system worked, verifying that all major functionalities performed according to specifications.

In essence, RojgarSetu, an advanced web technology and object-oriented programming approach, proved how it can be applied to solve real world problem in employment in Nepal and can create employment opportunities within local environment thus reduce the reliance on informal jobs searching system.

11 References

Geeksforgeeks. (2025) *introduction-of-er-model* [Online]. Available from: https://www.geeksforgeeks.org/dbms/introduction-of-er-model/ [Accessed 21 May 2026].

GeeksforGeeks. (2025) *second-normal-form-2nf* [Online]. Available from: https://www.geeksforgeeks.org/dbms/second-normal-form-2nf/ [Accessed 21 May 2026].

Geeksforgeeks. (2025) *third-normal-form-3nf* [Online]. Available from: https://www.geeksforgeeks.org/dbms/third-normal-form-3nf/ [Accessed 21 May 2026].

Geeksforgeeks. (2025) *unified-modeling-language-uml-class-diagrams* [Online]. Available from: https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/ [Accessed 21 May 2026].

GeeksforGeeks. (2026) *first-normal-form-1nf* [Online]. Available from: https://www.geeksforgeeks.org/dbms/first-normal-form-1nf/ [Accessed 21 May 2026].

GeeksforGeeks. (2026) *introduction-of-database-normalization* [Online]. Available from: https://www.geeksforgeeks.org/dbms/introduction-of-database-normalization/ [Accessed 21 May 2026].

Geeksforgeeks. (n.d.) *first-normal-form-1nf* [Online]. Available from: https://www.geeksforgeeks.org/dbms/first-normal-form-1nf/.